VÉRIFICATION ET VALIDATION DE LOGICIEL

1

INF563 · Master 2 professionnel SIGL · Semestre 1 · 50 h

Evaris Fomekong

CE QUE VOUS SAUREZ FAIRE À LA FIN DU COURS

- Expliquer **pourquoi** un logiciel calcule faux : débordements, trous entre flottants, instabilité
- Modéliser un test : **données, système sous test, oracle, résultat attendu, tolérance, mesure d'erreur**
- Concevoir des tests **systématiques** (boîte noire, boîte blanche) et en mesurer l'efficacité
- Vérifier des calculs par **injection, encadrement, analyse dimensionnelle, perturbation**
- Automatiser et intégrer les tests dans un **pipeline CI/CD**
- Piloter le test : risques, plan, défauts, métriques ; prouver des propriétés critiques

Objectif général

Concevoir, automatiser et piloter une stratégie de V&V complète pour un logiciel professionnel, en **justifiant** chaque choix : technique, oracle, tolérance, outil.

PLAN DU COURS : 50 HEURES, 13 SÉANCES

Séance	Contenu	Durée
1	Introduction à la V&V : enjeux, vocabulaire, principes, défaillances célèbres	4 h
2	Fondements machine : entiers, flottants IEEE 754, sources d'erreur	4 h
3	Anatomie formelle d'un test et problème de l'oracle	4 h
4	Conception en boîte noire : classes d'équivalence, valeurs limites, décisions, états	4 h
5	Conception en boîte blanche, couverture et test de mutation	4 h
6	Techniques de vérification des calculs	4 h
7	**Travaux dirigés** (séances 1 à 6)	4 h
8	**Contrôle continu**	2 h
9	Niveaux de test, doublures, TDD/BDD, automatisation	4 h
10	Analyse statique, revues, qualité, gestion du test	4 h
11	Initiation aux méthodes formelles	4 h
12	Intégration, API, bout en bout, performance	4 h
13	Sécurité, DevSecOps, IA, éthique et synthèse	4 h

SIX FILS ROUGES, UN PAR NATURE DE LOGICIEL

Chaque famille de techniques est illustrée sur le système **de même nature** : on ne teste pas un calcul comme on teste un formulaire.

A Solveur 2nd degré

Calcul numérique. Représentation des nombres, oracles, tolérances, encadrement, perturbation.

B Mobile Money

Règles métier et argent. Frais par tranche, plafonds, BigDecimal, tables de décision, invariants, TDD, mutation.

C Formulaire d'inscription

Saisie et texte. Matricule, nom accentué, téléphone +237, e-mail, date : classes par type, fuzzing, injection.

D GAB (distributeur)

Système à états. Carte, PIN, 3 essais, blocage, retrait : transitions d'états, model checking, contrats.

E Chute libre

Simulation physique. Unités, solution manufacturée, ordre de convergence, encadrement.

F API Scolarité

Application distribuée. API REST de notes + web/mobile : intégration, contrat, E2E, charge, sécurité, CI/CD.

MODALITÉS D'ÉVALUATION

20 %	30 %	50 %
Contrôle continu (séance 8, 2 h)	**Travaux pratiques** (rendus sur les fils rouges)	**Examen final**

- Les TP sont rendus sous forme de **dépôt Git** : code, tests, rapport court, pipeline qui passe
- Chaque rendu est évalué sur la **justification** des choix (classes, oracles, tolérances), pas seulement sur le nombre de tests
- Pédagogie active : on expérimente, on se trompe, on corrige (droit à l'erreur)

Conseil
Commencez les TP dès la séance 2 : chaque rendu réutilise le précédent, et le projet final assemble le tout dans un pipeline.

ENVIRONNEMENT DE TRAVAIL

Besoin	Outils utilisés dans le cours
Langage et build	Java 21, Maven (ou Gradle), Git
Tests unitaires	JUnit 5 (Jupiter), AssertJ, Mockito
Mesure des tests	JaCoCo (couverture), PIT (mutation)
Analyse statique	SpotBugs, PMD, Checkstyle, SonarQube
Intégration / API / E2E	Testcontainers, REST Assured, Pact, Playwright, Appium
Performance / sécurité	k6, JMeter, OWASP ZAP, dépendances (SCA)
CI/CD	GitHub Actions ou GitLab CI
Méthodes formelles	JML / OpenJML, TLA+ (TLC), Frama-C (démonstrations)

INTRODUCTION À LA VÉRIFICATION ET À LA VALIDATION

7

Séance 1 · 4 h · Enjeux, vocabulaire, principes et défaillances célèbres

CHAP I

OBJECTIFS DE LA SÉANCE

- Distinguer **vérification** et **validation** et les situer dans le cycle de vie
- Employer le vocabulaire normalisé : **erreur, défaut, défaillance, oracle, verdict**
- Énoncer et illustrer les **7 principes du test** (ISTQB v4.0)
- Situer les tests : **pyramide**, **quadrants agiles**, statique vs dynamique
- Tirer des **leçons V&V** de défaillances réelles
- Connaître les **6 fils rouges** du cours

Question d'entrée

Un programme qui passe 10 000 tests est-il correct ?

Gardez votre réponse : nous y reviendrons avec le principe n° 1.

POURQUOI LA V&V : CE QUE COÛTE UNE DÉFAILLANCE

1 Des vies

Therac-25 (1985–1987) : surdosages de radiothérapie dus au logiciel, plusieurs patients tués. Patriot (1991) : 28 soldats tués.

2 Des missions

Ariane 5 vol 501 (1996) détruite à 37 s de vol. Mars Climate Orbiter (1999) perdue à l'arrivée sur Mars.

3 L'économie

CrowdStrike (19 juillet 2024) : environ 8,5 millions de postes Windows bloqués, aéroports, hôpitaux et banques à l'arrêt.

4 La confiance

Une application Mobile Money qui débite deux fois ou se trompe de frais perd ses clients en quelques jours.

VÉRIFICATION ET VALIDATION

D

Les deux questions de Boehm (1979)

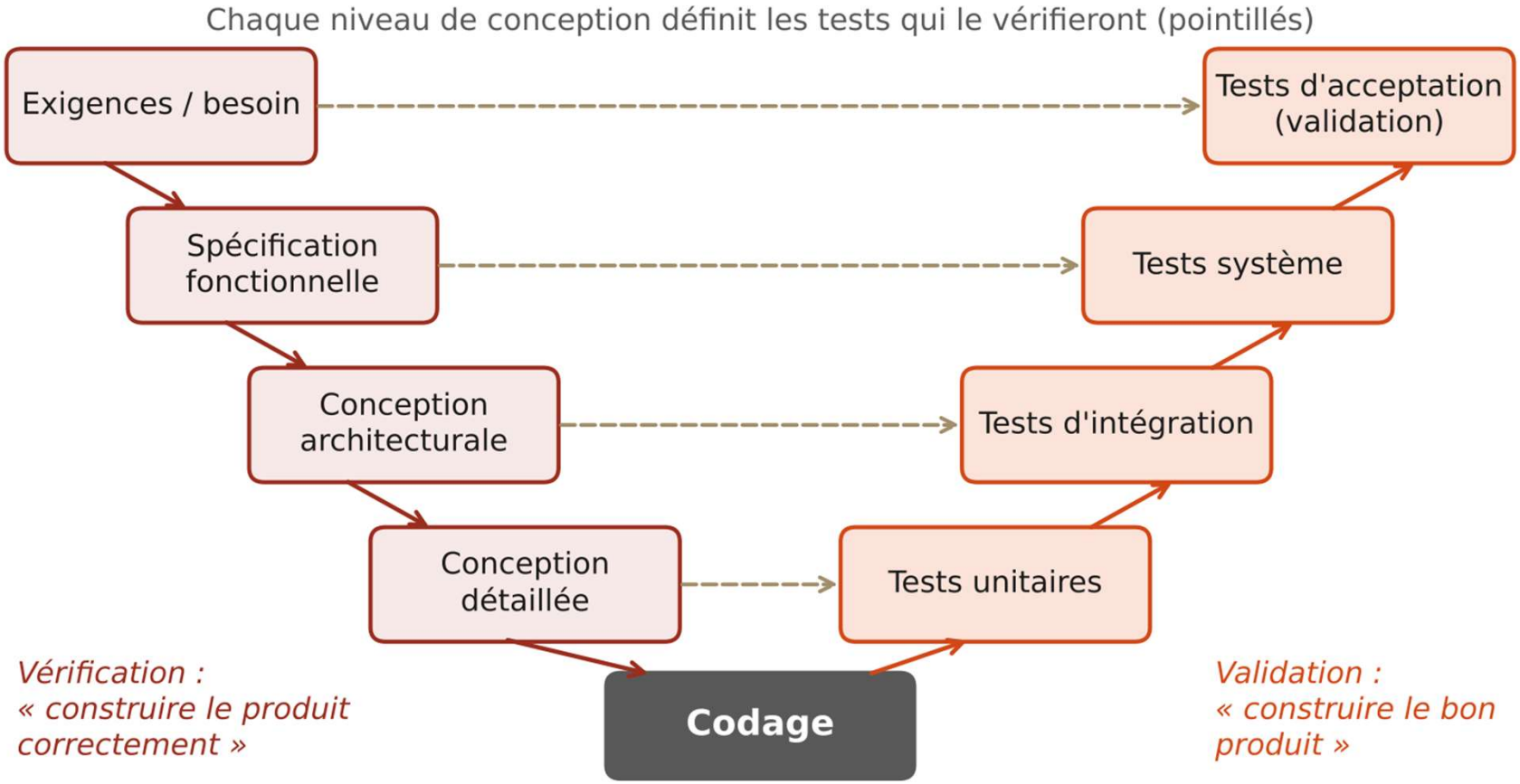
Vérification : « Are we building the product right? », construisons-nous le produit **correctement** (conforme à sa spécification) ?
Validation : « Are we building the right product? », construisons-nous le **bon** produit (conforme au besoin réel) ?

- Un logiciel peut être **vérifié mais non valide** : il respecte parfaitement une spécification qui ne correspond pas au besoin
- Il peut être **valide mais non vérifié** : il rend service, mais rien ne garantit qu'il continuera à le faire
- La V&V vise une **confiance mesurable**, jamais une certitude absolue

VÉRIFICATION VS VALIDATION

	Vérification	Validation
Question	Le produit est-il conforme à sa spécification ?	Le produit répond-il au besoin des utilisateurs ?
Référence	Spécifications, normes, conception	Besoin, usage réel, critères d'acceptation
Activités	Revues, analyse statique, tests unitaires et d'intégration, preuves	Tests système et d'acceptation, bêta, tests d'utilisabilité
Acteurs	Équipe de développement et de test	Clients, utilisateurs, métier
Quand	À chaque étape, dès les exigences	Surtout sur des versions exécutables
Exemple (B)	Les frais de 5 000 FCFA valent 50 FCFA comme spécifié	Les commerçants acceptent-ils cette grille et ce parcours ?

LE CYCLE EN V : À CHAQUE NIVEAU, SES TESTS



Shift-left

Concevoir les tests **au moment** où l'on écrit le niveau correspondant : un test d'acceptation s'écrit avec les exigences, pas après le codage.

LA V&V DANS LES CYCLES MODERNES

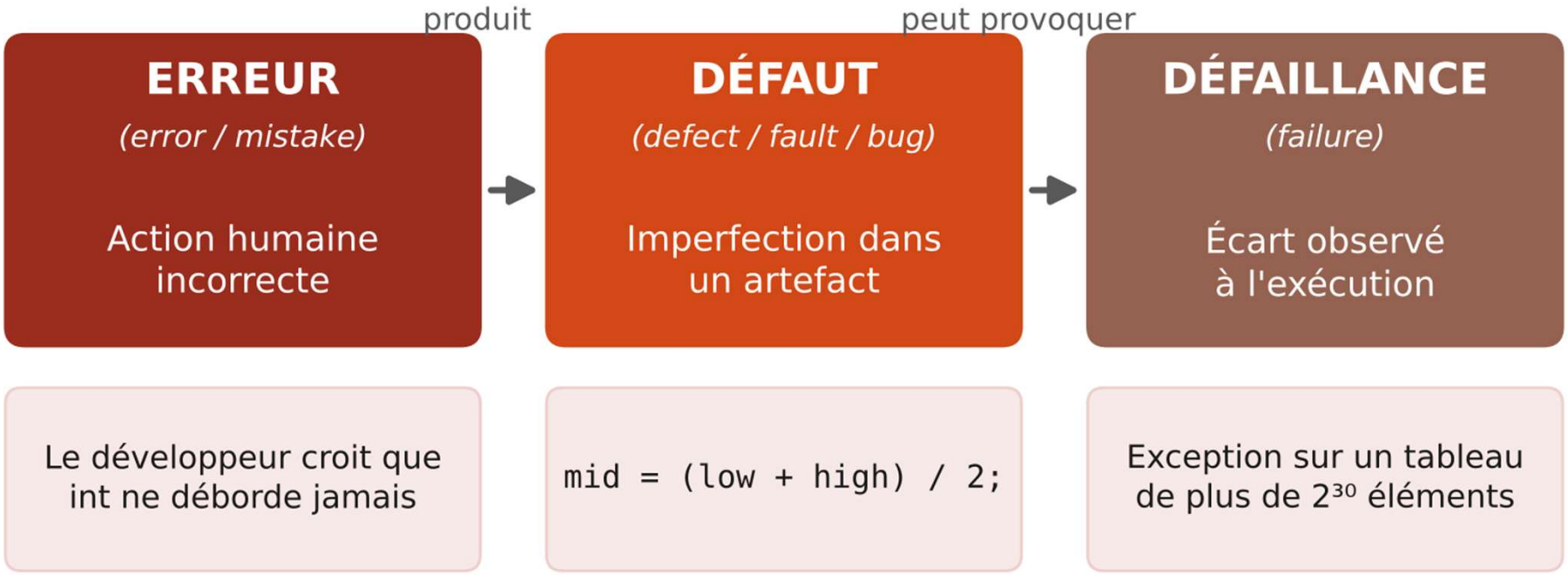
- **Agile :** tests continus à chaque itération, critères d'acceptation dans chaque user story
- **DevOps :** tests automatisés dans le pipeline CI/CD, chaque commit est vérifié
- **Shift-left :** revues des exigences, TDD, analyse statique dès l'écriture
- **Shift-right :** surveillance en production, déploiements progressifs (canary), tests A/B

Ce qui ne change pas

Quel que soit le cycle, il faut savoir **quoi** tester, **comment** décider qu'un résultat est correct (l'oracle) et **quand** s'arrêter (critères de sortie). C'est l'objet de ce cours.

ERREUR, DÉFAUT, DÉFAILLANCE

Vocabulaire ISTQB : une **erreur** humaine introduit un **défaut** dans un artefact ; exécuté dans certaines conditions, le défaut provoque une **défaillance**.



Un défaut peut rester **dormant** : il ne se manifeste que pour certaines entrées. Tester, c'est chercher ces entrées. Toute défaillance n'est pas due à un défaut (environnement, matériel, radiation).

ORACLE ET VERDICT

D

Oracle de test (test oracle)
Source qui permet de déterminer le **résultat attendu** et donc de décider si le résultat observé est correct : spécification, version de référence, propriété, expert humain…

- **Verdict :** résultat de la comparaison : **PASS** (succès) ou **FAIL** (échec), et dans la pratique aussi **ERROR** (le test lui-même n'a pas pu s'exécuter) et **INCONCLUSIF**
- **Faux positif :** le test échoue alors que le logiciel est correct (oracle faux, environnement instable)
- **Faux négatif :** le test passe alors que le logiciel est défaillant : le plus dangereux
- Sans oracle fiable, un test **n'est qu'une exécution**. Ce problème sera formalisé en séance 3

LE VOCABULAIRE MINIMAL DU TESTEUR

Terme (ISTQB)	Définition
Objet de test	Ce que l'on teste : fonction, composant, système, document
Base de test	Ce dont on dérive les tests : exigences, spécification, code, risques
Condition de test	Aspect testable de la base : « les frais pour 5 000 FCFA »
Cas de test	Préconditions, données d'entrée, actions, résultat attendu, postconditions
Couverture	Proportion d'éléments (exigences, branches…) exercés par les tests
Test de confirmation	Ré-exécution qui vérifie qu'un défaut est bien corrigé
Test de régression	Vérifie qu'une modification n'a rien cassé ailleurs
Débogage	Activité de développement : localiser et corriger le défaut (ce n'est pas du test)

LES 7 PRINCIPES DU TEST (ISTQB V4.0)

1 Présence, pas absence

Le test montre la présence de défauts, jamais leur absence.

2 Exhaustivité impossible

On ne peut pas tout tester : il faut choisir, par les risques et les techniques.

3 Tester tôt

Tester tôt fait gagner du temps et de l'argent.

4 Grappes

Les défauts se concentrent dans un petit nombre de composants.

5 Usure des tests

Les mêmes tests répétés finissent par ne plus rien trouver (paradoxe du pesticide).

6 Contexte

Le test dépend du contexte : on ne teste pas un GAB comme un site vitrine.

7 Absence d'erreurs

Un logiciel sans défaut connu peut être inutile : vérifier ne suffit pas, il faut valider.

PRINCIPE 2 : L'EXHAUSTIVITÉ EST IMPOSSIBLE

Une simple fonction `f(int a, int b)` en Java :

2^{64}

couples d'entrées possibles (≈ 1,8 × 10^{19})

10^{9} / s

tests exécutés par seconde (hypothèse très optimiste)

≈ 585 ans

pour tout tester, sans compter l'état ni l'ordre des appels

- Il faut donc **choisir** les entrées : classes d'équivalence, valeurs limites, combinatoire (séance 4)
- Et **mesurer** ce que l'on a couvert : couverture, mutation (séance 5)
- Ou **prouver** pour toutes les entrées à la fois : méthodes formelles (séance 11)

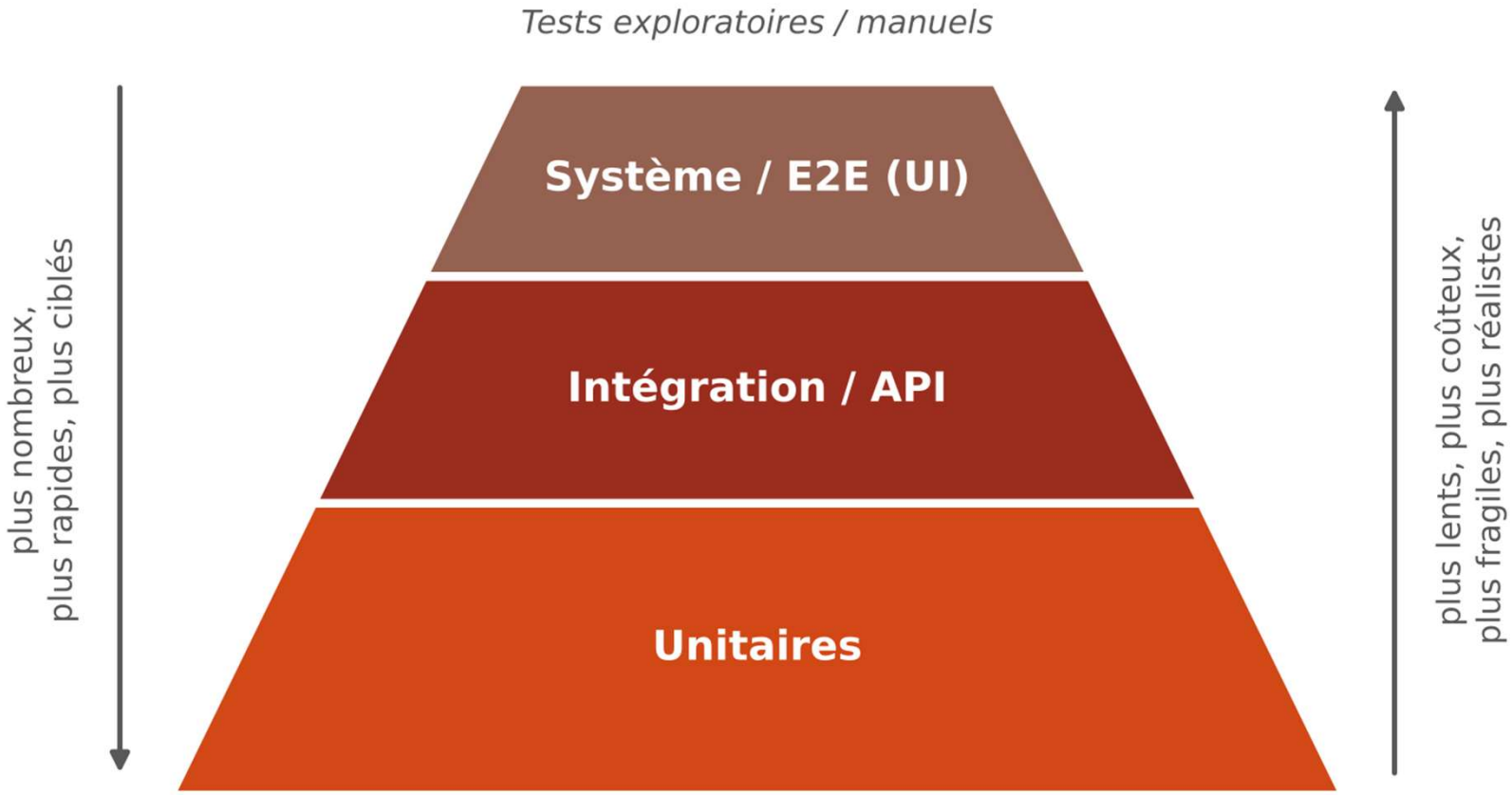
TEST STATIQUE ET TEST DYNAMIQUE

	Statique	Dynamique
Principe	Examiner sans exécuter	Exécuter et observer
Révèle	Des **défauts** directement	Des **défaillances**, puis on cherche le défaut
Techniques	Revues, inspections, analyse statique, preuve formelle	Tests unitaires à acceptation, profilage, fuzzing
Objets	Exigences, modèles, code, configuration	Logiciel exécutable
Moment	Dès qu'un artefact existe	Dès qu'une partie s'exécute
Séances	10 (statique), 11 (formel)	2 à 9, 12, 13

Complémentaires

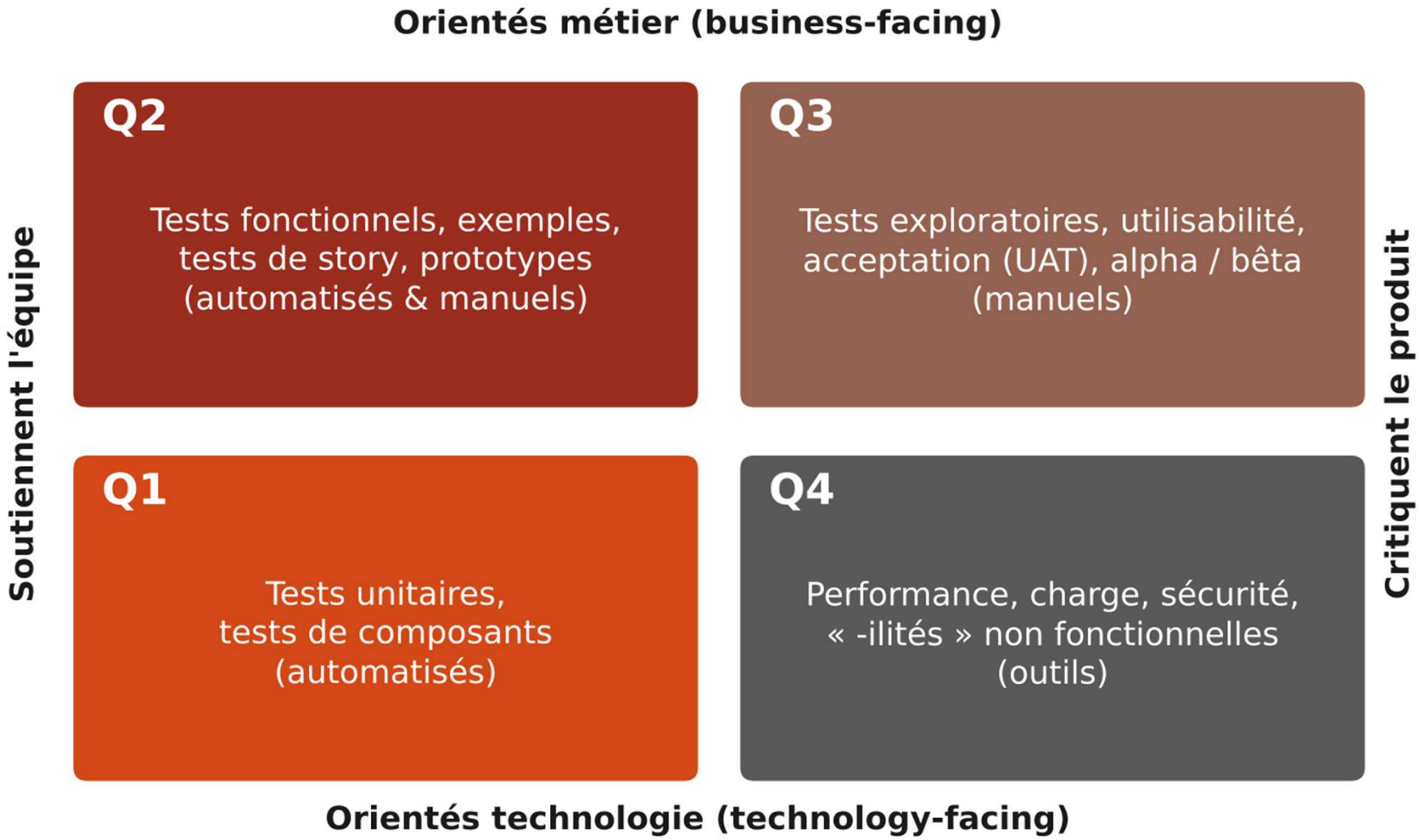
Un analyseur statique signale un débordement **possible** sur `low + high` ; un test dynamique avec un grand tableau le **confirme**. Le premier est exhaustif mais imprécis, le second précis mais partiel.

LA PYRAMIDE DES TESTS



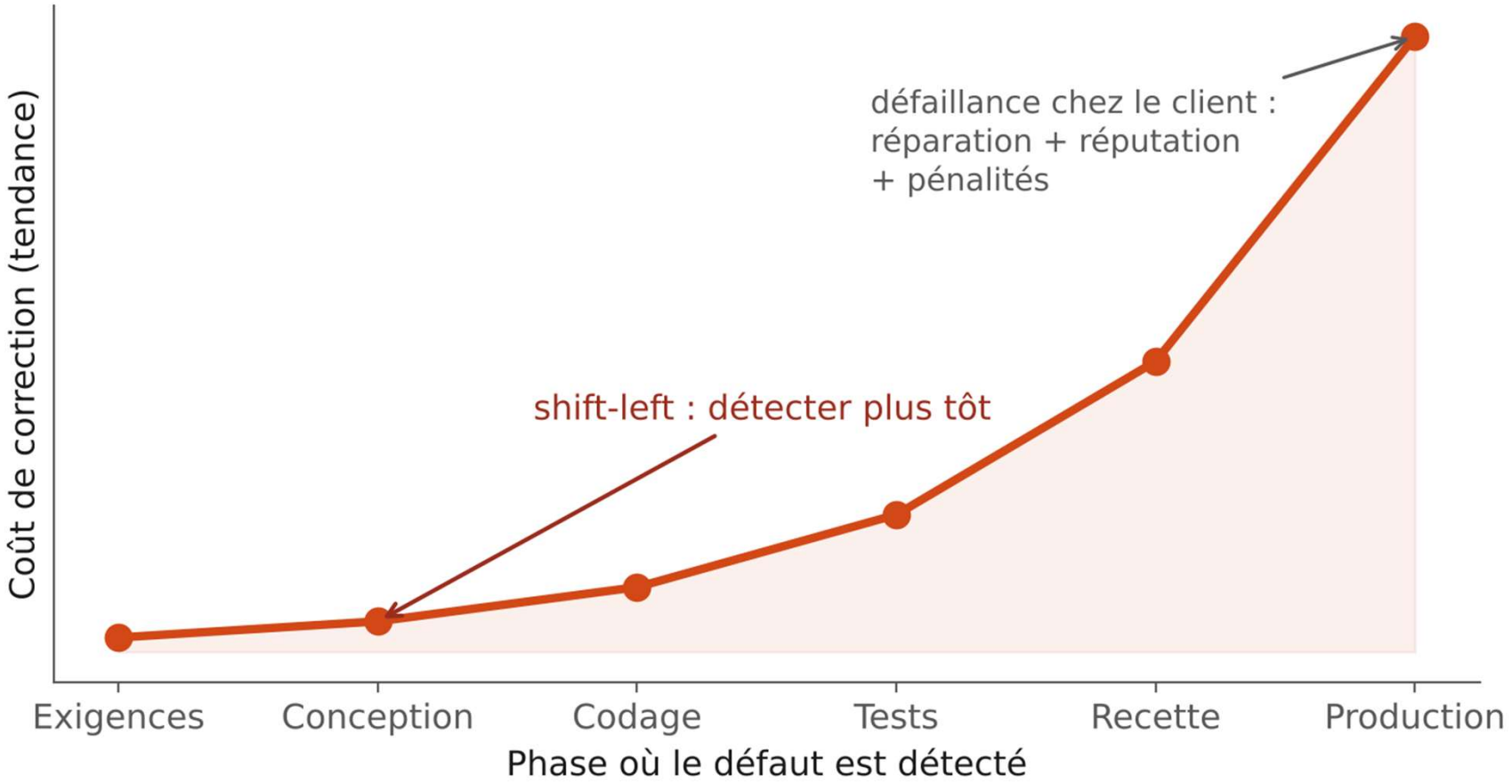
Mike Cohn, Succeeding with Agile (2009)
Beaucoup de tests unitaires rapides, peu de tests de bout en bout lents et fragiles. Anti-patron : le « cornet de glace », où tout repose sur des tests manuels ou UI. C'est un ordre de grandeur, pas une règle.

LES QUADRANTS DU TEST AGILE



D'après B. Marick (2003), repris par L. Crispin et J. Gregory, Agile Testing (2009)

PLUS UN DÉFAUT EST TROUVÉ TARD, PLUS IL COÛTE



Une tendance, pas une loi

Les multiplicateurs souvent cités (« ×100 en production ») proviennent d'études anciennes, mal reproduites. La tendance, elle, est solide.

ARIANE 5, VOL 501 (4 JUIN 1996)

CAS RÉEL

Contexte
Premier vol d'Ariane 5. Le logiciel de la plateforme inertielle (SRI) est **réutilisé d'Ariane 4**.

Défaillance
Vers 37 s de vol, les deux SRI s'arrêtent ; des données de diagnostic sont lues comme des données de vol ; braquage des tuyères, destruction du lanceur.

Cause technique
Conversion d'un flottant 64 bits (biais horizontal) en **entier signé 16 bits** : la valeur, plus grande sur la trajectoire d'Ariane 5, déborde ; exception non traitée. Le SRI de secours, identique, échoue de la même façon.

LEÇON V&V

Une hypothèse de domaine valide sur Ariane 4 n'a **pas été revérifiée** pour Ariane 5.

La redondance identique ne protège pas d'un défaut logiciel.

Les plages de valeurs et les conversions sont des valeurs limites à tester.

THERAC-25 (1985–1987)

CAS RÉEL

Contexte
Appareil de radiothérapie (AECL) où les **verrous de sécurité matériels** du modèle précédent ont été remplacés par du logiciel.

Défaillance
Au moins six accidents de surdosage massif ; plusieurs patients en sont morts.

Cause technique
Conditions de course (race conditions) lors d'une saisie rapide par l'opérateur, et un **compteur sur un octet** qui revient à 0 et fait sauter une vérification. Messages d'erreur cryptiques (« Malfunction 54 ») que l'on contournait.

LEÇON V&V

Du code réutilisé « éprouvé » ne l'était que **grâce aux verrous matériels**.

Les systèmes critiques exigent une analyse de sûreté indépendante, des tests de concurrence et des tests de débordement.

MISSILE PATRIOT, DHAHRAN (25 FÉVRIER 1991)

CAS RÉEL

Contexte
Une batterie Patriot fonctionne **sans redémarrage depuis environ 100 heures**.

Défaillance
Un missile Scud n'est pas intercepté et frappe une caserne : 28 soldats américains tués.

Cause technique
Le temps, compté en dixièmes de seconde, est multiplié par 0,1 stocké sur **24 bits** : 0,1 n'est pas représentable en binaire. L'erreur s'accumule : **0,34 s** après 100 h, soit plus de 500 m de décalage sur la cible.

LEÇON V&V

Une erreur d'arrondi minuscule devient fatale quand elle **s'accumule**.

Tester les scénarios de longue durée et calculer l'erreur propagée : voir séance 2.

MARS CLIMATE ORBITER (23 SEPTEMBRE 1999)

CAS RÉEL

Contexte
Sonde de la NASA en approche de Mars ; les corrections de trajectoire sont calculées au sol par un logiciel d'un sous-traitant.

Défaillance
La sonde passe beaucoup trop bas (≈ 57 km) lors de l'insertion en orbite et est perdue.

Cause technique
Le logiciel au sol produit des impulsions en **livre-force·seconde** alors que la spécification d'interface exige des **newton·seconde** (facteur ≈ 4,45).

LEÇON V&V

Défaut d'**interface** : chaque côté était « correct » isolément.

Vérifier les interfaces et les unités : analyse dimensionnelle, typage des grandeurs (séance 6), tests d'intégration (séance 12).

CROWDSTRIKE (19 JUILLET 2024)

CAS RÉEL

Contexte
Mise à jour de **contenu** (fichier de configuration « channel file 291 ») du capteur de sécurité Falcon, poussée à tous les clients Windows.

Défaillance
Écran bleu et boucle de redémarrage sur environ **8,5 millions** de machines : compagnies aériennes, hôpitaux, banques, médias.

Cause technique
Le modèle attendait 21 champs, le capteur n'en fournissait que 20 : **lecture mémoire hors limites**. Le validateur de contenu, lui-même défectueux, a laissé passer le fichier ; pas de déploiement progressif.

LEÇON V&V

La **configuration est du code** : elle se teste.

Tester le validateur (qui teste l'oracle ?), vérifier les bornes, déployer progressivement (canary).

LES SIX FILS ROUGES DU COURS

Fil rouge	Ce qu'il permet d'étudier	Séances principales
A · Solveur 2nd degré	Représentation des nombres, oracles, tolérance, encadrement	2, 3, 6, 10
B · Mobile Money	Règles métier, argent (BigDecimal), tables de décision, invariants, TDD, mutation	2, 4, 5, 6, 9, 11
C · Formulaire d'inscription	Classes d'équivalence sur texte et dates, fuzzing, injection	4, 7, 13
D · GAB (distributeur)	Transitions d'états, contrats, model checking	4, 5, 11
E · Chute libre	Unités, solution manufacturée, convergence	6
F · API Scolarité	Intégration, contrats d'API, E2E, charge, sécurité, CI/CD	4, 6, 12, 13

Pourquoi plusieurs fils rouges ?

On ne teste pas un calcul comme un formulaire ni un automate comme une API : chaque technique est montrée sur le système **dont la nature lui correspond**. La pastille en bas à gauche des slides indique le fil rouge utilisé.

À RETENIR

À RETENIR

- **Vérifier** : le produit est-il conforme à sa spécification ? **Valider** : est-ce le bon produit ?
- Erreur (humaine) → **défaut** (dans l'artefact) → **défaillance** (à l'exécution)
- Un test sans **oracle** n'est qu'une exécution ; un verdict peut être faux (faux positif, faux négatif)
- Tester ne prouve pas l'absence de défauts et ne peut pas être exhaustif : il faut **choisir** et **mesurer**
- Tester **tôt** (shift-left) et à tous les niveaux (pyramide, quadrants)
- Les grandes défaillances viennent souvent d'**hypothèses non revérifiées** : débordements, unités, arrondis, interfaces

FONDEMENTS MACHINE : REPRÉSENTATION DES NOMBRES

30

Séance 2 · 4 h · Entiers, flottants IEEE 754 et sources d'erreur de calcul

CHAP II

OBJECTIFS DE LA SÉANCE

- Expliquer le **complément à deux** et prévoir un **débordement** d'entier
- Décrire le format **IEEE 754** et ses valeurs spéciales
- Calculer un **ULP** et expliquer les **trous** entre flottants
- Reconnaître **absorption**, **élimination catastrophique**, non-associativité
- Distinguer **conditionnement** d'un problème et **stabilité** d'un algorithme
- En déduire des **valeurs de test** et des **tolérances** justifiées

Fil rouge du TP

Le solveur d'équation du second degré (fil rouge A) : une formule de lycée, trois façons de la faire échouer.

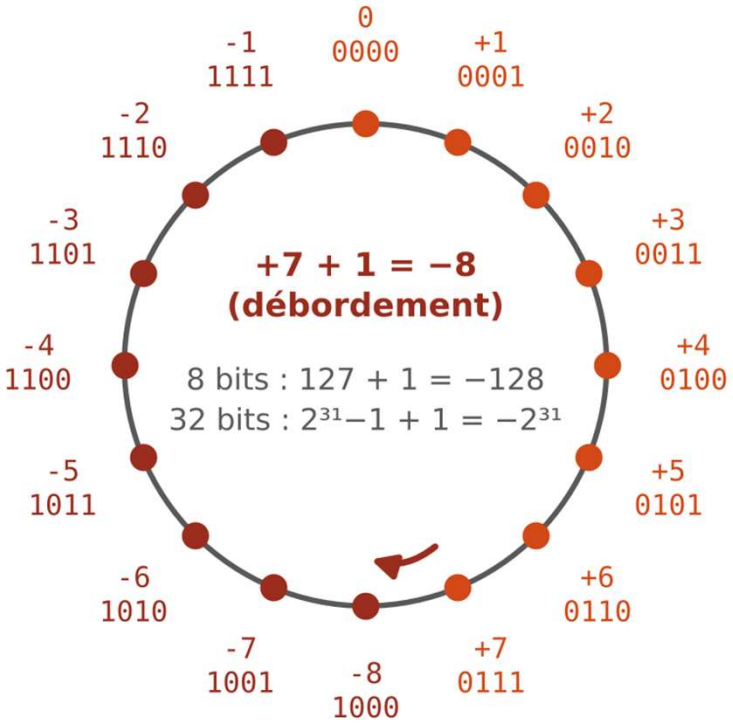
POURQUOI UN TESTEUR DOIT CONNAÎTRE LA MACHINE

- `int` n'est pas $\mathbb{Z}$: **2^{32} valeurs**, puis on revient au début
- `double` n'est pas $\mathbb{R}$: **2^{64} motifs**, l'immense majorité des réels n'est pas représentable
- Chaque opération flottante **arrondit** : le résultat exact n'existe pas en machine
- Ces écarts sont **silencieux** : pas d'exception, pas de message

Conséquence pour la V&V

Un oracle « résultat == attendu » est souvent **faux par construction**. Il faut une **tolérance** et une **mesure d'erreur** justifiées (séance 3), et des **valeurs de test** qui visent les limites de représentation.

LES ENTIERS EN COMPLÉMENT À DEUX



- Sur n bits : de -2^{n-1} à $2^{n-1} - 1$
- Opposé : `-x = ~x + 1` (inverser les bits, ajouter 1)
- L'addition se fait **modulo 2^n** : la roue tourne et repasse par le minimum
- Asymétrie : il y a **un négatif de plus** que de positifs, donc `-MIN` n'existe pas
- Java : `byte`, `short`, `int`, `long` sont signés ; `char` est non signé (16 bits)

BORNES DES TYPES PRIMITIFS JAVA

Type	Bits	Minimum	Maximum
byte	8	−128	127
short	16	−32 768	32 767
char	16	0 ('\u0000')	65 535 ('\uFFFF')
int	32	−2 147 483 648 (-2^{31})	2 147 483 647 ($2^{31} - 1$)
long	64	-2^{63}	9 223 372 036 854 775 807 ($2^{63} - 1$)
float	32	$\pm 1{,}4 \times 10^{-45}$ (plus petit > 0)	$\pm 3{,}4028235 \times 10^{38}$
double	64	$\pm 4{,}9 \times 10^{-324}$ (plus petit > 0)	$\pm 1{,}7976931348623157 \times 10^{308}$

Valeurs limites naturelles

Chaque borne, et chaque valeur voisine (MIN + 1, MAX − 1), est un candidat de test. Pour les flottants, « minimum » désigne la plus petite valeur positive, pas la plus négative.

LE DÉBORDEMENT EST SILENCIEUX

Pieges.java

```java
System.out.println(Integer.MAX_VALUE + 1);
// -2147483648

System.out.println(Math.abs(Integer.MIN_VALUE));
// -2147483648   (une valeur absolue négative !)

System.out.println((byte) 200);
// -56

Math.addExact(Integer.MAX_VALUE, 1);
// ArithmeticException: integer overflow
```

- Aucune exception par défaut : le résultat est **faux mais plausible**
- `Math.abs` peut renvoyer un **négatif**
- Le transtypage (cast) **tronque** les bits de poids fort
- `Math.addExact`, `multiplyExact`, `toIntExact` : **échouer bruyamment** plutôt que calculer faux

LE BUG DU MILIEU : DES ANNÉES DANS LE JDK

```java
int low = 1_500_000_000, high = 2_000_000_000;

int mid1 = (low + high) / 2;         // -397483648
int mid2 = low + (high - low) / 2;   // 1750000000
int mid3 = (low + high) >>> 1;       // 1750000000
```

- J. Bloch (2006), « Nearly All Binary Searches and Mergesorts are Broken »
- Défaut présent dans `Arrays.binarySearch` depuis 9 ans, et dans l'algorithme publié depuis des décennies
- Se manifeste seulement pour des tableaux de **plus de 2^{30} éléments**
- Le code avait été **prouvé** et testé… sur des tableaux plus petits

Leçon : une preuve ou un test n'est valable que dans **son modèle** (ici, des entiers mathématiques). Les tests doivent viser les grandes valeurs.

DÉBORDEMENTS DANS LA NATURE

1 Boeing 787 (2015)

Un compteur 32 bits signé en centièmes de seconde déborde après 2^{31} / 100 s ≈ **248 jours** : les unités de contrôle de l'alimentation se coupent. Consigne FAA : redémarrer avant.

2 An 2038

Un `time_t` signé 32 bits compte les secondes depuis 1970. Après **2038-01-19T03:14:07Z**, il repasse en **1901-12-13**.

3 Therac-25 (1985–1987)

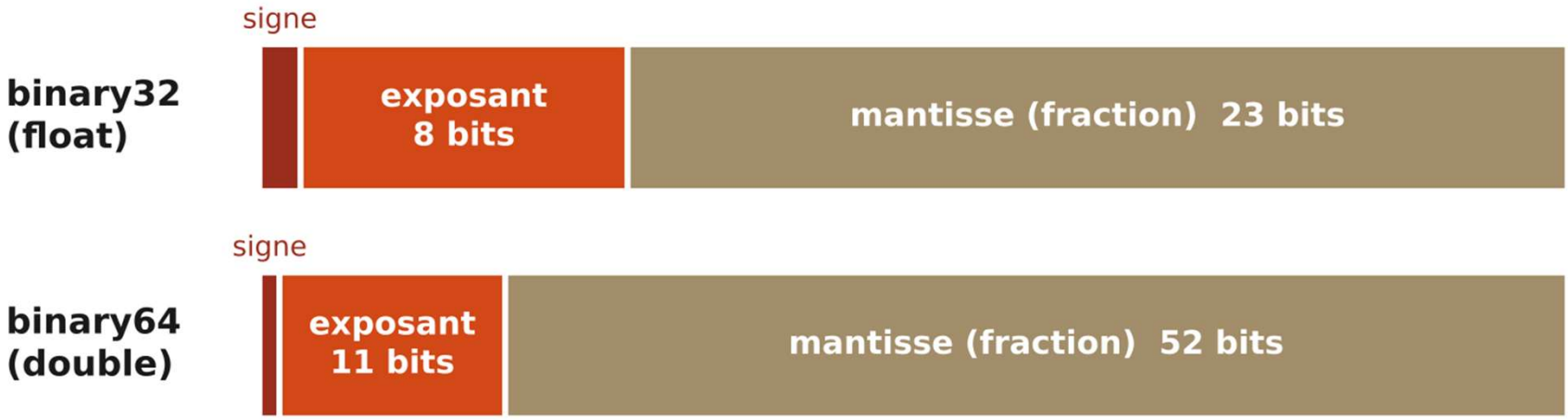
Un compteur sur **un octet** revient à 0 et une vérification de sécurité est sautée.

4 YouTube (2014)

Le compteur de vues de « Gangnam Style » approche $\mathbf{2^{31} - 1}$; YouTube passe le compteur en 64 bits.

LES FLOTTANTS IEEE 754

Norme IEEE 754 (1985, révisée en 2008 et 2019) : un nombre est un **signe**, un **exposant** biaisé et une **mantisse** avec un 1 implicite.



$$\text{valeur} = (-1)^S \times 1,\text{fraction}_2 \times 2^{(\text{exposant} - \text{biais})} \qquad \text{biais} = 127 \text{ (float)}, 1023 \text{ (double)}$$

Précision : 24 bits significatifs en float (≈ 7 chiffres décimaux), **53 bits** en double (≈ 15 à 16 chiffres). Au-delà, les chiffres affichés ne sont que du bruit.

LES VALEURS SPÉCIALES

Valeur	Comment on l'obtient	Piège pour le test
+0.0 et −0.0	`-1e-320 * 1e-10`, `-0.0`	`0.0 == -0.0` est vrai, mais `1/0.0` = +∞ et `1/-0.0` = −∞
+∞, −∞	`1/0.0`, débordement de `Double.MAX_VALUE * 2`	Se propage sans erreur ; ∞ − ∞ = NaN
NaN	`0.0/0.0`, `Math.sqrt(-1)`, ∞ − ∞	`NaN == NaN` est **faux** ; toute comparaison renvoie faux
Sous-normaux	Entre `Double.MIN_VALUE` ($4{,}9 \times 10^{-324}$) et `MIN_NORMAL` ($2{,}2 \times 10^{-308}$)	Perte progressive de précision, calculs parfois très lents
MAX_VALUE	$1{,}7976931348623157 \times 10^{308}$	`MAX_VALUE * 2` = +∞ sans exception

Java
`Double.valueOf(NaN).equals(NaN)` est **vrai** et `Double.compare(0.0, -0.0)` vaut 1 : `==` et `equals` ne suivent pas les mêmes règles.

ULP ET EPSILON MACHINE

D

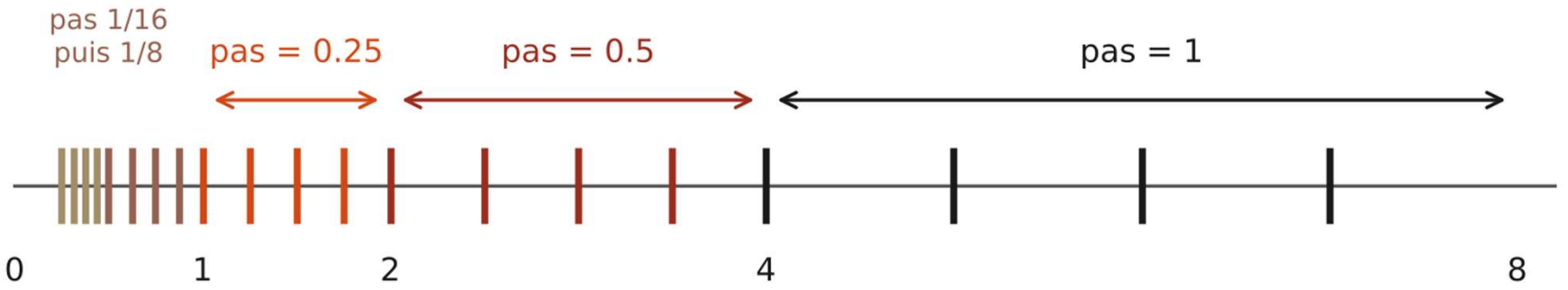
ULP (unit in the last place)

`ulp(x)` est l'écart entre x et le flottant suivant. L'**epsilon machine** est $\varepsilon = \text{ulp}(1) = 2^{-52} \approx 2{,}22 \times 10^{-16}$ en double ($2^{-23} \approx 1{,}19 \times 10^{-7}$ en float).

- Avec l'arrondi au plus proche, chaque opération $+ \ - \ \times \ \div \ \sqrt{}$ donne le flottant le plus proche du résultat exact : **erreur relative ≤ ε/2**
- L'erreur d'**une** opération est minuscule ; c'est l'**accumulation** et l'**amplification** qui posent problème
- Java : `Math.ulp(x)`, `Math.nextUp(x)`, `Math.nextDown(x)`

DES TROUS QUI DOUBLENT À CHAQUE PUISSANCE DE 2

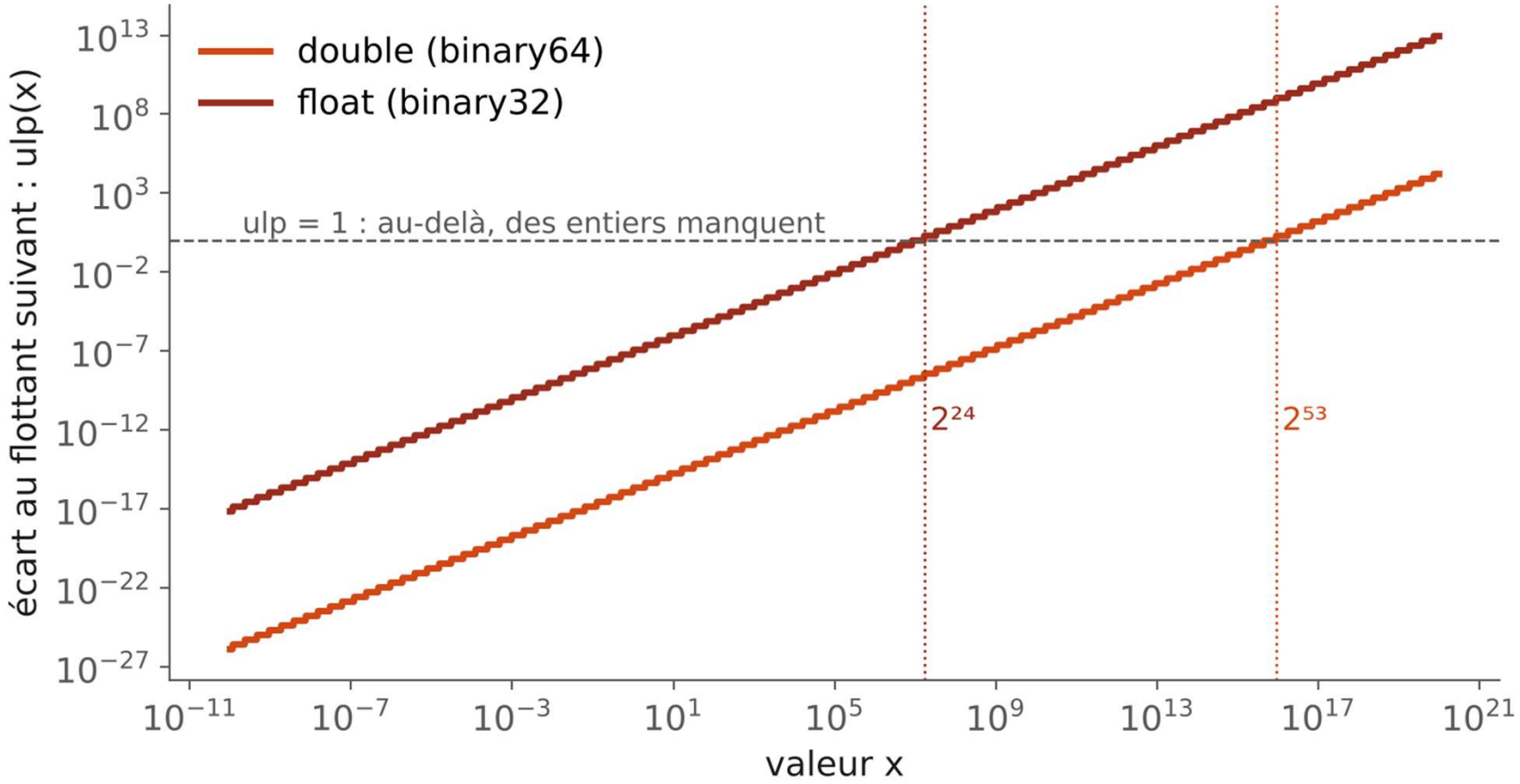
Entre deux puissances de 2 consécutives (une **binade**), les flottants sont régulièrement espacés ; d'une binade à la suivante, l'**écart double**.



Système jouet : 1,ff₂ × 2^e (2 bits de fraction, e ∈ [−2, 2]) — 4 flottants par binade, écarts qui doublent

Conséquence
La précision est **relative** : on a autant de flottants entre 1 et 2 qu'entre 1 000 000 et 2 000 000. L'erreur absolue, elle, grandit avec la valeur.

L'ÉCART ENTRE FLOTTANTS VOISINS



En double : ulp(1) ≈ 2,2 × 10^{-16}, ulp(10^{16}) = **2**, ulp(10^{308}) ≈ 2 × 10^{292}. Au-delà de **2^{53} ≈ 9 × 10^{15}** (**2^{24} = 16 777 216** en float), des entiers manquent : `(double) 9007199254740993L` vaut 9007199254740992.

LES PIÈGES CLASSIQUES EN UNE PAGE

```
double s = 0;
for (int i = 0; i < 10; i++) s += 0.1;

0.1 + 0.2               // 0.30000000000000004
0.1 + 0.2 == 0.3        // false
s                       // 0.9999999999999999
1e16 + 1 == 1e16        // true   (absorption)
(0.1 + 0.2) + 0.3       // 0.6000000000000001
0.1 + (0.2 + 0.3)       // 0.6    (non associatif)
16777217f == 16777216f // true   (au-delà de 2^24)
Double.NaN == Double.NaN  // false
```

Pourquoi

0,1 = 0,000110011…$_2$ est **infini** en binaire. Un petit terme ajouté à un grand **disparaît** (absorption). L'**ordre** des opérations change le résultat : une réduction parallèle peut varier d'une exécution à l'autre. Donc : jamais == entre flottants calculés.

ÉLIMINATION CATASTROPHIQUE

- Soustraire deux nombres **proches** annule les chiffres significatifs communs
- Les erreurs d'arrondi déjà présentes deviennent **dominantes**
- Exemple : 1,000 000 3 − 1,000 000 1 = 0,000 000 2 ; si chaque opérande porte une erreur de 10^{-7}, le résultat peut être faux à **50 %**
- La soustraction elle-même est exacte : c'est l'**information perdue avant** qui est révélée

Où la chercher

`-b + √(b² - 4ac)` quand $b^2 \gg 4ac$

`1 - cos(x)` pour x petit

`√(x+1) - √x` pour x grand

`(a + b) - a` quand $|b| \ll |a|$

Remède : **réécrire** la formule pour éviter la soustraction (formule conjuguée, identités).

COMPARER DES FLOTTANTS DANS UN TEST

```java
// Faux : échoue pour 0.1 + 0.2
assertEquals(0.3, 0.1 + 0.2, 0.0);

// Tolérance absolue (JUnit 5)
assertEquals(0.3, 0.1 + 0.2, 1e-12);

// Tolérance relative
double att = 1e20, obs = calcule();
assertTrue(Math.abs(obs - att)
           <= 1e-12 * Math.abs(att));

// En ULP
assertTrue(Math.abs(obs - att)
           <= 4 * Math.ulp(att));
```

- Une tolérance **absolue** fixe n'a pas de sens pour toutes les échelles : 10^{-12} est énorme près de 10^{-15} et infime près de 10^{20}
- Une tolérance **relative** échoue près de 0
- Une tolérance se **justifie** : précision machine × conditionnement × nombre d'opérations
- Cas spéciaux à traiter à part : **NaN, ±∞, ±0**

CONDITIONNEMENT ET STABILITÉ

D

Deux questions différentes

Conditionnement (propriété du **problème**) : une petite perturbation des données change-t-elle beaucoup la solution exacte ?

Stabilité (propriété de l'**algorithme**) : l'algorithme ajoute-t-il plus d'erreur que ce que le conditionnement impose ?

Équation (exécution Java)	Racines calculées	Diagnostic
$x^2 - 2x + 1 = 0$	[1.0]	Racine double
$c = 1 - 1{,}1 \times 10^{-16}$	[0.99999998946, 1.00000001054]	c bouge de 10^{-16}, les racines de 10^{-8} : **mal conditionné**
$x^2 + 10^8 x + 1 = 0$	naïf : $-7{,}45 \times 10^{-9}$; stable : $-1{,}0 \times 10^{-8}$	Bien conditionné, formule naïve **instable**

PATRIOT : LE CALCUL DE L'ERREUR

CAS RÉEL

$9{,}5 \cdot 10^{-8}$ s	$3{,}6 \cdot 10^{6}$	0,34 s	> 500 m
erreur par tic : 0,1 tronqué sur 24 bits	tics de 0,1 s en 100 heures	erreur accumulée sur l'horloge	décalage pour un Scud à ≈ 1 676 m/s

- Une erreur de représentation **constante** × un **grand nombre** d'opérations = une erreur macroscopique
- Test manquant : un scénario de **longue durée** (le système avait été conçu pour des déploiements courts)

BOURSE DE VANCOUVER (1982–1983)

CAS RÉEL

1 000	≈ 524	≈ 1 099
valeur de départ de l'indice (janvier 1982)	valeur affichée 22 mois plus tard	valeur correcte après recalcul

- L'indice était recalculé à chaque transaction et **tronqué** à 3 décimales (au lieu d'être arrondi)
- Chaque troncature perd en moyenne une demi-unité de la dernière décimale, **toujours dans le même sens**
- Leçon : le **mode d'arrondi** fait partie de la spécification et doit être testé

L'ARGENT NE SE CALCULE PAS EN DOUBLE

Argent.java

```java
new BigDecimal(0.1)
// 0.1000000000000000055511151231257827...
new BigDecimal("0.1").add(new BigDecimal("0.2"))
// 0.3

// 1 % de frais sur 5 001 FCFA
BigDecimal f = new BigDecimal("5001")
                .multiply(new BigDecimal("0.01"));
// 50.01
f.setScale(0, RoundingMode.HALF_UP)   // 50
f.setScale(0, RoundingMode.CEILING)   // 51

new BigDecimal("2.0").equals(new BigDecimal("2.00"))
// false  -> utiliser compareTo(...) == 0
```

- Construire un `BigDecimal` depuis une **chaîne**, jamais depuis un `double`
- Le FCFA n'a pas de subdivision : la règle d'**arrondi des frais** doit être spécifiée, puis testée
- `equals` compare aussi l'échelle : en test, utiliser `compareTo`
- Autre option : des montants en **entiers** (`long`), avec contrôle du débordement

Oracle pour l'argent

Égalité **exacte** au franc près, comparée avec `compareTo` : aucune tolérance n'est acceptable sur un montant.

TP : FAIRE ÉCHOUER LE SOLVEUR

TP

- **Objectif :** implémenter `Solveur.racines(a, b, c)`, trouver des entrées qui le mettent en défaut, puis le corriger
- **Étape 1 :** coder la formule du lycée et des tests JUnit « évidents » $(1, -3, 2) \rightarrow [1, 2]$
- **Étape 2 :** chercher des entrées qui révèlent : élimination catastrophique, débordement, racine double
- **Étape 3 :** pour chaque échec : est-ce l'**algorithme** (instable) ou le **problème** (mal conditionné) ?
- **Étape 4 :** implémenter la formule stable et la mise à l'échelle ; refaire passer tous les tests
- **Livrable :** dépôt Git : code, tests, tableau des cas avec résultat naïf / stable / attendu

VERSION NAÏVE : LA FORMULE DU LYCÉE

Solveur.java

```java
static double[] naif(double a, double b, double c) {
    if (a == 0) throw new IllegalArgumentException("a = 0");
    double d = b * b - 4 * a * c;
    if (d < 0) return new double[0];
    if (d == 0) return new double[]{-b / (2 * a)};
    double r1 = (-b - Math.sqrt(d)) / (2 * a);
    double r2 = (-b + Math.sqrt(d)) / (2 * a);
    return new double[]{Math.min(r1, r2),
                        Math.max(r1, r2)};
}
```

Question

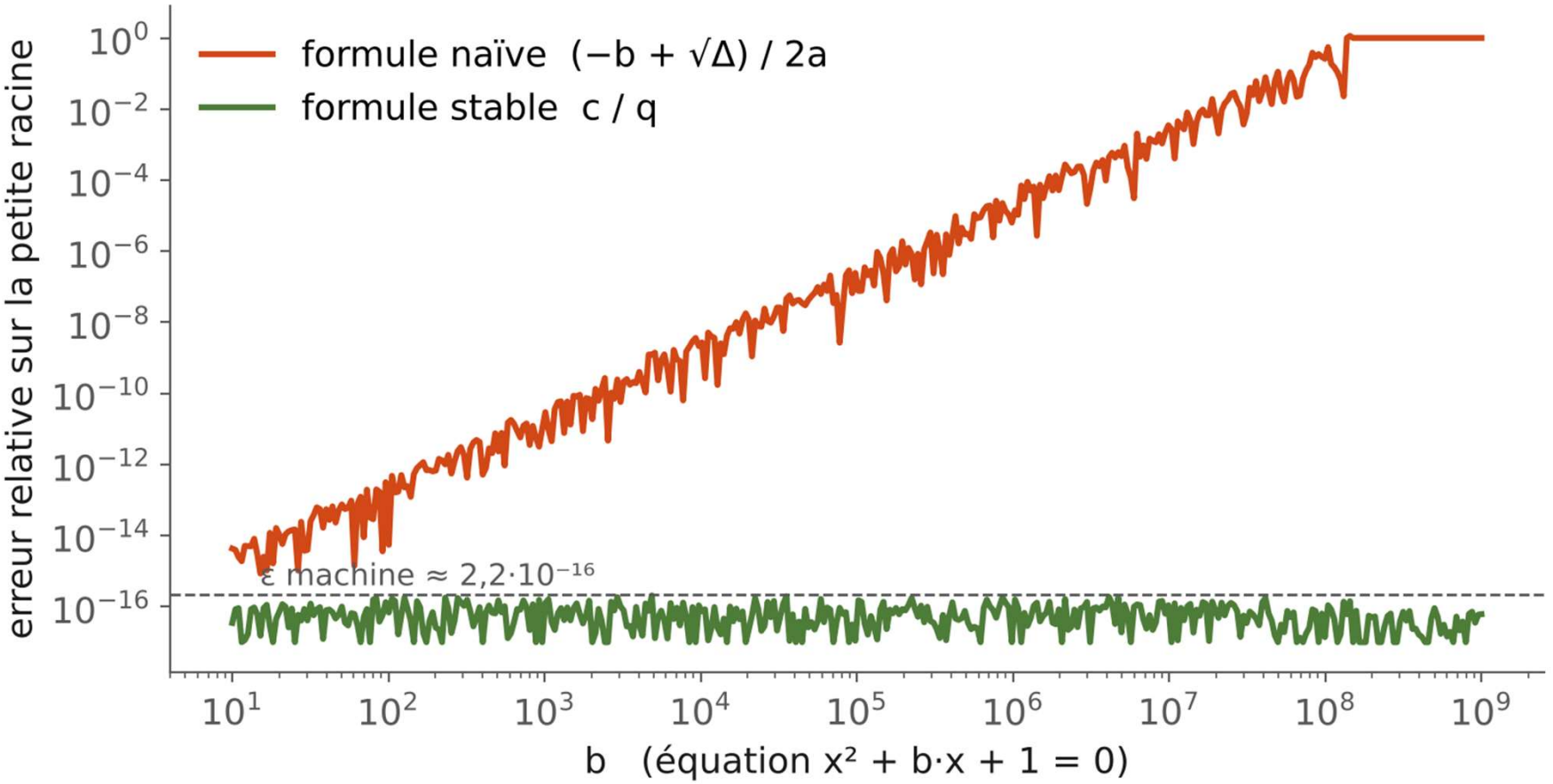
Ce code est « correct » au sens mathématique. Pour quelles entrées donne-t-il un résultat faux ? Cherchez avant de passer à la slide suivante.

RÉSULTATS RÉELS : NAÏF VS STABLE

(a, b, c)	Naïf	Stable	Diagnostic
(1, −3, 2)	[1.0, 2.0]	[1.0, 2.0]	Cas nominal
(1, 10^8, 1)	[−1.0E8, −7.450580596923828E-9]	[−1.0E8, −1.0E-8]	Élimination : **25 %** d'erreur
(1, 10^{200}, 1)	[−Infinity, Infinity]	[−1.0E200, −1.0E-200]	b^2 déborde en $+\infty$
(10^{200}, $-3\cdot10^{200}$, $2\cdot10^{200}$)	[NaN, NaN]	[1.0, 2.0]	$\infty - \infty$ = NaN, **silencieux**
(10^{-200}, 1, 10^{-200})	[−1.0E200, 0.0]	[−1.0E200, −1.0E-200]	Absorption : racine nulle fausse
(1, −2, 1 − 1,1·10^{-16})	[0.99999998946…, 1.00000001054…]	idem	Mal conditionné : les deux sont justes

Toutes ces valeurs ont été obtenues en **exécutant** `Solveur.java` (Java 21). Un test « (1, −3, 2) → [1, 2] » passe pour les deux versions : il ne distingue rien.

L'ERREUR DE LA FORMULE NAÏVE GRANDIT AVEC B



Erreur relative de la petite racine de $x^2 + bx + 1 = 0$, mesurée contre un calcul à 60 chiffres (`Decimal` Python). La formule naïve perd environ **deux chiffres par décade** de b ; la formule stable reste au niveau de ε.

VERSION STABLE ET MISE À L'ÉCHELLE

```java
static double[] stable(double a, double b, double c) {
    if (a == 0) throw new IllegalArgumentException("a = 0");
    // mise à l'échelle exacte par une puissance de 2
    double m = Math.max(Math.abs(a), Math.max(Math.abs(b), Math.abs(c)));
    int e = Math.getExponent(m);
    a = Math.scalb(a, -e); b = Math.scalb(b, -e); c = Math.scalb(c, -e);
    double d = b * b - 4 * a * c;
    if (d < 0) return new double[0];
    if (d == 0) return new double[]{-b / (2 * a)};
    double q = -0.5 * (b + Math.copySign(Math.sqrt(d), b));
    double r1 = q / a, r2 = c / q;          // Viète : r1 * r2 = c / a
    return new double[]{Math.min(r1, r2), Math.max(r1, r2)};
}
```

Ce qui change

`q` additionne deux termes **de même signe** : plus de soustraction de nombres proches. La seconde racine vient de **Viète**. Multiplier par $\mathbf{2^k}$ est exact en binaire : les racines ne changent pas et b^2 ne déborde plus. Reste ouvert : $\Delta \approx 0$ (conditionnement).

DES TYPES AUX VALEURS DE TEST

Type	Valeurs à inclure systématiquement	Ce qu'elles révèlent
int / long	MIN, MIN + 1, −1, 0, 1, MAX − 1, MAX ; valeurs dont la **somme ou le produit** déborde	Débordement, Math.abs(MIN), casts
double	±0.0, MIN_VALUE, MIN_NORMAL, 1 ± ulp, 2^{53}, 2^{53} + 1, MAX_VALUE, ±∞, NaN	Valeurs spéciales, perte des entiers
Calculs	Termes d'ordres de grandeur très différents ; différences de nombres proches	Absorption, élimination
Problème	Entrées près d'une **singularité** ($\Delta \approx 0$, matrice presque singulière)	Mal conditionnement
Durée	Exécutions longues, cumuls sur des millions d'opérations	Accumulation (Patriot)
Argent	Montants avec arrondi à la frontière (x,5), sommes de nombreux petits montants	Mode d'arrondi, BigDecimal

À RETENIR

À RETENIR

- Les entiers machine **bouclent** sans prévenir : viser MIN, MAX et les calculs qui les dépassent
- Un double a **53 bits** de précision ; les trous entre flottants **doublent** à chaque puissance de 2
- **NaN, ±∞, ±0** se propagent silencieusement et trompent ==
- Absorption, élimination catastrophique et accumulation transforment ε en erreur visible
- Distinguer **mal conditionné** (le problème) et **instable** (l'algorithme) avant d'accuser le code
- Jamais == sur des flottants calculés : une **tolérance justifiée** (séance 3)

ANATOMIE FORMELLE D'UN TEST ET PROBLÈME DE L'ORACLE

57

Séance 3 · 4 h · Modéliser un test, choisir la mesure d'erreur et la tolérance, construire des oracles

CHAP III

OBJECTIFS DE LA SÉANCE

- Décrire un test avec un **modèle formel** : DE, g, oracle O, RA, Tol, N, verdict
- Distinguer les **quatre verdicts** : PASS, FAIL, ERROR, INCONCLUSIF
- Choisir une **mesure d'erreur** adaptée et **justifier** une tolérance
- Traduire le modèle en **tests paramétrés** JUnit 5 / AssertJ
- Énoncer le **problème de l'oracle** et la taxonomie des oracles
- Construire des **oracles sans résultat attendu** : propriétés, relations métamorphiques, test différentiel

LE MODÈLE DU COURS : UN TEST EN SIX ÉLÉMENTS

Illustration sur le solveur du fil rouge A (`Solveur.racines`).

Élément	Notation	Rôle	Exemple (fil rouge A)
Scénario / cas de test	SC	Situation précise à vérifier	« Grande racine de $x^2 + 10^8 x + 1$ »
Données d'entrée	DE	Valeurs et conditions fournies	$(a, b, c) = (1, 10^8, 1)$
Système sous test	g	Ce qu'on exécute	`Solveur.racines`
Résultat attendu	RA	Sortie de référence	$-1 \cdot 10^{-8}$
Mesure d'erreur	N	Quantifie l'écart sortie / RA	$\lvert y - RA \rvert \,/\, \lvert RA \rvert$
Tolérance	Tol	Écart maximal accepté	10^{-6} (relatif)
Verdict	res	Décision	$N(g(DE), RA) \leq Tol$

Idée clé

Un test n'est pas « un appel + un affichage » : c'est une **décision** prise par un **oracle** à partir de critères explicites.

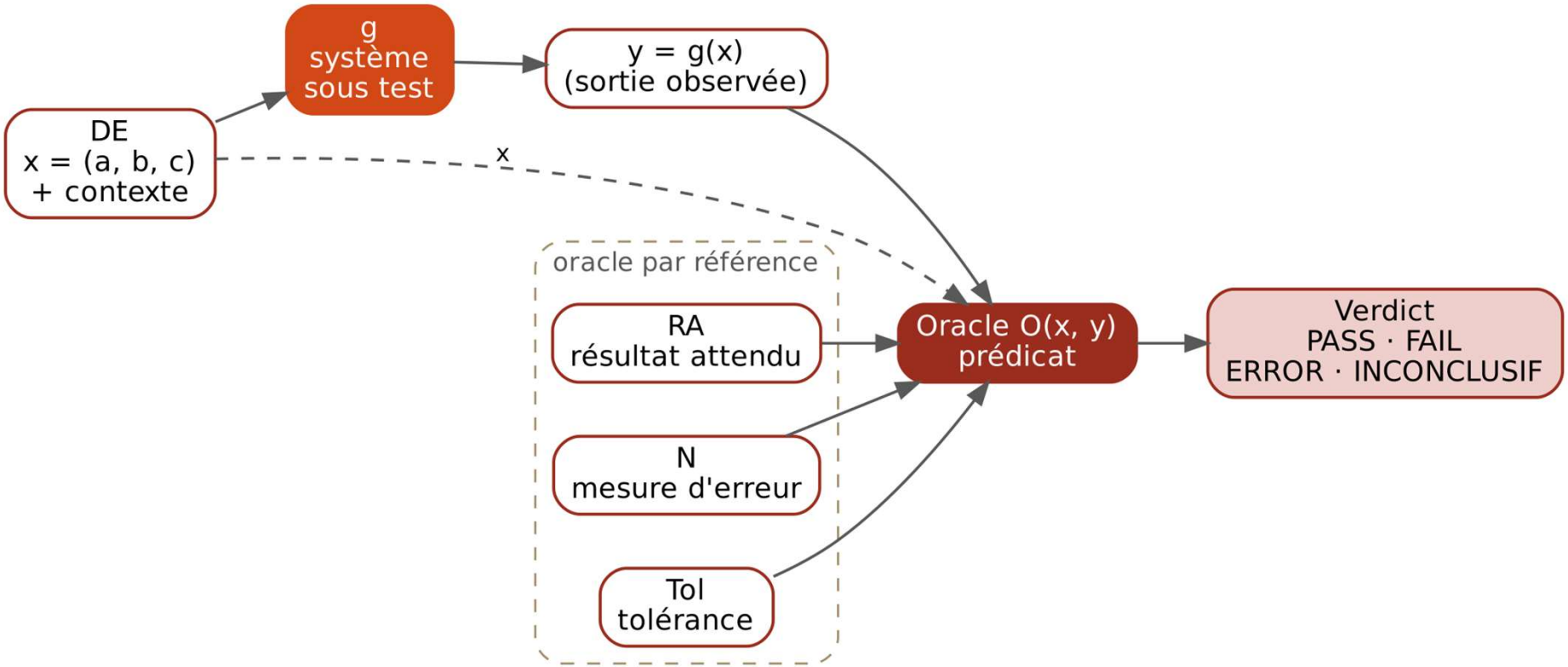
LA FORMULATION D'ORIGINE ET SES LIMITES

```
res = []
for s in SC:
    res[s] = N(g(DE), RA) <= Tol

# le problème de l'oracle :
# déterminer RA, g(X), N
```

- **À garder :** l'idée d'un verdict calculé à partir de N, RA et Tol
- **Manque 1 :** l'**oracle** reste implicite : on suppose qu'un RA existe toujours
- **Manque 2 :** ni **contexte** (état initial) ni **effets de bord**
- **Manque 3 :** pas d'**exception attendue**, pas de blocage : verdict seulement vrai/faux
- **Manque 4 :** Tol et N ne sont pas **justifiés**

LE MODÈLE CORRIGÉ : L'ORACLE AU CENTRE



À retenir

`N(y, RA) ≤ Tol` n'est **qu'une forme** d'oracle : l'oracle **par référence**. D'autres oracles décident sans connaître RA.

DÉFINITION : ORACLE DE TEST

D

Oracle de test (test oracle)
Mécanisme qui décide si la sortie **y** observée pour l'entrée **x** est acceptable : un prédicat **O(x, y) → {vrai, faux}**.

- **Par référence** : $O \equiv N(y, RA) \leq Tol$
- **Par propriété** : $O \equiv |a \cdot y^2 + b \cdot y + c| \leq Tol'$ (aucun RA nécessaire)
- **Implicite** : O ≡ « pas de plantage, pas de fuite mémoire, pas de dépassement de délai »
- **Humain** : O ≡ jugement d'un expert ou d'un utilisateur (recette)

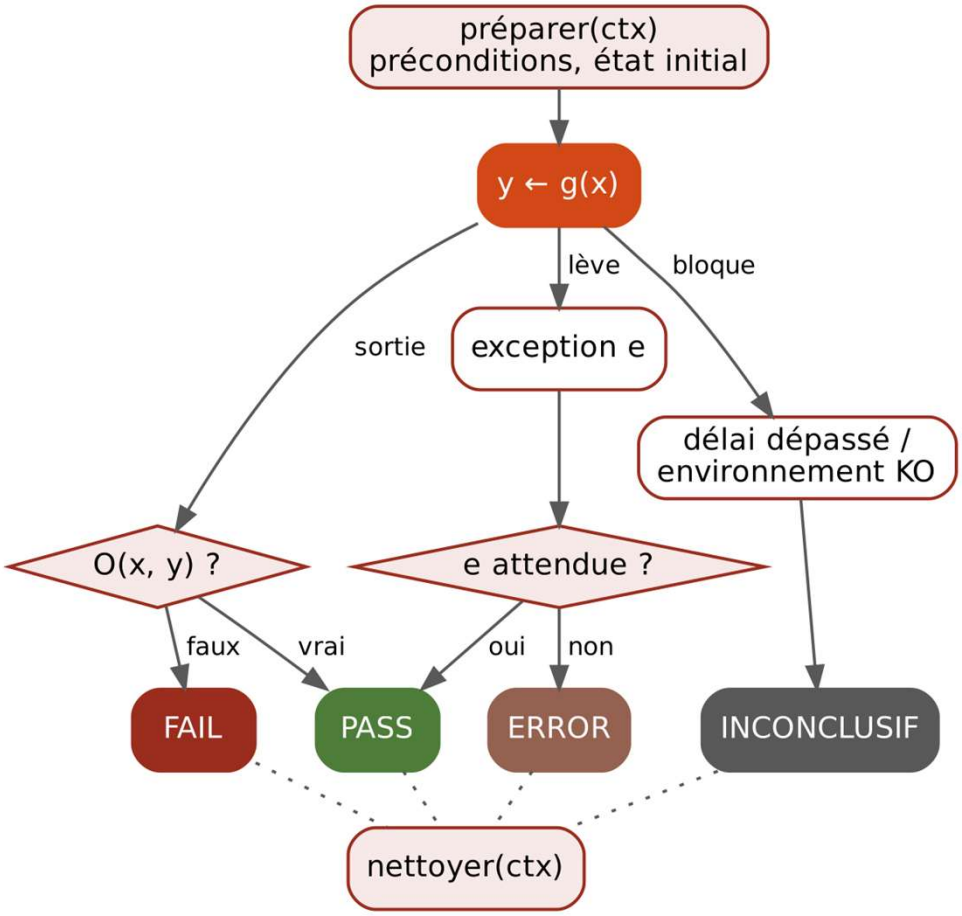
LE PSEUDO-CODE REFORMULÉ

```
pour chaque cas c = (x, ctx, RA, Tol) de SC :
    préparer(ctx)           # préconditions, état
    essayer :
        y ← g(x)            # exécution du SUT
        si O(x, y) :        # ex. N(y, RA) <= Tol
            verdict[c] ← PASS
        sinon :
            verdict[c] ← FAIL
    si exception e :
        si e attendue : verdict[c] ← PASS
        sinon :         verdict[c] ← ERROR
    si délai dépassé :
        verdict[c] ← INCONCLUSIF
    nettoyer(ctx)

verdict_suite ← tous les verdict[c] = PASS
```

- **ctx :** état initial maîtrisé (données, horloge, réseau)
- **exception :** peut être le **résultat attendu** (a = 0)
- **ERROR :** le test lui-même n'a pas pu conclure normalement
- **INCONCLUSIF :** environnement défaillant : ni succès ni échec du SUT

EXÉCUTION D'UN CAS DE TEST : QUATRE VERDICTS



- **PASS :** l'oracle accepte, ou l'exception attendue est levée
- **FAIL :** l'oracle rejette la sortie : **défaut probable du SUT**
- **ERROR :** exception imprévue : défaut du SUT **ou** du test
- **INCONCLUSIF :** délai, base indisponible, réseau : on ne sait pas
- Un FAIL se **reproduit** avant d'ouvrir un ticket

AU-DELÀ DE Y = G(X) : CONTEXTE ET EFFETS

1 Contexte et préconditions

- Données en base, fichiers, configuration
- Horloge, fuseau, langue (locale)
- À fixer dans `préparer(ctx)`

2 Sorties multiples

- Valeur de retour **et** effets de bord
- Ligne en base, message envoyé, journal
- L'oracle doit observer tout ce qui compte

3 Exceptions attendues

- `a = 0` → `IllegalArgumentException`
- Montant < 100 FCFA → refus
- Une exception **attendue** est un PASS

4 Non-déterminisme

- Heure, aléatoire, concurrence, réseau
- Injecter l'horloge, fixer la graine
- Isoler les dépendances (séances 9 et 12)

CHOISIR LA MESURE D'ERREUR N

Mesure	Formule	Quand l'utiliser	Piège
Absolue	$\lvert y - \hat{y} \rvert$	Grandeurs d'échelle connue (°C, FCFA)	Inadaptée aux très grandes ou petites valeurs
Relative	$\lvert y - \hat{y} \rvert \,/\, \lvert \hat{y} \rvert$	Calcul flottant, échelles variées	Explose quand $\hat{y} \approx 0$
Mixte	$\lvert y - \hat{y} \rvert \leq \text{atol} + \text{rtol} \cdot \lvert \hat{y} \rvert$	Cas général (`math.isclose`, NumPy)	Deux paramètres à justifier
Distance en ULP	nb de flottants entre y et $\hat{y}$	Bibliothèques mathématiques	Sans sens près de 0 ou entre signes
Norme	$\lVert y - \hat{y} \rVert_2$ ou $\lVert y - \hat{y} \rVert_\infty$	Vecteurs, matrices, séries	Une composante peut cacher les autres
Ensembles	appariement puis N élément par élément	Racines, résultats non ordonnés	Comparer dans l'ordre de sortie
Levenshtein	nb d'éditions de caractères	Texte, OCR, transcription	Ne mesure pas le sens
Percentile	p95, p99 d'une distribution	Temps de réponse, charge	La moyenne cache les pires cas

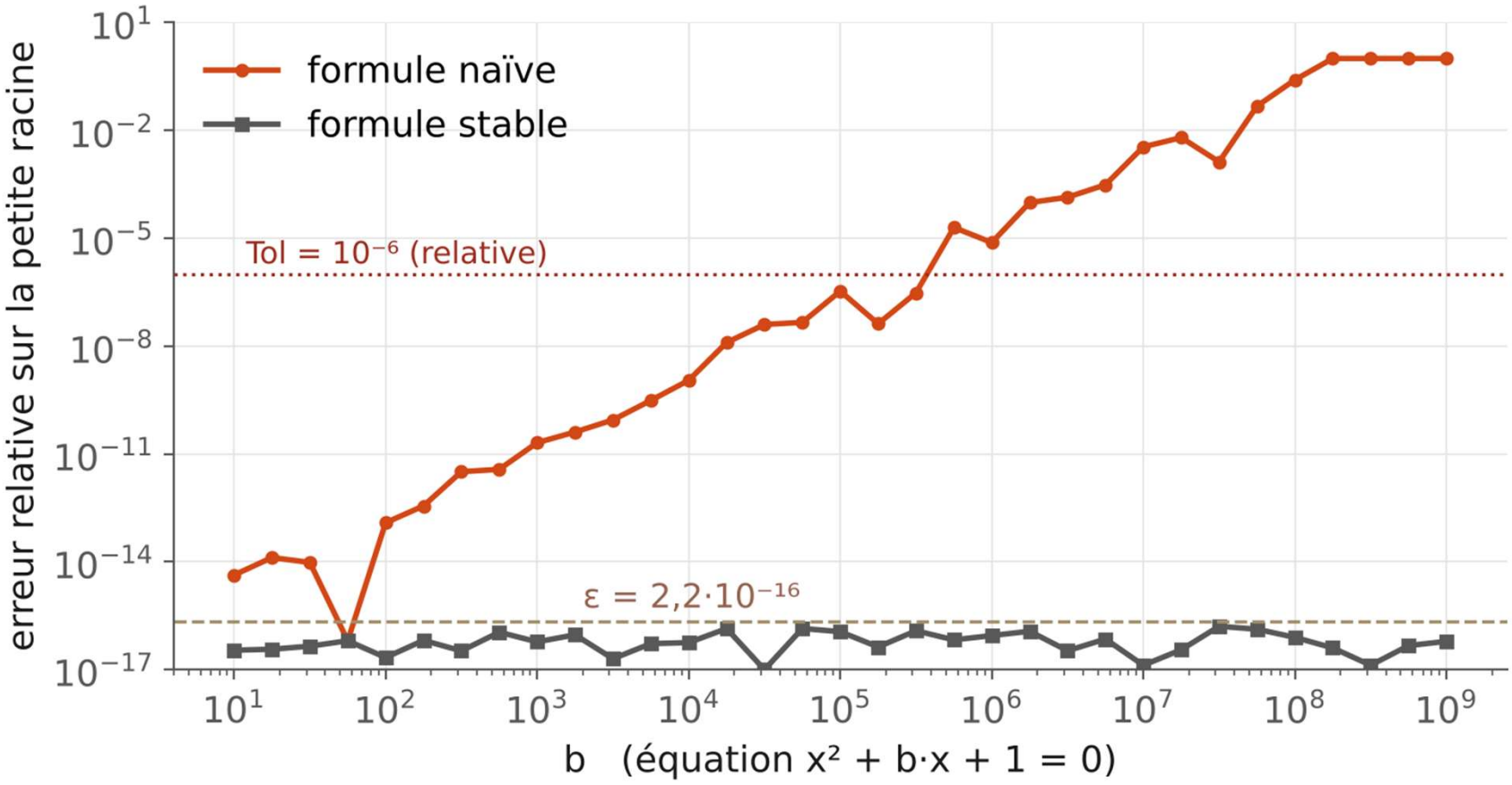
JUSTIFIER LA TOLÉRANCE : PRÉCISION ET CONDITIONNEMENT

- Erreur relative attendue d'un bon algorithme : $\approx \kappa \cdot \varepsilon$
- ε : précision machine, `Math.ulp(1.0)` $= 2^{-52} \approx 2{,}2 \cdot 10^{-16}$ en double
- κ : conditionnement : sensibilité du **problème** aux perturbations des données
- Règle pratique : **Tol = c · κ · ε**, avec c petit (≈ 10), écrite dans le test
- Racine double ($\Delta \approx 0$) : κ très grand, on ne garde que $\sim\sqrt{\varepsilon} \approx 10^{-8}$ en relatif

Deux erreurs symétriques

Tol **trop large** : le test laisse passer les bugs. Tol **trop étroite** : faux échecs, puis l'équipe « élargit au hasard » ou désactive le test.

UNE TOLÉRANCE RELATIVE RÉVÈLE L'INSTABILITÉ



Petite racine de $x^2 + b{\cdot}x + 1 = 0$, calculée en double ; référence en précision 60 chiffres

Au-delà de b ≈ 10^5, la formule naïve dépasse Tol = 10^{-6} ; à b = 10^8, l'erreur vaut **25 %**. La formule stable reste à l'échelle de ε.

LE MODÈLE EN JUNIT 5 : TEST PARAMÉTRÉ

```
// DE = (a, b, c)    RA = ra    Tol = rtol
@ParameterizedTest
@CsvSource({ "1, -3,  2,  2.0,  1e-12",
             "1, 1e8, 1, -1e-8, 1e-6" })
void plusGrandeRacine(double a, double b, double c,
                      double ra, double rtol) {
    double[] r = Solveur.racines(a, b, c); // y = g(DE)
    double y = r[r.length - 1];
    // N = erreur relative ; verdict : N <= Tol
    assertTrue(Math.abs(y - ra) <= rtol * Math.abs(ra));
}
```

- Chaque **ligne CSV** = un cas : DE, RA, Tol
- La **méthode** = g + oracle
- Ajouter un cas = ajouter une ligne
- Formule naïve : le 2e cas **échoue** (erreur 0,25)
- Formule stable : les 2 cas **passent**

PIÈGE : UNE TOLÉRANCE ABSOLUE MASQUE 25 % D'ERREUR

PIÈGE

```java
@ParameterizedTest
@CsvSource({ "1, 1e8, 1, -1e-8, 1e-6" })
void piege(double a, double b, double c,
           double ra, double tol) {
    double[] r = Solveur.racinesNaif(a, b, c);
    double y = r[r.length - 1];   // -7.45e-9
    assertTrue(Math.abs(y - ra)
               <= tol * Math.max(1, Math.abs(ra)));
}   // PASS !
```

- $|y - RA| = 2{,}5 \cdot 10^{-9} \leq 10^{-6}$: le test **passe**
- Or y est faux de **25 %**
- Le `max(1, |RA|)` transforme la tolérance en tolérance **absolue** pour $|RA| < 1$
- Correction : erreur **relative**, ou mixte avec atol justifié

ASSERTIONS NUMÉRIQUES : JUNIT 5 ET ASSERTJ

```java
double y = Solveur.racines(1, -3, 2)[1];

// JUnit 5 : delta ABSOLU uniquement
assertEquals(2.0, y, 1e-12);

// AssertJ : absolu ou relatif
assertThat(y).isCloseTo(2.0, within(1e-12));
assertThat(y).isCloseTo(2.0, withinPercentage(1e-9));

// Exception attendue = résultat attendu
assertThrows(IllegalArgumentException.class,
             () -> Solveur.racines(0, 2, 1));
```

- `assertEquals(attendu, obtenu, delta)` : |obtenu − attendu| ≤ delta
- `within` : absolu ; `withinPercentage` : relatif en % (1e-9 % = 10^{-11})
- **Jamais** `assertEquals(2.0, y)` sur un résultat calculé en flottant
- `assertThrows` matérialise l'exception attendue

UN MÊME MODÈLE POUR TOUS LES TESTS

Type de test	DE	Oracle O	N	Tol
Unitaire numérique	(a, b, c)	Référence ou propriété	Erreur relative	$c \cdot \kappa \cdot \varepsilon$
Unitaire métier	Montant FCFA	Frais exacts (BigDecimal)	$\lvert y - RA \rvert$	0
Performance	Profil de charge k6	Niveau de service	Latence p95	≤ 300 ms
Sécurité	Charges d'injection	Implicite : pas d'erreur 500, pas de fuite	Nb d'anomalies	0
Recette	Scénario utilisateur	Critères d'acceptation, humain	Critères remplis	Tous
ML / IA	Jeu de validation	Statistique	Exactitude (accuracy)	≥ seuil

Colonne vertébrale du cours

À chaque nouvelle technique, on se demandera : **quelles DE ? quel oracle ? quelle N ? quelle Tol ?**

LE PROBLÈME DE L'ORACLE

D

Programme « non testable » (Weyuker, 1982)

Un programme est non testable quand **aucun oracle n'existe**, ou qu'il est **trop coûteux** de décider si une sortie est correcte.

- Calcul scientifique et simulation : on calcule justement parce qu'on ne connaît pas la réponse
- Compilateurs, optimiseurs, moteurs de recherche : sorties trop complexes à vérifier à la main
- Systèmes distribués et concurrents : plusieurs sorties correctes possibles
- IA, LLM : pas de « bonne réponse » unique
- Conséquence : on remplace l'oracle complet par des **oracles partiels**

CONTRÔLABILITÉ, OBSERVABILITÉ, TESTABILITÉ

Avant même l'oracle : peut-on **placer** le système dans l'état voulu et **voir** ce qu'il produit ?

C Contrôlabilité

- Fixer les entrées **et** l'état
- Obstacles : `now()`, aléatoire, base partagée
- Remède : injection de dépendances

O Observabilité

- Capturer toutes les sorties
- Obstacles : état privé, effets externes
- Remède : journaux, sondes, accesseurs de test

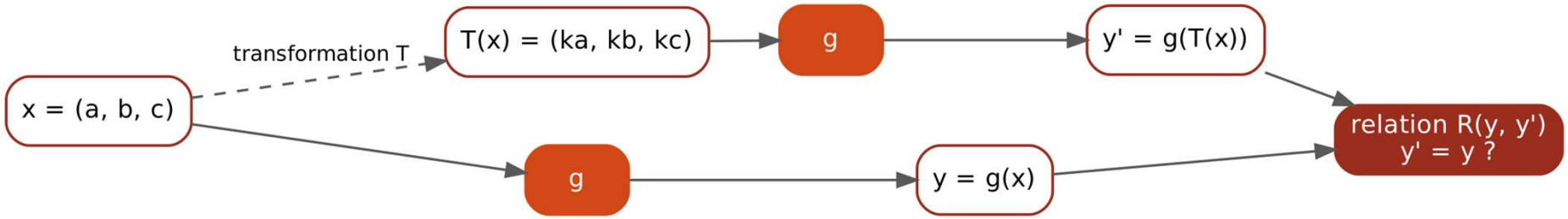
T Testabilité

- Contrôlabilité + observabilité + oracle disponible
- Se **conçoit** dès l'architecture
- Un code testable est souvent mieux découplé

TAXONOMIE DES ORACLES (BARR ET AL., 2015)

Famille	Principe	Exemple	Limite
Spécifié	Contrats, assertions, spécification formelle	Postcondition JML, invariant	Il faut écrire la spécification
Dérivé : métamorphique	Relation entre plusieurs exécutions	(ka, kb, kc) donne les mêmes racines	Ne voit que ce qui viole la relation
Dérivé : différentiel	Comparer à une autre implémentation	Version BigDecimal, NumPy	Les deux peuvent se tromper pareil
Dérivé : régression	Comparer à une version antérieure (golden master)	Instantané de sortie	Détecte un changement, pas une erreur
Implicite	Défaillances évidentes	Plantage, fuite, délai	Ignore les résultats faux
Humain	Jugement d'un expert	Recette utilisateur	Coûteux, à réduire

LE TEST MÉTAMORPHIQUE



- Proposé par **T. Y. Chen (1998)** : on ne connaît pas y, mais on sait comment y **doit varier** quand x varie
- Relation métamorphique : transformation T des entrées + relation R entre les sorties
  - Exemples : $\sin(x) = \sin(\pi - x)$; trier une permutation donne le même résultat ; recherche avec un filtre en plus $\subseteq$ recherche sans filtre
- Pour le solveur : $(ka, kb, kc) \rightarrow$ mêmes racines ; $(a, -b, c) \rightarrow$ racines opposées
- Choisir $k = 2^n$: la multiplication est **exacte** en binaire, donc l'égalité doit être **stricte**

FIL ROUGE A : SIX ORACLES SANS RÉSULTAT ATTENDU

Oracle	Propriété vérifiée	Détecte la formule naïve ?
Résidu (erreur inverse)	$\lvert a\cdot r^2 + b\cdot r + c\rvert \le k\cdot\varepsilon\cdot(\lvert a\rvert r^2 + \lvert b\rvert\lvert r\rvert + \lvert c\rvert)$	**Oui** (0,25 contre $8\cdot\varepsilon\cdot 1{,}75$)
Viète	$r_1 + r_2 = -b/a$ et $r_1\cdot r_2 = c/a$	**Oui** (produit 0,745 au lieu de 1)
Métamorphique : échelle	racines(ka, kb, kc) = racines(a, b, c), $k = 2^{10}$	Non
Métamorphique : opposé	racines(a, −b, c) = −racines(a, b, c)	Non
Construction	à partir de r_1, r_2 choisis : retrouver r_1, r_2	**Oui**
Différentiel	comparaison à un calcul BigDecimal 50 chiffres	**Oui**

Leçon
Les relations métamorphiques passent aussi sur la version naïve : elle est **fausse mais cohérente**. Un oracle partiel ne voit que ce qui **viole sa propriété** : on en combine plusieurs.

ORACLES PAR PROPRIÉTÉ : RÉSIDU ET VIÈTE

```java
static final double EPS = Math.ulp(1.0);    // 2^-52

@ParameterizedTest
@CsvSource({ "1, 1e8, 1", "3, 7, 2", "2, -4, 1.99" })
void residu(double a, double b, double c) {
    for (double r : Solveur.racines(a, b, c)) {
        double res = Math.abs(a*r*r + b*r + c);
        double ech = Math.abs(a)*r*r + Math.abs(b*r) + Math.abs(c);
        assertTrue(res <= 8 * EPS * ech);    // erreur inverse
    }
}

@ParameterizedTest
@CsvSource({ "1, 1e8, 1", "3, 7, 2" })
void viete(double a, double b, double c) {
    double[] r = Solveur.racines(a, b, c);
    assertThat(r[0] + r[1]).isCloseTo(-b / a, withinPercentage(1e-10));
    assertThat(r[0] * r[1]).isCloseTo( c / a, withinPercentage(1e-10));
}
```

Le résidu seul ne suffit pas : il faut le comparer à l'**échelle** des termes (erreur inverse), sinon un résidu « non nul » dû à l'arrondi passe pour un bug.

ORACLES MÉTAMORPHIQUES

```java
@ParameterizedTest
@CsvSource({ "3, 7.1, 2", "1, 1e8, 1", "-2, 3, 5" })
void echelle(double a, double b, double c) {
    double k = 1024;      // 2^10 : produit exact
    assertArrayEquals(Solveur.racines(a, b, c),
        Solveur.racines(k*a, k*b, k*c)); // strict
}

@ParameterizedTest
@CsvSource({ "3, 7.1, 2", "1, 1e8, 1" })
void oppose(double a, double b, double c) {
    double[] r = Solveur.racines(a, b, c);
    double[] s = Solveur.racines(a, -b, c);
    assertEquals(-r[0], s[1], 0.0);
    assertEquals(-r[1], s[0], 0.0);
}
```

- Aucun RA dans ces tests
- Multiplier par 2^{10} ne change que l'exposant : **aucun arrondi**
- Avec k = 10, résultats différents au dernier bit : il faudrait une Tol
- Les racines sont **triées** : l'opposé inverse l'ordre

ORACLE PAR CONSTRUCTION : PARTIR DE LA RÉPONSE

```java
@ParameterizedTest
@CsvSource({ "-1e8, -1e-8", "1, 2", "-3, 1e-5" })
void construction(double r1, double r2) {
    // (x - r1)(x - r2) = x² - (r1 + r2)x + r1·r2
    double[] r =
        Solveur.racines(1, -(r1 + r2), r1 * r2);
    assertThat(r[0]).isCloseTo(Math.min(r1, r2),
                               withinPercentage(1e-9));
    assertThat(r[1]).isCloseTo(Math.max(r1, r2),
                               withinPercentage(1e-9));
}
```

- On **choisit** la solution, puis on fabrique le problème
- Le RA devient connu **par construction**
- Attention : $b = -(r_1 + r_2)$ et $c = r_1 \cdot r_2$ sont eux-mêmes arrondis
- Même idée que les **solutions manufacturées** (séance 6)

ORACLE DIFFÉRENTIEL : UNE RÉFÉRENCE EN BIGDECIMAL

```java
@ParameterizedTest
@CsvSource({ "1, 1e8, 1", "3, 7.1, 2", "1e-3, -5, 1e3" })
void differentiel(double a, double b, double c) {    // a > 0
    MathContext mc = new MathContext(50);            // 50 chiffres
    BigDecimal A = new BigDecimal(a), B = new BigDecimal(b);
    BigDecimal C = new BigDecimal(c), DEUX = BigDecimal.valueOf(2);
    BigDecimal s = B.multiply(B)
        .subtract(A.multiply(C).multiply(BigDecimal.valueOf(4)))
        .sqrt(mc);
    BigDecimal ref = B.negate().add(s).divide(A.multiply(DEUX), mc);
    assertThat(Solveur.racines(a, b, c)[1])
        .isCloseTo(ref.doubleValue(), withinPercentage(1e-12));
}
```

Pourquoi ça marche

Avec 50 chiffres, la formule naïve n'a plus d'élimination catastrophique : elle sert de **référence**. `new BigDecimal(a)` reprend la valeur exacte du double.

ORACLES POUR L'IA ET LES LLM

- Pas de résultat attendu unique : le problème de l'oracle **dans sa forme extrême**
- **Statistique :** métrique sur un jeu de validation (exactitude, F1) ≥ seuil
- **Métamorphique :** paraphrase, permutation, bruit léger → même décision
- **Propriétés :** format JSON valide, pas de donnée personnelle, langue respectée
- **LLM juge :** un modèle évalue un autre ; biais à étalonner sur des cas notés par des humains
- **Humain :** échantillon évalué à l'aveugle

Même modèle

DE = jeu d'invites ; O = combinaison d'oracles partiels ; N = taux d'échec ; Tol = seuil accepté et **documenté**.

TP DE LA SÉANCE : DES ORACLES POUR LE SOLVEUR

TP

- Reprendre `Solveur` (versions naïve et stable) de la séance 2
- Écrire un test paramétré **DE / RA / Tol** de 8 cas, avec une N **justifiée**
- Écrire **au moins quatre oracles sans RA** : résidu, Viète, deux relations métamorphiques
- Exécuter toute la suite contre les **deux** versions et remplir le tableau « détecte / ne détecte pas »
- Bonus : test différentiel en BigDecimal ; cas $\Delta \approx 0$ avec une Tol justifiée par le conditionnement
- Rendu : dépôt Git + court rapport (1 page) qui justifie chaque tolérance

À RETENIR

1 Un test = une décision

DE, g, **oracle O**, et selon l'oracle : RA, N, Tol. Quatre verdicts : PASS, FAIL, ERROR, INCONCLUSIF.

2 N et Tol se justifient

Erreur relative ou mixte pour les flottants ; Tol $\approx c \cdot \kappa \cdot \varepsilon$, écrite et expliquée. Une Tol absolue sur de petites valeurs masque les bugs.

3 Le problème de l'oracle

Souvent, on ne connaît pas RA. On combine des **oracles partiels** : propriétés, métamorphiques, différentiels, implicites.

4 Aucun oracle ne voit tout

Une version fausse mais cohérente passe les relations métamorphiques : **combiner** plusieurs oracles.

CONCEPTION DE TESTS EN BOÎTE NOIRE

85

Séance 4 · 4 h · Classes d'équivalence, valeurs limites, tables de décision, états, combinatoire

OBJECTIFS DE LA SÉANCE

- Construire des **classes d'équivalence** valides et invalides, pour les entrées **et** les sorties
- Connaître les **classes usuelles** de chaque type : entier, réel, booléen, caractère, chaîne, date, collection
- Appliquer l'**analyse des valeurs limites** à 2 et à 3 valeurs
- Modéliser des règles métier par **tables de décision** et graphes cause-effet
- Tester un **système à états** : couverture des états, des transitions, des transitions invalides
- Réduire la combinatoire : **each-choice**, **pairwise**, outils (PICT)

BOÎTE NOIRE : DÉRIVER LES TESTS DE LA SPÉCIFICATION

On ne regarde **pas le code** : les cas de test viennent des exigences, des cas d'utilisation et des modèles.



- Lire la spécification
- Repérer entrées, sorties, règles
- Noter les **ambiguïtés**

- Domaines de chaque entrée et sortie
- Classes valides et invalides

- Un représentant par classe
- **Valeurs limites** entre classes

- Tables de décision
- États
- Pairwise

- DE, RA, oracle (séance 3)
- Tests paramétrés

Techniques du syllabus ISTQB v4.0
Partitions d'équivalence, valeurs limites, tables de décision, transitions d'états ; on y ajoute ici la combinatoire et les cas d'utilisation.

CLASSE D'ÉQUIVALENCE

D

Classe d'équivalence (equivalence partitioning)
Sous-ensemble du domaine d'entrée (ou de sortie) pour lequel on **suppose** que le système se comporte de la même façon : **hypothèse d'uniformité**.

- Les classes forment une **partition** : disjointes, et leur union couvre tout le domaine
- Si l'hypothèse est vraie, **un représentant par classe suffit**
- **Valides** (traitement normal) et **invalides** (rejet attendu)
- Partitionner aussi les **sorties** : chaque comportement distinct observable = une classe
- L'hypothèse est souvent fausse **aux frontières** : d'où l'analyse des valeurs limites

RÈGLES DE CONSTRUCTION DES CLASSES

Règles classiques (Myers), illustrées sur les fils rouges B et C.

Spécification	Classes valides	Classes invalides
Un intervalle : montant de 100 à 500 000 FCFA	1 : [100 ; 500 000]	2 : < 100 et > 500 000
Un nombre de valeurs : 1 à 3 bénéficiaires	1 : de 1 à 3	2 : 0 et plus de 3
Un ensemble : opérateur Orange ou MTN	1 par valeur (2)	1 : autre opérateur
Une condition obligatoire : commence par +237	1 : commence par +237	1 : ne commence pas par +237
Un doute sur l'uniformité du traitement	Subdiviser la classe	Idem

Chaque règle métier distincte (tranche de frais, plafond) crée une **frontière** : autant de classes que de comportements.

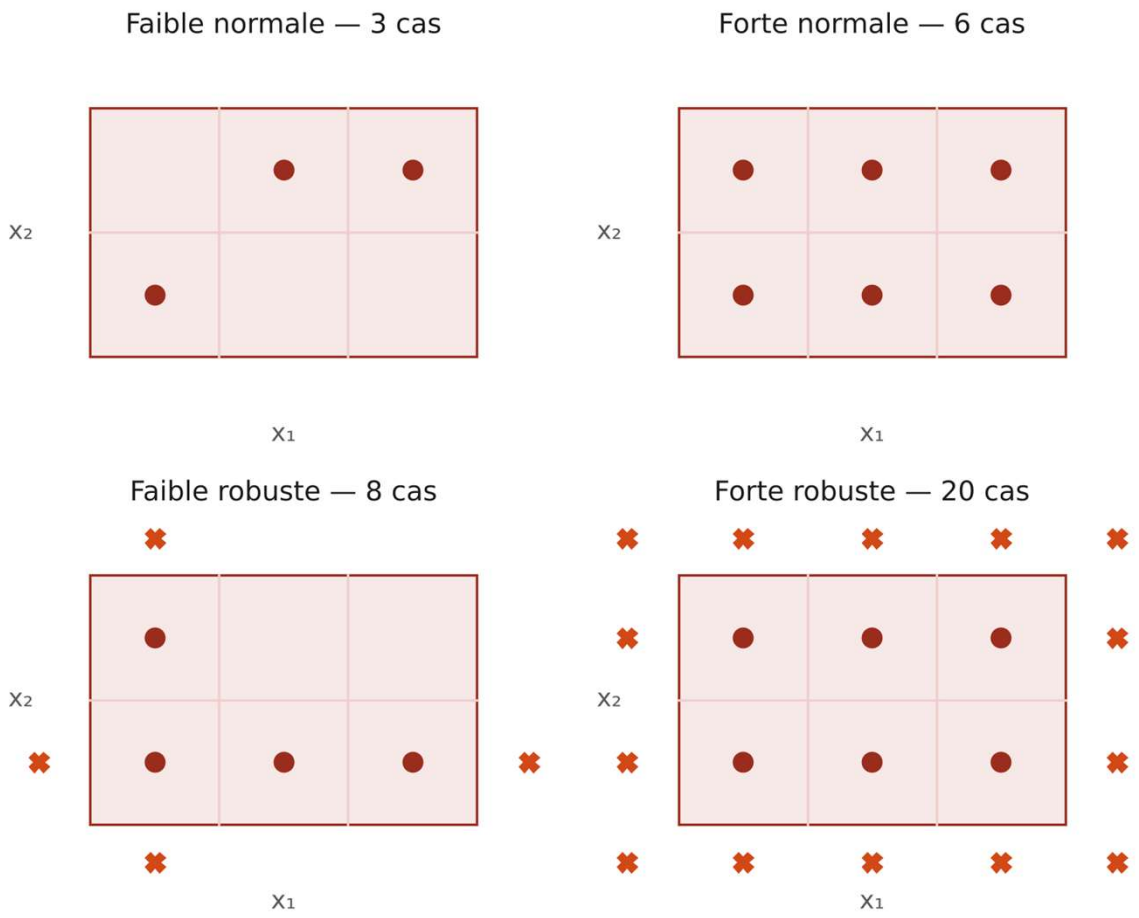
UNE SEULE CLASSE INVALIDE PAR CAS DE TEST

PIÈGE

- Un cas qui combine **deux** valeurs invalides ne teste en pratique **qu'une** des deux validations
- La première erreur détectée **masque** la seconde
- Règle : chaque cas invalide = **une** classe invalide + des valeurs valides partout ailleurs
- Les classes **valides**, elles, peuvent être combinées dans un même cas

Montant	Téléphone	Résultat
50 ✗	+2375… ✗	Refus montant ; téléphone **jamais vérifié**
50 ✗	valide	Refus montant ✓
10 000	+2375… ✗	Refus téléphone ✓

FAIBLE OU FORTE, NORMALE OU ROBUSTE (JORGENSEN)



- Deux entrées x_1 (3 classes valides) et x_2 (2 classes valides)
- **Faible :** chaque classe couverte **au moins une fois** : $\max(n_i)$ cas
- **Forte :** **produit cartésien** des classes : $\prod n_i$ cas
- **Normale :** classes **valides** seulement
- **Robuste :** on ajoute les classes **invalides** (✕)
- Le fort robuste explose vite : on revient au **pairwise**

CLASSES USUELLES : ENTIERS

Classe	Représentant	Limites et pièges
Négatif	−42	`MIN_VALUE` (-2^{31}), `MIN_VALUE + 1`, −1
Zéro	0	Division par zéro, cas « aucun »
Positif	42	1, `MAX_VALUE - 1`, `MAX_VALUE` ($2^{31} - 1$)
Hors domaine métier	50 (< 100 FCFA)	Borne métier − 1 et + 1 (99, 500 001)
Fait déborder un calcul	`MAX_VALUE / 2 + 1` pour une somme	`MAX_VALUE + 1 == MIN_VALUE` ; `Math.abs(MIN_VALUE) < 0`
Saisie non numérique	"12a", "", " 12"	`parseInt(" 12")` lève une exception ; "+12" et "١٢" sont acceptés

Les limites **du type** (`MIN`, `MAX`) s'ajoutent aux limites **du métier** : on teste les deux.

CLASSES USUELLES : RÉELS (DOUBLE)

Classe	Représentant	Limites et pièges
Négatif / positif « normal »	−3,7 / 2,5	Ordres de grandeur variés dans un même test
Zéro signé	+0.0 et −0.0	`0.0 == -0.0` est vrai mais `1/-0.0` = −∞
Très petit	1e−300	Sous-normaux : `MIN_NORMAL` ≈ $2{,}2 \cdot 10^{-308}$, `MIN_VALUE` ≈ $4{,}9 \cdot 10^{-324}$
Très grand	1e300	`MAX_VALUE` ; `x * x` donne +∞
Valeurs spéciales	NaN, +∞, −∞	`NaN == NaN` est faux ; toute comparaison avec NaN est fausse
Non représentable exactement	0,1 ; 1/3	`0.1 + 0.2 != 0.3`
Entier au-delà de 2^{53}	9 007 199 254 740 993	Devient 9 007 199 254 740 992 en double
Valeur entière	3.0 ; −3.99	`(int) -3.99` = −3 (troncature) ; `(int) 1e20` = `MAX_VALUE`

CLASSES USUELLES : BOOLÉENS ET CARACTÈRES

Booléen	Pièges
`true`	Cas nominal
`false`	Cas nominal
`null` **(objet** `Boolean`**)**	NPE au déballage (unboxing)
Saisie texte (`parseBoolean`**)**	« TRUE » → true ; « yes », « 1 » → **false**

Caractère : classe	Exemples et pièges
Lettre minuscule / majuscule	a, Z ; casse dépendante de la langue
Chiffre	0–9, mais aussi chiffres Unicode (١)
Blanc	espace, tabulation, **espace insécable** U+00A0
Ponctuation, spécial	' " ; < > / \
Contrôle	\0, \n, \r
Accentué	é, ç, É, ô (noms camerounais : Ngô, Bétí)
Hors plan de base	emoji 😜 : **2** `char` en Java
Langue (locale)	"TITLE".toLowerCase() en turc → "tıtle"

Piège du copier-coller

L'espace insécable U+00A0 n'est retirée ni par `trim()` ni par `strip()`, et `isBlank()` renvoie **false** pour une chaîne qui n'en contient que.

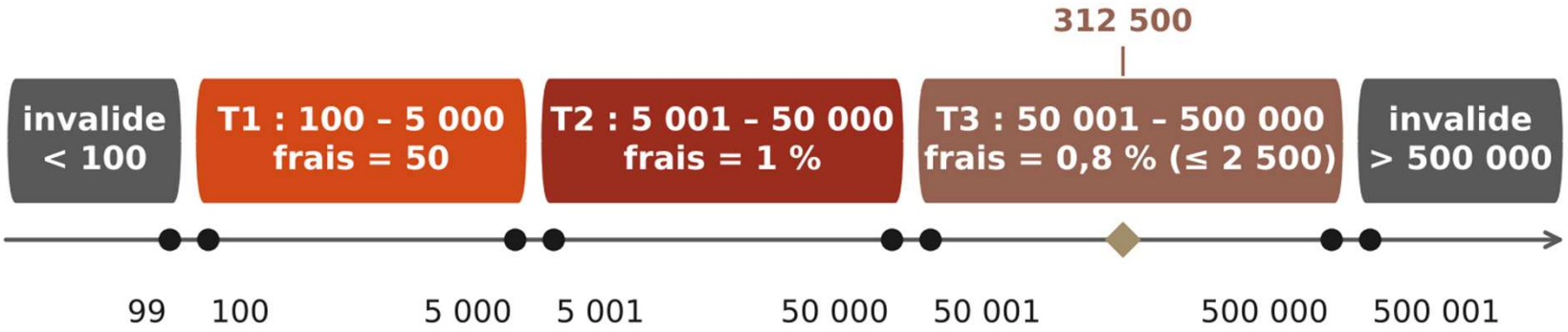
CLASSES USUELLES : CHAÎNES DE CARACTÈRES

Classe	Représentant	Limites et pièges
`null` / vide	null ; ""	NullPointerException ; `isEmpty` ≠ `isBlank`
Longueur minimale	1 caractère, 2 caractères	min − 1 et min
Nominale	"Fomekong"	—
Longueur maximale	50 et 51 caractères	Compter en **points de code**, pas en `char`
Blancs	" " ; " Ali "	Espaces autour, espace insécable
Unicode	"José" (NFC ou NFD) ; "Ali 😀"	"José" décomposé fait 5 `char` : **normaliser**
Injection	`' OR 1=1 --` ; `<script>` ; `../../`	SQL, XSS, chemin : lien avec la séance 13
Très longue	10^6 caractères	Performance, expressions régulières catastrophiques (ReDoS)
Format	e-mail, téléphone valide / invalide	Spécifier le format **exact**

CLASSES USUELLES : DATES, COLLECTIONS, ÉNUMÉRATIONS

Type	Classes	Limites et pièges
Date	Valide / invalide ; passé / futur ; année bissextile ou non	29/02, 31/04, 31/12 → 01/01, 01/01/1970, 19/01/2038
Heure, fuseau	Avant / après midi ; même jour / jour suivant	00:00, 23:59:59 ; changement d'heure (Europe) ; Cameroun UTC+1 sans heure d'été
Durée	Nulle, positive, négative	Fin avant début
Collection, tableau	`null`, vide, 1 élément, plusieurs, taille maximale	Doublons, déjà triée, triée à l'envers, contient `null`
Énumération	Chaque valeur ; valeur hors liste	`null` ; valeur ajoutée plus tard (cas `default` d'un `switch`)
Référence, objet	`null`, objet valide, objet dans un état invalide	Objet partagé modifié ailleurs (aliasing)

ANALYSE DES VALEURS LIMITES



- **À 2 valeurs (ISTQB v4) :** la limite et sa voisine de l'autre classe : 5 000 et 5 001
- **À 3 valeurs :** la limite et ses deux voisines : 4 999, 5 000, 5 001 et 5 000, 5 001, 5 002
- **Limite de sortie :** 312 500 FCFA : 0,8 % atteint le plafond de 2 500 FCFA
- **Question à la spécification :** 50 000 → 500 FCFA mais 50 001 → 400 FCFA : est-ce voulu ?

FIL ROUGE B : FRAIS PAR TRANCHE

Classe	Domaine	Limites (2 valeurs)	Frais attendus
Invalide basse	< 100	99	Refus
T1	100 – 5 000	100 ; 5 000	50 ; 50
T2	5 001 – 50 000	5 001 ; 50 000	50 ; 500
T3 sous plafond	50 001 – 312 500	50 001 ; 312 500	400 ; 2 500
T3 plafonnée	312 501 – 500 000	312 501 ; 500 000	2 500 ; 2 500
Invalide haute	> 500 000	500 001	Refus
Invalide : format	montant non entier	1 000,5	Refus

10 valeurs suffisent à couvrir toutes les classes et toutes les frontières, contre 500 000 montants possibles.

FIL ROUGE B : LES LIMITES EN TEST PARAMÉTRÉ

```
@ParameterizedTest(name = "{0} FCFA -> {1}")
@CsvSource({
    "100,    50",   "5000,   50",      // T1 : bornes
    "5001,   50",   "50000,  500",     // T2 : 1 % arrondi
    "50001,  400",  "312500, 2500",    // T3 : 0,8 %
    "312501, 2500", "500000, 2500" })  // plafond des frais
void fraisAuxLimites(BigDecimal montant, BigDecimal attendu) {
    assertEquals(0, s.frais(montant).compareTo(attendu));
}

@ParameterizedTest
@CsvSource({ "99", "500001", "0", "-5000", "1000.5" })
void montantsInvalides(BigDecimal montant) {
    assertThrows(MontantInvalideException.class,
                 () -> s.frais(montant));
}
```

Oracle : égalité **exacte** en BigDecimal (`compareTo`), Tol = 0. JUnit convertit le texte CSV en `BigDecimal` automatiquement.

FIL ROUGE C : CLASSES PAR CHAMP DU FORMULAIRE

Champ	Classes valides	Classes invalides
Matricule	2 chiffres + 1 majuscule + 4 chiffres (24G1234)	Minuscule ; 1 ou 3 chiffres au début ; 3 ou 5 chiffres à la fin ; vide
Nom	2 à 50 lettres (accents), espace, tiret, apostrophe internes	< 2 ou > 50 ; chiffre ; emoji ; commence ou finit par un séparateur ; injection
Téléphone	+237 puis 9 chiffres, 1er chiffre 6 ou 2	Sans indicatif ; 8 ou 10 chiffres ; commence par 5 ; espaces (?)
E-mail	local@domaine.tld	Sans @ ; deux @ ; domaine sans point ; espaces
Date de naissance	Âge de 16 à 60 ans à la date d'inscription	< 16 ans ; > 60 ans ; date future ; date impossible (31/04)

Le « (?) » est un défaut de spécification
« +237 699 12 34 56 » avec espaces : valide ou non ? À trancher **avant** d'écrire le test.

FIL ROUGE C : NOMS ET TÉLÉPHONES

```
@ParameterizedTest
@CsvSource(quoteCharacter = '"', value = {
    "Fomekong, true", "N'Guessan, true", "Évariste-Junior, true",
    "É, false", "Ali2, false", "Ali😀, false", "\"\", false",
    "\"Robert'); DROP TABLE etudiants;--\", false" })
void nom(String s, boolean ok) {
    assertEquals(ok, Validateur.nom(s));
}

@ParameterizedTest
@CsvSource({ "+237699123456, true",  "+237299123456, true",
             "+2375599123456, false", "699123456, false",
             "+23769912345, false",  "+2376991234567, false" })
void telephone(String s, boolean ok) {
    assertEquals(ok, Validateur.telephone(s));
}
```

`Validateur.nom` **normalise en NFC** puis compte les **points de code** : « José » décomposé (5 `char`) est accepté, « Ali😀 » est refusé.

FIL ROUGE C : LES DATES QUI CASSENT

Date d'inscription fixée au **15/10/2025** (horloge injectée), âge accepté : 16 à 60 ans révolus.

Date de naissance	Âge au 15/10/2025	Attendu	Classe testée
15/10/2009	16 ans (aujourd'hui)	Valide	Limite basse
16/10/2009	15 ans (demain)	Refus	Voisine invalide
16/10/1964	60 ans	Valide	Limite haute
15/10/1964	61 ans	Refus	Voisine invalide
29/02/2008	17 ans	Valide	Né un 29 février
01/11/2026	Futur	Refus	Date future

« Entre 16 et 60 ans » : bornes **incluses** ? Âge **révolu** ? Né un 29/02 : quand a-t-il 16 ans une année non bissextile ? Autant de questions à poser.

TABLES DE DÉCISION : AUTORISER UN TRANSFERT

3 conditions → $2^3 = 8$ combinaisons ; les « – » (indifférent) réduisent la table à **4 règles**.

Conditions et actions	R1	R2	R3	R4
C1 montant valide (100 – 500 000)	N	O	O	O
C2 cumul du jour + montant ≤ 1 000 000	–	N	O	O
C3 solde ≥ montant + frais	–	–	N	O
A1 refus : montant	X			
A2 refus : plafond journalier		X		
A3 refus : solde insuffisant			X	
A4 exécuter le transfert				X

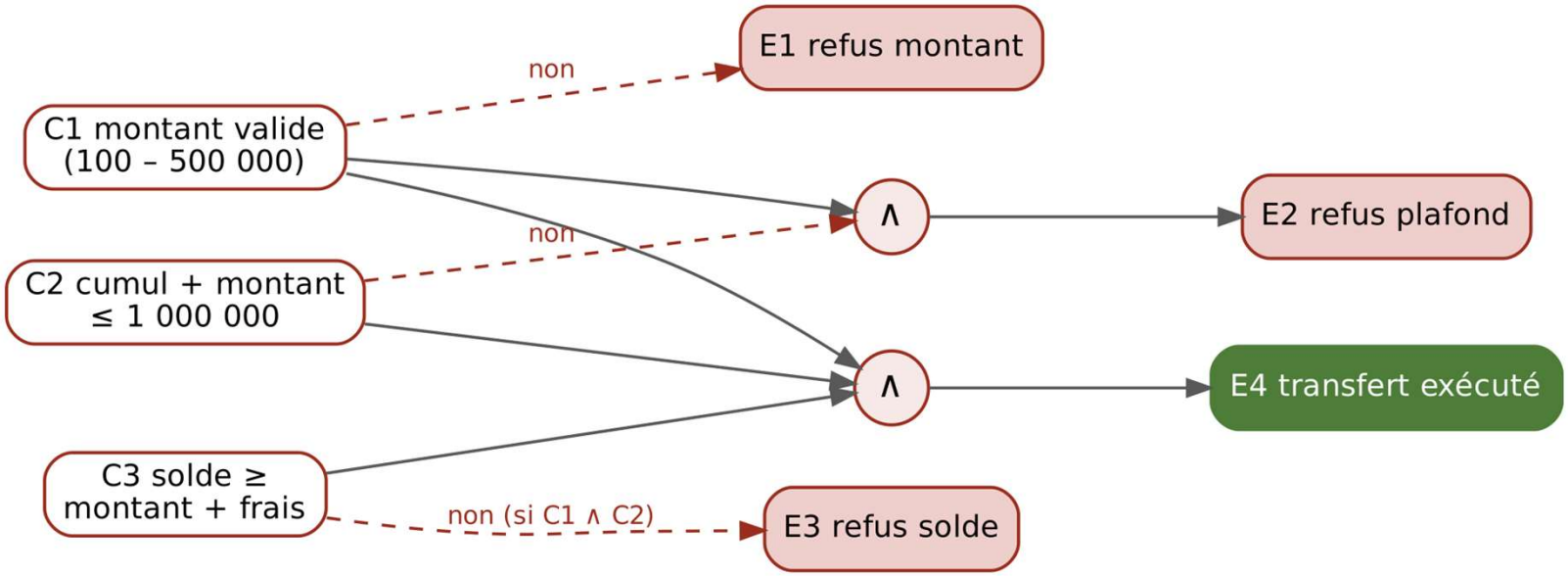
La table révèle une règle cachée : **l'ordre des contrôles**. Plafond et solde tous deux KO → quel message ? Ici : le plafond (R2).

UNE RÈGLE DE LA TABLE = UN CAS DE TEST

```
@ParameterizedTest
@CsvSource({
    // montant, solde,  cumul,  décision attendue
    "99,      1000000, 0,      REFUS_MONTANT",   // R1
    "10000,   1000000, 995000, REFUS_PLAFOND",   // R2
    "10000,   10099,   0,      REFUS_SOLDE",     // R3
    "10000,   10100,   0,      EXECUTE",         // R4
    "10000,   0,       995000, REFUS_PLAFOND" }) // priorité
void tableDeDecision(BigDecimal m, BigDecimal solde,
                     BigDecimal cumul, String d) {
    assertEquals(TransfertService.Decision.valueOf(d),
                 s.autoriser(m, solde, cumul));
}
```

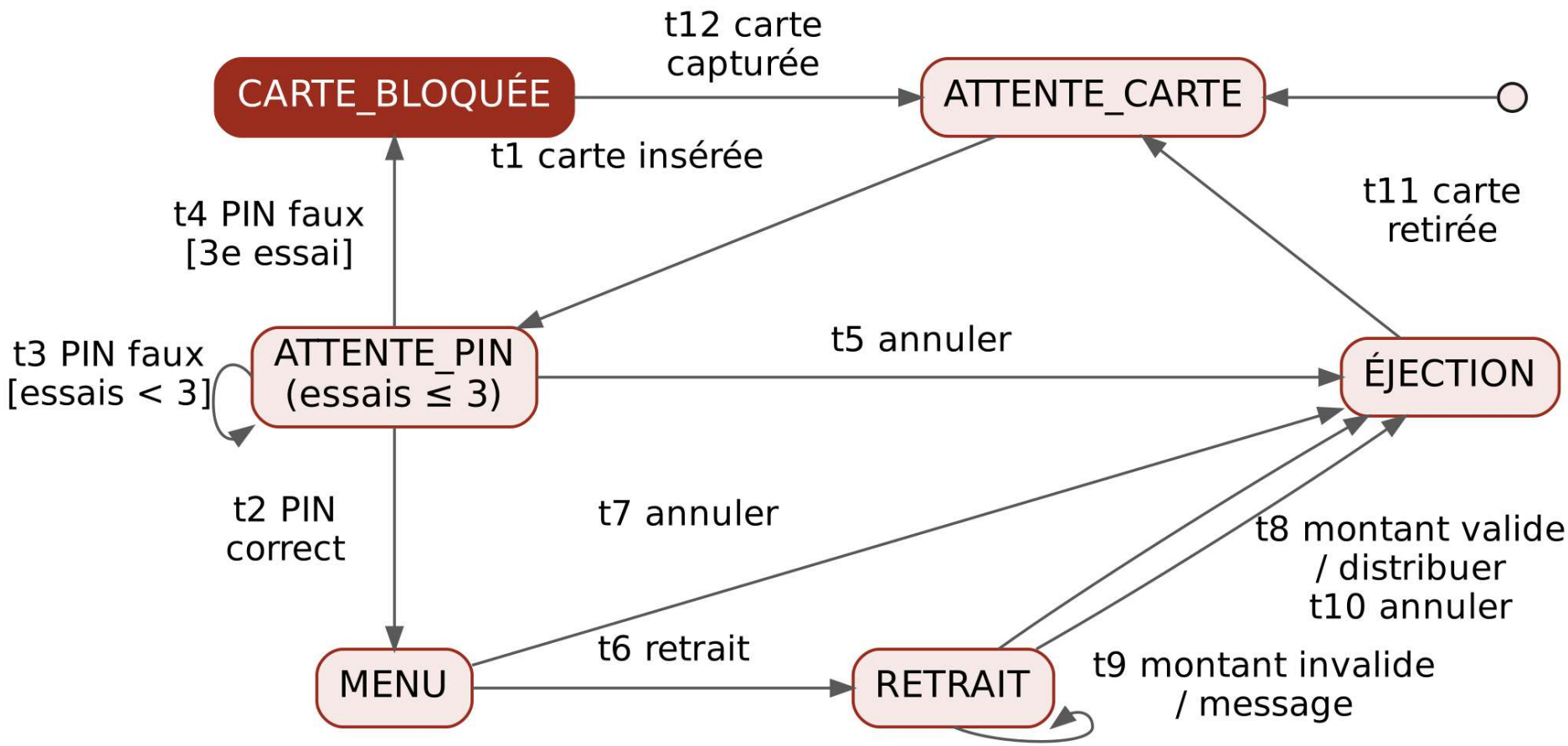
R3 et R4 sont à la **limite** : frais de 10 000 FCFA = 100, donc solde 10 099 refusé et 10 100 accepté. On combine tables de décision **et** valeurs limites.

GRAPHE CAUSE-EFFET



- Méthode de **Myers** : causes (conditions) et effets reliés par des opérateurs logiques (ET, OU, NON)
- Le graphe se **traduit** en table de décision ; utile quand les règles sont nombreuses
- Contraintes entre causes : **E** exclusive, **I** inclusive, **R** requiert, **M** masque

FIL ROUGE D : DIAGRAMME D'ÉTATS DU GAB



6 états, 12 transitions numérotées t1 à t12. Les gardes [essais < 3] et [3e essai] séparent deux transitions sur le **même** événement.

TABLE ÉTATS × ÉVÉNEMENTS

État \ événement	Carte	PIN ok	PIN faux	Retrait	Montant	Annuler	Retirer carte
ATTENTE_CARTE	t1	—	—	—	—	—	—
ATTENTE_PIN	—	t2	t3 / t4	—	—	t5	—
MENU	—	—	—	t6	—	t7	—
RETRAIT	—	—	—	—	t8 / t9	t10	—
ÉJECTION	—	—	—	—	—	—	t11
CARTE_BLOQUÉE	—	—	—	—	—	—	—

33 couples invalides (—)
Chacun doit être **ignoré** ou **rejeté** proprement : insérer une carte en MENU, saisir un montant en ATTENTE_PIN… Ce sont des tests robustes.

COUVERTURE D'UN MODÈLE D'ÉTATS

- **Tous les états :** critère faible
- **Toutes les transitions :** (0-switch) : chaque transition au moins une fois
- **Paires de transitions :** (1-switch) : chaque enchaînement de 2 transitions
- **Transitions invalides :** chaque couple « — » de la table

Cas	Séquence	Couvre
S1	carte, PIN faux, PIN ok, retrait, montant invalide, montant valide, retirer	t1 t3 t2 t6 t9 t8 t11
S2	carte, annuler, retirer	t5
S3	carte, PIN ok, annuler, retirer	t7
S4	carte, PIN ok, retrait, annuler, retirer	t10
S5	carte, PIN faux × 3	t4 t12

5 séquences couvrent les **12 transitions** (0-switch). Le 1-switch demanderait par exemple t3 → t3 (deux PIN faux de suite).

TEST COMBINATOIRE : EACH-CHOICE ET PAIRWISE

- **Each-choice (1-wise) :** chaque valeur de chaque paramètre **au moins une fois** : max $|V_i|$ cas
- **Pairwise (2-wise) :** chaque **paire** de valeurs de deux paramètres au moins une fois
- Fondement empirique : la plupart des défaillances sont déclenchées par **1 ou 2** paramètres (études du NIST, Kuhn et al.)
- Exemple : 3 paramètres binaires A, B, C → exhaustif 2^3 = **8** cas, pairwise **4** cas

Cas	A	B	C
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	0

Vérification : les paires (A,B), (A,C) et (B,C) prennent chacune les 4 valeurs 00, 01, 10, 11. C'est un **tableau orthogonal** $L4(2^3)$.

PAIRWISE SUR UNE APPLICATION WEB ET MOBILE

Navigateur (3) × OS (3) × réseau (3) × langue (2) : **54** cas exhaustifs → **9** cas pairwise (tableau orthogonal L9).

Cas	Navigateur	OS	Réseau	Langue
1	Chrome	Windows	Wi-Fi	fr
2	Chrome	Android	4G	en
3	Chrome	iOS	2G (EDGE)	fr
4	Firefox	Windows	4G	fr
5	Firefox	Android	2G (EDGE)	fr
6	Firefox	iOS	Wi-Fi	en
7	Edge	Windows	2G (EDGE)	en
8	Edge	Android	Wi-Fi	fr
9	Edge	iOS	4G	fr

OUTILS : PICT ET LES CONTRAINTES

```
# modele.txt
Navigateur: Chrome, Firefox, Edge, Safari
OS:         Windows, Android, iOS, macOS
Reseau:     WiFi, 4G, 2G
Langue:     fr, en

IF [Navigateur] = "Safari" THEN [OS] IN {"iOS", "macOS"};

# génération (ordre 2 = pairwise, par défaut)
pict modele.txt /o:2
```

Outils
PICT (Microsoft, libre), **ACTS** (NIST), AllPairs. Les **contraintes** évitent les combinaisons impossibles (Safari sous Windows) ; `/o:3` passe en 3-wise si le risque le justifie.

TEST PAR CAS D'UTILISATION

- Un cas d'utilisation = un **scénario nominal** + des **alternatives** + des **exceptions**
- Au moins **un cas de test par scénario**
- Les données de chaque scénario viennent des techniques précédentes (classes, limites)
- Naturel pour les tests **système** et la **recette**

N **Nominal**

Carte, PIN correct, retrait de 20 000, billets, carte rendue

A1 **Alternative**

PIN faux une fois, puis correct

A2 **Alternative**

Montant non multiple de 5 000 : nouvelle saisie

E1 **Exception**

3 PIN faux : carte capturée

QUELLE TECHNIQUE POUR QUEL SYSTÈME ?

Nature du système	Techniques prioritaires	Fil rouge
Calcul numérique	Classes de réels, limites du type, oracles (séance 3)	A Solveur
Règles métier, argent	Classes, valeurs limites, tables de décision	B Mobile Money
Saisie, texte	Classes par type de chaîne, Unicode, injection	C Formulaire
Système à états	Transitions d'états, cas d'utilisation	D GAB
Configurations multiples	Each-choice, pairwise, contraintes	F API Scolarité

TP DE LA SÉANCE : CONCEVOIR AVANT DE CODER

TP

- **B Mobile Money** : classes, valeurs limites à 3 valeurs, table de décision complète ; tests paramétrés
- **C Formulaire** : classes valides et invalides pour les 5 champs ; une seule classe invalide par cas ; liste des **ambiguïtés** trouvées
- **D GAB** : implémenter l'automate ; tests couvrant toutes les transitions et 5 transitions invalides
- **Pairwise** : modèle PICT de la configuration de l'application Scolarité, avec au moins une contrainte
- Rendu : tableaux de conception (classes, limites, décisions) **avant** le code de test, dans le même dépôt

À RETENIR

1 Partitionner, puis viser les frontières

Classes valides et invalides, entrées et sorties ; une seule classe invalide par cas ; limites à 2 ou 3 valeurs.

2 Chaque type a ses classes

Limites du type (MIN, MAX, NaN, -0.0, 2^{53}) + limites du métier ; Unicode et formats pour les chaînes.

3 Modéliser les règles et les états

Tables de décision pour les combinaisons de conditions ; transitions d'états pour les systèmes à mémoire.

4 Maîtriser la combinatoire

Each-choice, pairwise, contraintes : 54 configurations → 9 cas.

BOÎTE BLANCHE, COUVERTURE ET TEST DE MUTATION

116

Séance 5 · 4 h · Mesurer ce que les tests exécutent… et ce qu'ils détectent

CHAP V

OBJECTIFS DE LA SÉANCE

- Construire le **graphe de flot de contrôle** d'une méthode et calculer sa **complexité cyclomatique**
- Dériver une **base de chemins indépendants** et les données de test associées
- Distinguer les critères de couverture : instructions, décisions, conditions, **MC/DC**, conditions multiples, chemins, flot de données
- Mesurer la couverture avec **JaCoCo** et savoir ce qu'elle ne dit pas
- Évaluer la **capacité de détection** d'une suite avec le **test de mutation** (PIT)
- Renforcer une suite de tests jusqu'à un score de mutation cible

BOÎTE NOIRE ET BOÎTE BLANCHE : DEUX REGARDS COMPLÉMENTAIRES

	Boîte noire (séance 4)	Boîte blanche (séance 5)
Base des tests	Spécification, exigences, modèles	Code source : structure et données
Question posée	Le comportement attendu est-il obtenu ?	Quelles parties du code n'ont jamais été exercées ?
Techniques	Classes, limites, tables de décision, états	Graphe de flot, couvertures, flot de données, mutation
Détecte bien	Fonctionnalités manquantes ou mal comprises	Code mort, branches oubliées, conditions inutiles
Aveugle à	Code non prévu par la spécification	Fonctionnalités **absentes** du code
Moment	Dès que la spécification existe	Quand le code existe (unitaire, intégration)

Démarche recommandée

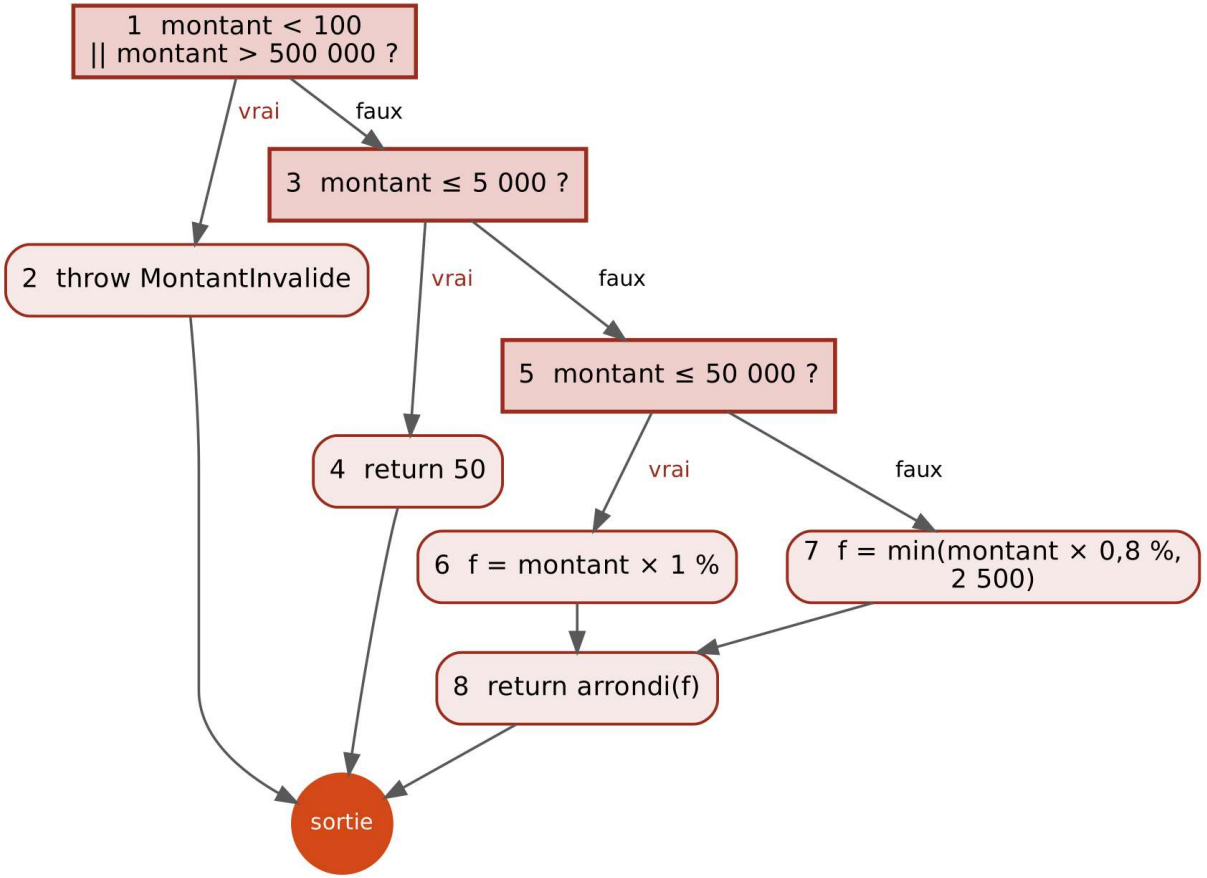
Concevoir d'abord en boîte noire, puis **mesurer** en boîte blanche et compléter les trous. Un test ajouté pour « faire du vert » doit quand même vérifier un résultat attendu.

MÉTHODE ÉTUDIÉE : CALCUL DES FRAIS DE TRANSFERT

```
public BigDecimal frais(BigDecimal montant) {
  if (montant.compareTo(MIN) < 0              // d1
      || montant.compareTo(MAX) > 0)
    throw new MontantInvalideException(montant);
  if (montant.compareTo(SEUIL_1) <= 0)        // d2
    return FORFAIT;                           // 50
  BigDecimal f;
  if (montant.compareTo(SEUIL_2) <= 0)        // d3
    f = montant.multiply(TAUX_1);             // 1 %
  else
    f = montant.multiply(TAUX_2)              // 0,8 %
                .min(PLAFOND);                // 2 500
  return f.setScale(0, RoundingMode.HALF_UP);
}
```

- Trois **décisions** : d1, d2, d3
- d1 est **composée** de deux conditions reliées par `||`
- Montants en FCFA, `BigDecimal` (pas de `double` pour l'argent)
- Spécification : 100–5 000 → 50 ; ≤ 50 000 → 1 % ; au-delà 0,8 % plafonné à 2 500 ; sinon refus

GRAPHE DE FLOT DE CONTRÔLE (GFC)



GFC de frais() : 9 nœuds, 11 arcs

- **Nœud :** bloc d'instructions exécutées en séquence
- **Arc :** transfert de contrôle possible
- **Décision :** nœud à plusieurs arcs sortants
- Un **chemin** va de l'entrée à la sortie
- Les boucles créent des cycles : le nombre de chemins peut devenir infini

COMPLEXITÉ CYCLOMATIQUE DE MCCABE

D

V(G) = E − N + 2P

E = nombre d'arcs, N = nombre de nœuds, P = composantes connexes (1 pour une méthode). V(G) est le nombre de **chemins linéairement indépendants** du graphe (McCabe, 1976).

- Sur frais() : V(G) = 11 − 9 + 2 × 1 = **4**
- Formule équivalente : **nombre de décisions + 1** = 3 + 1 = 4
- Si l'on compte chaque **condition** atomique (d1 en contient 2) : 4 + 1 = **5**, c'est ce que mesurent JaCoCo ou SonarQube sur le bytecode
- Usage : borne inférieure du nombre de tests pour couvrir une base de chemins ; seuil d'alerte courant **V(G) > 10** : méthode à découper

BASE DE CHEMINS INDÉPENDANTS

Chemin	Nœuds parcourus	Donnée	Attendu	Rôle
P1	1 → 2 → sortie	50	refus	montant trop faible
P2	1 → 3 → 4 → sortie	3 000	50	forfait
P3	1 → 3 → 5 → 6 → 8 → sortie	20 000	200	taux 1 %
P4	1 → 3 → 5 → 7 → 8 → sortie	100 000	800	taux 0,8 %

Test structuré (structured testing)

4 chemins = V(G) : chaque autre chemin est une combinaison linéaire de ceux-ci. Cette base couvre toutes les instructions et toutes les décisions… mais aucune **valeur limite** (5 000, 50 000) ni le **plafond** de 2 500. On le paiera à la mutation.

INSTRUCTIONS OU DÉCISIONS ?

Gab.java

```
int fraisRetrait(int montant,
                 boolean horsReseau) {
  int frais = 0;
  if (horsReseau)        // carte d'une
    frais = 500;         // autre banque
  return montant + frais;
}
```

- **Couverture des instructions :** chaque instruction exécutée au moins une fois
  - Un seul test (`horsReseau = true`) suffit : 100 %
- **Couverture des décisions (branches) :** chaque décision prend **vrai et faux**
  - Il faut aussi `horsReseau = false` : la branche « vide » existe
- Un `if` sans `else` est le cas typique où 100 % des instructions laisse une branche non testée

CONDITIONS, DÉCISIONS ET LEURS COMBINAISONS

Décision `if (a || b)` : deux conditions atomiques a et b.

Critère	Exigence	Tests minimaux pour a || b
Décisions	la décision prend vrai et faux	(V,F) et (F,F)
Conditions	chaque condition prend vrai et faux	(V,F) et (F,V) : **la décision vaut toujours vrai !**
Conditions / décisions	les deux exigences précédentes	(V,V) et (F,F)
MC/DC	chaque condition **influence seule** la décision	(F,F), (V,F), (F,V)
Conditions multiples	toutes les combinaisons : 2^n	(V,V), (V,F), (F,V), (F,F)

Piège classique
La couverture des conditions **ne subsume pas** celle des décisions : on peut faire prendre vrai et faux à chaque condition sans jamais emprunter la branche « faux ».

MC/DC : COUVERTURE MODIFIÉE CONDITIONS/DÉCISIONS

D

Modified Condition / Decision Coverage

Chaque point d'entrée et de sortie est exercé, chaque décision prend toutes ses issues, chaque condition prend toutes ses valeurs, **et chaque condition est montrée influençant seule l'issue de sa décision** (les autres conditions fixées).

- Pour **n** conditions : au minimum **n + 1** tests (au plus 2n), contre $\mathbf{2^n}$ pour les conditions multiples
- Pour chaque condition, on cherche une **paire d'indépendance** : deux tests qui ne diffèrent que par cette condition et donnent deux issues différentes
- Prouve qu'aucune condition n'est inutile ou masquée par une autre
- Avec 10 conditions : 11 tests au lieu de 1 024

MC/DC PAS À PAS : AUTORISATION DE RETRAIT

Condition : `pinOk && (montant <= solde || decouvert)`, notée A ∧ (B ∨ C)

#	A pinOk	B montant ≤ solde	C découvert	Décision
1	V	V	V	V
2	V	V	F	**V**
3	V	F	V	**V**
4	V	F	F	**F**
5	F	V	V	F
6	F	V	F	**F**
7	F	F	V	F
8	F	F	F	F

Paires d'indépendance → jeu MC/DC minimal {2, 3, 4, 6}
A : (1,5), (2,6), (3,7) · B : (2,4) · C : (3,4). On garde (2,6) pour A, (2,4) pour B, (3,4) pour C : **4 tests = n + 1** au lieu de 8.

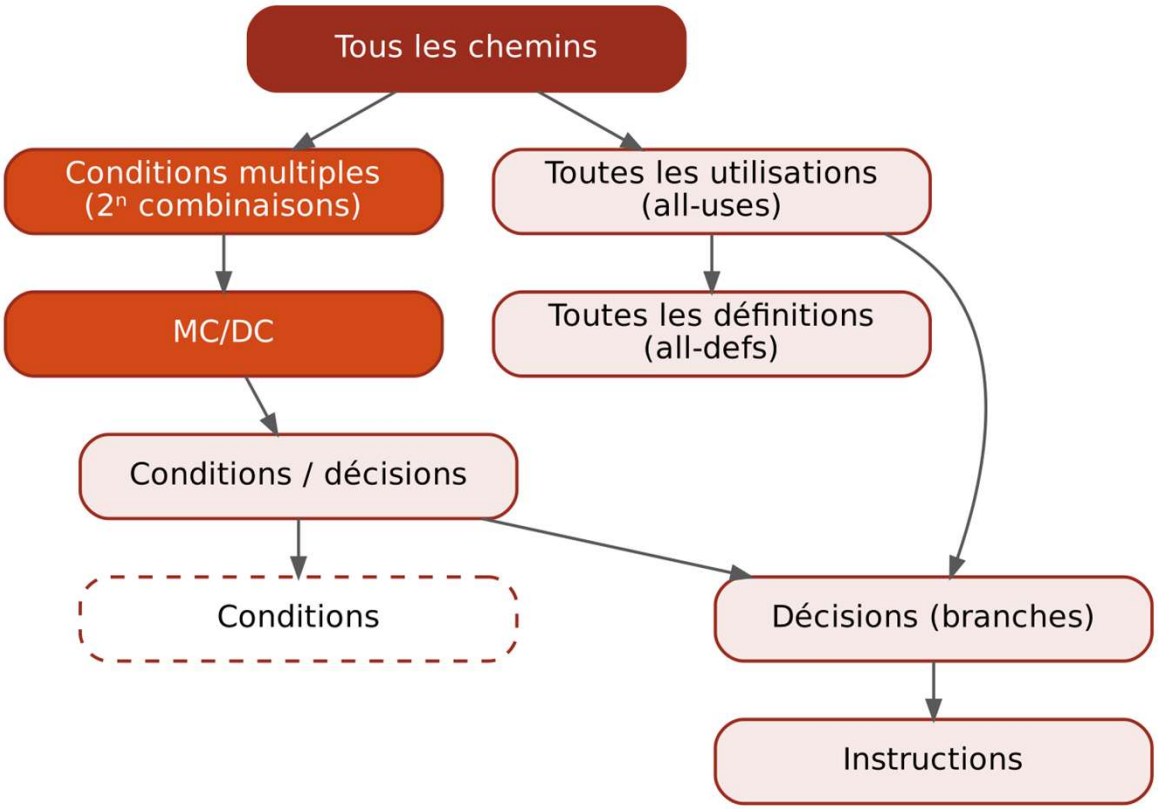
MC/DC DANS L'INDUSTRIE : DO-178C

DO-178C (2011) : norme de certification du logiciel embarqué en aéronautique. Le niveau dépend de l'effet d'une défaillance.

Niveau	Effet d'une défaillance	Couverture structurelle exigée
A	Catastrophique (perte de l'avion)	**MC/DC** + décisions + instructions
B	Dangereux (hazardous)	Décisions + instructions
C	Majeur	Instructions
D	Mineur	Aucune exigence structurelle
E	Sans effet sur la sécurité	Aucune

Même logique ailleurs
ISO 26262 (automobile, ASIL D) recommande fortement MC/DC ; l'EN 50128 (ferroviaire) exige des couvertures croissantes selon le niveau SIL. La couverture exigée **suit le risque**.

HIÉRARCHIE DE SUBSOMPTION DES CRITÈRES



X → Y : toute suite qui satisfait X satisfait Y

- Satisfaire un critère **fort** garantit les critères qu'il subsume
- Les « conditions » seules ne garantissent même pas les décisions
- « Tous les chemins » est le sommet… et souvent **infini** avec des boucles
- Plus on monte, plus le nombre de tests croît : choisir selon le **risque**

COUVERTURE DU FLOT DE DONNÉES

- **Définition (def) :** affectation d'une valeur à une variable
- **Utilisation (use) :** lecture : dans un calcul (c-use) ou une condition (p-use)
- **Paire def-use :** chemin de la def à l'use sans redéfinition
- **all-defs :** chaque def atteint **au moins une** de ses uses
- **all-uses :** chaque def atteint **toutes** ses uses

Variable	Def	Uses (nœuds du GFC)
montant	entrée	p-use 1, 3, 5 ; c-use 2, 6, 7
f	6	c-use 8
f	7	c-use 8

Ce que ça détecte

Variables utilisées sans avoir été définies sur un chemin, définitions jamais lues (code mort), mauvaise valeur propagée. Pour f : all-defs exige les chemins 6→8 **et** 7→8, donc P3 et P4.

MESURER LA COUVERTURE AVEC JACOCO

pom.xml

```xml
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
  <executions>
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>rapport</id>
      <phase>verify</phase>
      <goals><goal>report</goal><goal>check</goal></goals>
      <configuration><rules><rule>
        <element>BUNDLE</element>
        <limits><limit>
          <counter>BRANCH</counter>
          <value>COVEREDRATIO</value>
          <minimum>0.80</minimum>
        </limit></limits>
      </rule></rules></configuration>
    </execution>
  </executions>
</plugin>
```

- `prepare-agent` instrumente le bytecode pendant les tests
- `mvn verify` produit `target/site/jacoco/index.html`
- `check` **fait échouer le build** sous 80 % de branches : une quality gate (séance 10)
- Fonctionne aussi avec Gradle et dans l'IDE

LIRE UN RAPPORT JACOCO

Compteur	Ce qu'il mesure	Remarque
Instructions	Instructions de bytecode exécutées	Le plus fin, indépendant du formatage
Branches	Issues des sauts conditionnels (`if`, `switch`, `&&`, `||`)	Proche des **conditions** du source
Lignes	Lignes source avec au moins une instruction exécutée	Lisible mais grossier
Complexité	Complexité cyclomatique (non) couverte	Chemins de la base restant à tester
Méthodes, classes	Exécutées au moins une fois	Vue d'ensemble

Code couleur dans le source

Vert : ligne entièrement couverte · **Jaune** : partiellement (une branche manque, losange dans la marge) · **Rouge** : jamais exécutée. Les lignes jaunes sont les plus instructives.

100 % DE COUVERTURE, ZÉRO DÉTECTION

PIÈGE

```java
@Test
void couvreTout() {
  var s = new TransfertService();
  assertThrows(MontantInvalideException.class,
      () -> s.frais(fcfa(50)));
  s.frais(fcfa(3_000));
  s.frais(fcfa(20_000));
  s.frais(fcfa(100_000));
  // aucune assertion sur les frais calculés
}
```

- JaCoCo affiche **100 %** d'instructions et de branches
- Remplacer `0.01` par `0.1`, ou `<=` par `<` : le test reste **vert**
- La couverture mesure ce qui est **exécuté**, pas ce qui est **vérifié**
- Elle ne voit pas l'oracle : il faut une autre mesure

Conséquence

Une couverture basse prouve que des tests manquent ; une couverture haute ne prouve rien sur leur qualité. D'où le **test de mutation**.

TEST DE MUTATION : PRINCIPE

On évalue les tests en injectant volontairement des fautes dans le code (séance 6 : injection de fautes à la compilation).



Appliquer une petite modification syntaxique au code : on obtient un **mutant**

Lancer la suite de tests sur chaque mutant

Au moins un test échoue : le mutant est **tué**. Sinon il **survit**

- Survivant : il manque un test…
- … ou le mutant est **équivalent** (même comportement)

Deux hypothèses fondatrices (DeMillo, Lipton, Sayward, 1978)

Programmeur compétent : le code est proche du code correct, les fautes réelles sont de petites erreurs. **Effet de couplage** : une suite qui détecte les fautes simples détecte la plupart des fautes complexes.

OPÉRATEURS DE MUTATION DE PIT (PAR DÉFAUT)

Opérateur	Exemple	Faute simulée
CONDITIONALS_BOUNDARY	`<` → `<=`	Erreur de limite (off-by-one)
NEGATE_CONDITIONALS	`==` → `!=`, `<=` → `>`	Condition inversée
MATH	`*` → `/`, `+` → `-`	Mauvais opérateur arithmétique
INCREMENTS / INVERT_NEGS	`i++` → `i--`, `-x` → `x`	Pas ou signe faux
VOID_METHOD_CALLS	appel supprimé	Effet de bord oublié (journal, notification)
EMPTY / NULL_RETURNS	`return x` → `return ""` / `null`	Résultat non vérifié
PRIMITIVE / TRUE / FALSE_RETURNS	`return x` → `return 0` / `true`	Valeur de retour ignorée

SCORE DE MUTATION ET MUTANTS ÉQUIVALENTS

tués ÷ (générés − équiv.)

Score de mutation : part des fautes injectées que la suite détecte

équivalent

Mutant syntaxiquement différent mais **sémantiquement identique** : aucun test ne peut le tuer

- Décider si un mutant est équivalent est **indécidable** en général : analyse humaine, au cas par cas
- Un survivant non équivalent indique **une entrée ou une assertion manquante**
- Coût : nombre de mutants × durée des tests ; PIT limite le coût en ne lançant que les tests qui couvrent la ligne
- Pratique : viser le score sur le code **critique**, justifier par écrit chaque mutant déclaré équivalent

DÉMO : 7 MUTANTS DE FRAIS() FACE À DEUX SUITES

DÉMO

Mutant	T1 : 4 tests de chemins	T2 : T1 + valeurs limites
M1 `<= 5000` → `< 5000`	survit	survit (**équivalent**)
M2 `<= 50000` → `< 50000`	survit	tué par 50 000 (500 / 400)
M3 décision d2 niée	tué par 3 000	tué
M4 `multiply` → `divide`	tué par 20 000	tué
M5 `||` → `&&` **dans d1**	tué par 50	tué
M6 `< 100` → `<= 100`	survit	tué par 100 (50 / refus)
M7 `.min(PLAFOND)` **supprimé**	survit	tué par 400 000 (2 500 / 3 200)
Score	**3 / 7**	**6 / 6 non équivalents**

Résultats obtenus en exécutant les mutants (code fourni). T2 ajoute 99, 100, 5 000, 5 001, 50 000, 50 001, 312 500, 400 000, 500 000, 500 001 : les **valeurs limites de la séance 4**.

ZOOM : UN MUTANT TUÉ, UN MUTANT ÉQUIVALENT

M2 : ≤ 50 000 devient < 50 000
Seule l'entrée **50 000** distingue le mutant (balayage exhaustif des 499 901 montants entiers de 100 à 500 000) : original 1 % = **500**, mutant 0,8 % = **400**. Un test « au hasard » a moins d'une chance sur 100 000 de le tuer ; un test de **valeur limite** le tue à coup sûr.

M1 : ≤ 5 000 devient < 5 000
Seule l'entrée 5 000 emprunte une autre branche : forfait **50** d'un côté, 1 % × 5 000 = **50** de l'autre. Aucun montant ne distingue les deux versions : le mutant est **équivalent**. La spécification est continue en 5 000, la limite exacte n'a pas d'effet.

Leçon
Les mutants de limite (CONDITIONALS_BOUNDARY) survivent précisément quand les tests évitent les frontières : la mutation **mesure** la qualité de l'analyse des valeurs limites.

METTRE EN PLACE PIT

pom.xml

```xml
<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>1.15.3</version>
  <dependencies>
    <dependency>
      <groupId>org.pitest</groupId>
      <artifactId>pitest-junit5-plugin</artifactId>
      <version>1.2.1</version>
    </dependency>
  </dependencies>
  <configuration>
    <targetClasses>
      <param>cm.mobilemoney.frais.*</param>
    </targetClasses>
    <mutators><mutator>DEFAULTS</mutator></mutators>
    <mutationThreshold>85</mutationThreshold>
  </configuration>
</plugin>
```

- Lancement : `mvn test-compile org.pitest:pitest-maven:mutationCoverage`
- Rapport : `target/pit-reports/index.html`, ligne par ligne, mutants tués / survivants
- `mutationThreshold` fait échouer le build sous 85 %
- Cibler les paquets **critiques** : le calcul des frais, pas les DTO
- Option `withHistory` : analyse incrémentale en CI

COUVERTURE ET MUTATION : COMPLÉMENTAIRES

	Couverture (JaCoCo)	Mutation (PIT)
Question	Quel code est exécuté ?	Quelles fautes sont détectées ?
Voit l'oracle ?	Non	Oui : un test sans assertion ne tue rien
Coût	Faible (une exécution)	Élevé (une exécution par mutant)
Bon usage	Repérer le code jamais testé	Mesurer la force des tests du code critique
Limite	Rassurante à tort	Mutants équivalents à analyser

En pratique
Couverture de branches en quality gate sur tout le projet, score de mutation sur les modules à risque (calculs financiers, sécurité, règles métier).

TP : RENFORCER LA SUITE DU FIL ROUGE B

TP

- **1.** Implémenter `TransfertService.frais` (et le plafond journalier de 1 000 000 FCFA) avec des tests JUnit 5 issus de la séance 4
- **2.** Mesurer avec JaCoCo : atteindre **100 % des branches** ; dessiner le GFC et vérifier V(G)
- **3.** Lancer PIT : relever le score, lister les survivants
- **4.** Pour chaque survivant : ajouter le test qui le tue **ou** justifier par écrit qu'il est équivalent
- **5.** Objectif : score de mutation **≥ 85 %** hors équivalents justifiés ; seuils JaCoCo et PIT dans le `pom.xml`
- **Livrable** (dépôt Git) : code, tests, rapports JaCoCo et PIT, tableau des survivants analysés

À RETENIR

1 Structure

GFC et V(G) = E − N + 2P = décisions + 1 : nombre de chemins indépendants, donc de tests d'une base.

2 Critères

Instructions ⊂ décisions ⊂ conditions/décisions ⊂ MC/DC ⊂ conditions multiples. Les conditions seules ne garantissent pas les décisions.

3 MC/DC

Chaque condition influence seule la décision : n + 1 tests minimum, exigé au niveau A de DO-178C.

4 Couverture

Mesure ce qui est exécuté, pas ce qui est vérifié : 100 % de branches peut ne rien détecter.

5 Mutation

Injecter des fautes pour évaluer les tests. Score = tués / (générés − équivalents).

6 Lien boîte noire

Les survivants révèlent les valeurs limites et les oracles manquants.

TECHNIQUES DE VÉRIFICATION DES CALCULS

142

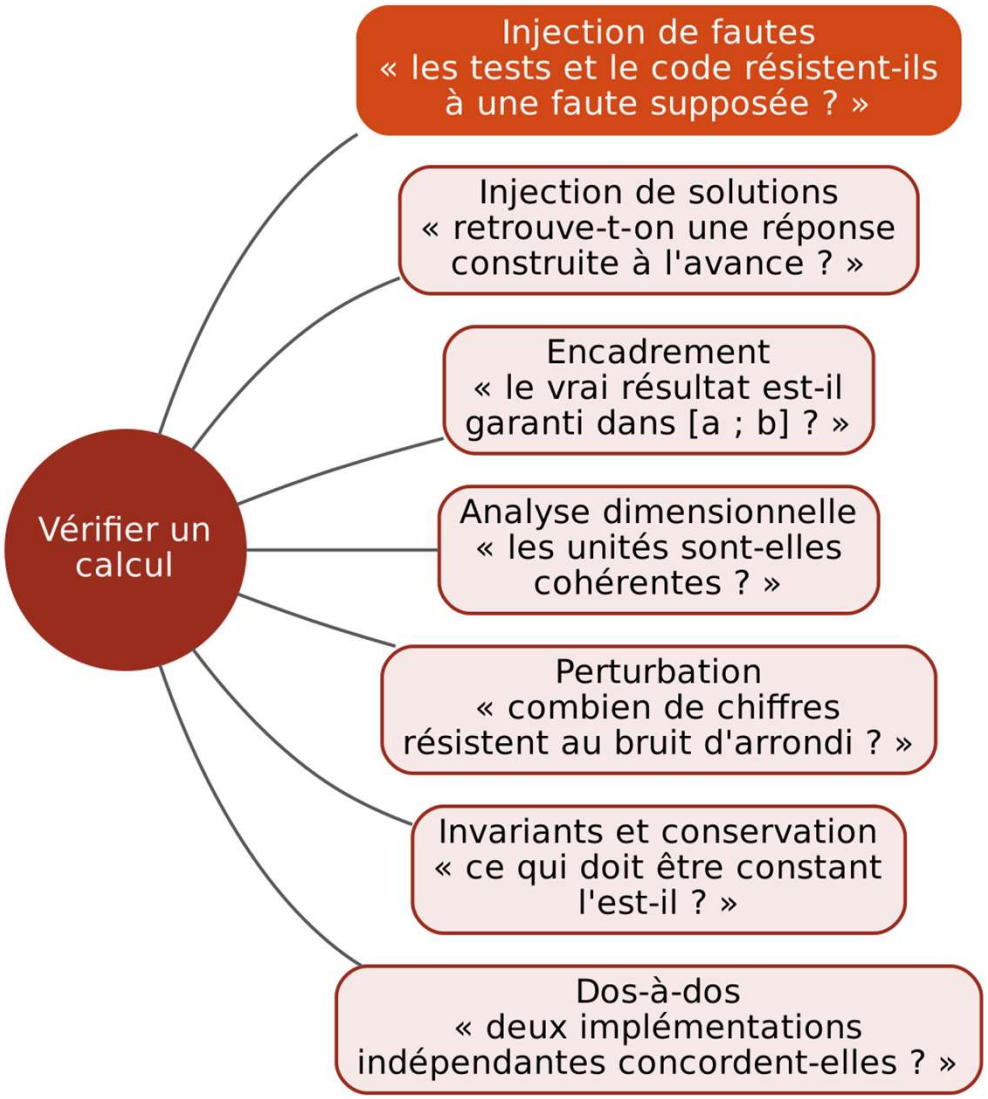
Séance 6 · 4 h · Injection, encadrement, analyse dimensionnelle, perturbation

CHAP VI

OBJECTIFS DE LA SÉANCE

- Concevoir une campagne d'**injection de fautes** : modèle de faute, point d'injection, comportement attendu
- Construire un oracle exact par **injection de solutions** (solutions manufacturées) et vérifier un **ordre de convergence**
- **Encadrer** un résultat par arithmétique d'intervalles et interpréter la largeur obtenue
- Détecter les incohérences d'**unités** par analyse dimensionnelle et typage des grandeurs
- Estimer le nombre de **chiffres exacts** d'un résultat par perturbation des arrondis
- Choisir la technique adaptée à un calcul donné

SEPT TECHNIQUES, SEPT QUESTIONS



- Chaque technique répond à **une question précise** sur le calcul
- Elles fournissent des **oracles** quand le résultat exact est inconnu (séance 3)
- Elles se **combinent** : on injecte une solution, on encadre le résultat, on vérifie les unités
- L'injection de fautes s'applique à **tout logiciel**, pas seulement au calcul scientifique

ANALYSE PAR INJECTION DE FAUTES

D

Injection de fautes (fault injection)

Introduire **délibérément** une faute dans le code, les données ou l'environnement pour observer si le système (ou la suite de tests) la **détecte, la tolère ou la confine**.

- **Hypothèse de faute :** quelles fautes le système est censé supporter (panne d'un service, message perdu, valeur corrompue…)
- **Modèle de faute :** **type** (exception, latence, corruption), **lieu** (composant, interface), **moment**, **durée** : transitoire, intermittente ou permanente
- **Comportement attendu :** l'oracle : erreur propre, repli, reprise, invariant préservé, alerte émise
- **Deux cibles :** évaluer **les tests** (mutation, séance 5) ou évaluer **la robustesse du système**

INJECTER À LA COMPILATION OU À L'EXÉCUTION

	À la compilation	À l'exécution
Ce qu'on modifie	Le code source ou le bytecode	L'environnement, les données, les messages, la mémoire
Ce qu'on évalue	La **suite de tests** : détecte-t-elle la faute ?	Le **système** : tolère-t-il la faute ?
Exemples	Mutation (< → <=), suppression d'appel	Exception, latence, coupure réseau, disque plein, bit-flip
Outils	PIT, mutmut (Python), Stryker (JS)	Mockito, Toxiproxy, Byteman, tc netem, Chaos Monkey
Niveau	Unitaire	Intégration, système, production

Même idée, deux usages
On suppose une faute plausible et on vérifie qu'elle ne passe pas inaperçue : pour la suite de tests, ou pour l'utilisateur.

CATALOGUE DES FAUTES À INJECTER

Faute	Exemple concret	Comportement attendu à vérifier
Exception d'un service	La base de comptes lève une erreur au crédit	Annulation complète, aucun argent créé ni perdu
Latence, timeout	L'API de notes répond en 3 s	Délai borné, réponse 503 propre, pas de blocage
Coupure réseau	Connexion fermée en plein transfert	Reprise idempotente, pas de double débit
Corruption de données	Montant négatif, champ tronqué, JSON invalide	Rejet à la frontière, message clair
Ressource épuisée	Disque plein, pool de connexions vide	Dégradation contrôlée, alerte émise
Bit-flip	Un bit de la mémoire change (rayonnement)	Détection par contrôle de vraisemblance ou redondance
Horloge	Heure décalée, changement de jour à minuit	Plafond journalier calculé sur le bon jour

UN SEUL BIT PEUT TOUT CHANGER

DÉMO

Valeur `9.81` (double IEEE 754) après inversion d'un seul bit (`Double.doubleToRawLongBits`, exécuté) :

9.809999999999999	**13.81**	**$5{,}46 \cdot 10^{-308}$**	**−9.81**
bit 0 (mantisse, poids faible) : erreur invisible	bit 51 (mantisse, poids fort)	bit 62 (exposant)	bit 63 (signe)

- Sur un `int` : un solde de 150 000 devient **1 073 891 824** (bit 30) ou **−2 147 333 648** (bit 31)
- Défenses : mémoire ECC, **contrôles de vraisemblance** ($g \in [9{,}7\ ;\ 9{,}9]$, solde ≤ plafond), redondance et vote
- L'injection de bit-flips sert à valider ces défenses dans les systèmes critiques (spatial, ferroviaire)

INJECTER UNE PANNE DANS UN TEST UNITAIRE

```java
@Test
void unePanneNeCreeNiNeDetruitDArgent() {
  Comptes comptes = spy(new ComptesMemoire());
  comptes.crediter("A", fcfa(100_000));
  var service = new TransfertService(comptes);
  // injection : le crédit du destinataire échoue
  doThrow(new IllegalStateException("base KO"))
      .when(comptes).crediter(eq("B"), any());
  BigDecimal avant = total(comptes);

  assertThrows(IllegalStateException.class,
      () -> service.transferer("A", "B",
                               fcfa(20_000)));

  assertEquals(0, avant.compareTo(total(comptes)),
      "invariant de conservation violé");
}
```

- `spy` + `doThrow` (Mockito) : la doublure **injecte** la faute à un point précis
- L'oracle n'est pas « pas d'exception » mais un **invariant métier** : émetteur + destinataire + frais collectés = constante
- Répéter pour **chaque appel** externe (débit, crédit, frais) : une faute par point d'injection

RÉSULTAT : LA CORRECTION ELLE-MÊME ÉTAIT FAUSSE

DÉMO

Transfert de 20 000 FCFA (frais 200) depuis A = 100 000. Panne injectée au 1er puis au 2e crédit (code exécuté) :

Version	Panne injectée	Total après	Invariant
Sans compensation	crédit de B	79 800	**violé** : −20 200
Sans compensation	crédit des frais	99 800	**violé** : −200
Avec compensation	crédit de B	100 000	respecté
Avec compensation	crédit des frais	120 000	**violé** : +20 000 créés

Leçon

La compensation recrédite A mais oublie d'annuler le crédit déjà fait à B : de l'argent est **créé**. Un seul test de panne (au 1er appel) l'aurait déclarée correcte. Il faut injecter **à chaque point** et vérifier l'invariant, ou utiliser une vraie transaction.

INJECTER LATENCE ET COUPURES RÉSEAU : TOXIPROXY

```
@Container
static ToxiproxyContainer toxiproxy =
    new ToxiproxyContainer(
        "ghcr.io/shopify/toxiproxy:2.5.0");

@Test
void notesRepondEn503SiLaBaseEstLente() throws Exception {
  Proxy proxy = client.createProxy("pg",
      "0.0.0.0:8666", "postgres:5432");
  // injection : 3 s de latence vers la base
  proxy.toxics().latency("lent",
      ToxicDirection.DOWNSTREAM, 3_000);

  given().when().get("/etudiants/42/notes")
     .then().statusCode(503)
     .time(lessThan(2_000L));      // délai borné
}
```

- Toxiproxy (Shopify) : proxy TCP placé entre l'API et la base
- « Toxiques » : `latency`, `timeout`, `bandwidth`, `reset_peer`, `slow_close`, `limit_data`
- Oracle : **délai borné** et code 503, pas de requête bloquée ni de thread épuisé
- Testcontainers fournit la base et le proxy (séance 12)

CHAOS ENGINEERING : INJECTER EN PRODUCTION

Chaos Monkey (Netflix, 2011) arrête au hasard des instances en production. Démarche formalisée sur principlesofchaos.org :

Régime stable	Hypothèse	Injecter	Comparer	Limiter
Définir une métrique métier : ≥ 99,5 % de requêtes `/notes` réussies, p95 < 300 ms	Le régime stable se maintient malgré la faute	Événements réels : instance arrêtée, latence, zone indisponible	Groupe expérimental contre groupe témoin	▪ **Rayon d'impact** minimal ▪ Arrêt d'urgence ▪ Automatiser

Prérequis
Observabilité (métriques, traces), arrêt immédiat de l'expérience, accord de l'exploitation. On commence en préproduction, jamais par la production.

ANALYSE PAR INJECTION DE SOLUTIONS

D

Méthode des solutions manufacturées (MMS)

On **choisit d'abord la solution**, on calcule le problème (coefficients, terme source, conditions initiales) dont elle est la solution exacte, on le fait résoudre par le programme et on compare. Formalisée par Roache, Salari et Knupp (Sandia, 2000) pour la vérification des codes de calcul.

1 Choisir

une solution u* simple et régulière

2 Fabriquer

le problème P(u*) = données

3 Résoudre

u = programme(données)

4 Comparer

$\|u - u^*\| \leq$ tolérance, et son évolution avec le pas

INJECTION DE RACINES DANS LE SOLVEUR

On choisit (r_1, r_2), on fabrique $a = 1$, $b = -(r_1 + r_2)$, $c = r_1 \cdot r_2$, puis on résout. Erreur relative maximale mesurée :

Racines injectées	Coefficients fabriqués	Formule naïve	Formule stable
1 et 2	$b = -3, c = 2$	0	0
-10^{-8} et -10^{8}	$b \approx 10^{8}, c = 1$	**0,49** ($-1{,}49 \cdot 10^{-8}$)	0
10^{-3} et 10^{3}	$b \approx -1\,000{,}001, c = 1$	$2{,}4 \cdot 10^{-11}$	0
3 et 3,000001	$b \approx -6{,}000001, c \approx 9{,}000003$	$1{,}6 \cdot 10^{-10}$	$1{,}6 \cdot 10^{-10}$

Lecture

L'oracle est exact : on sait ce qu'on a injecté. La 2e ligne révèle l'élimination catastrophique ; la dernière, un problème **mal conditionné** (racines presque doubles) où même la formule stable perd 6 chiffres : la tolérance doit refléter le conditionnement.

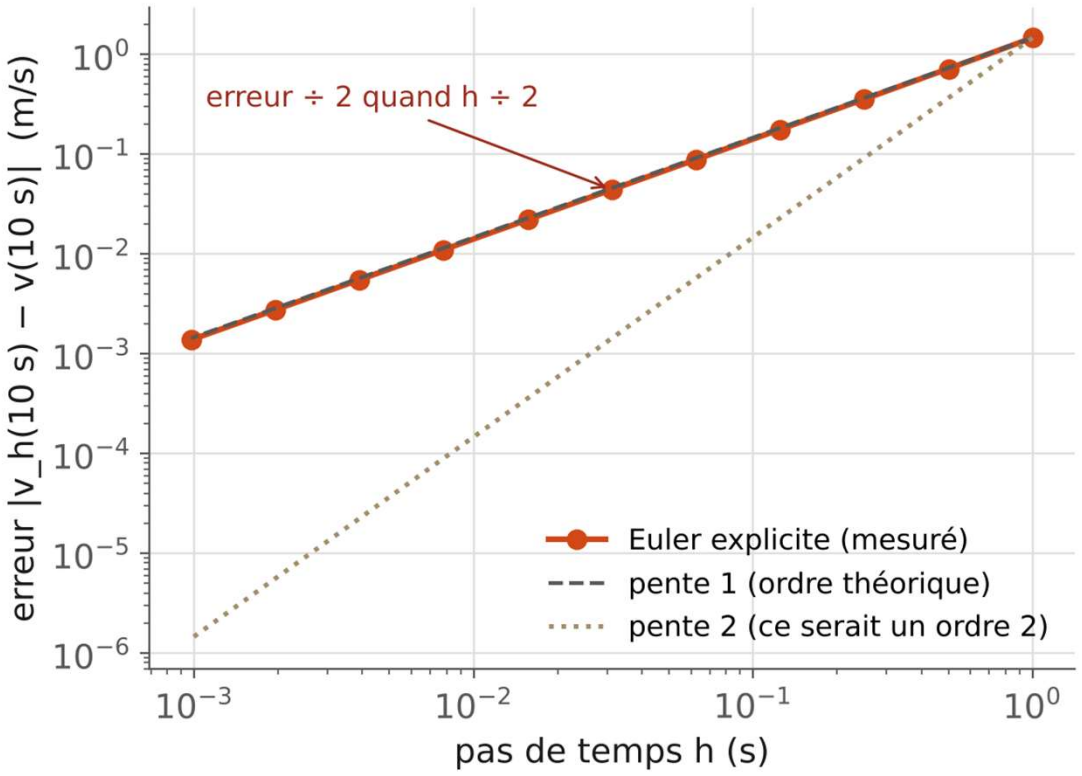
SOLUTION MANUFACTURÉE POUR UNE ÉQUATION DIFFÉRENTIELLE

- Modèle : $v' = g - (k/m)\cdot v$
- On **choisit** $v^*(t) = t^2$ (sans lien avec la physique !)
- On **fabrique** le terme source : $S(t) = v^{*\prime} - g + (k/m)\cdot v^* = 2t - g + (k/m)\cdot t^2$
- On résout $v' = g - (k/m)\cdot v + S(t)$, $v(0) = 0$
- Le résultat doit tendre vers $\mathbf{t^2}$ quand $h \to 0$
- Autre oracle injecté : sans frottement, $x(t) = \frac{1}{2}\cdot g\cdot t^2$; Euler donne une erreur **exactement** $g\cdot T\cdot h/2$

MMS

```
double v = 0, t = 0;
for (int i = 0; i < n; i++) {
  double s = 2*t - G + K/M*t*t;
  v += h * (G - K/M*v + s);
  t += h;
}
double err = Math.abs(v - t*t);
```

VÉRIFIER L'ORDRE DE CONVERGENCE D'EULER



v(10 s), m = 80 kg, k = 15 kg/s, comparée à la solution exacte

h (s)	erreur (m/s)	ordre
1	1,464	–
0,5	0,718	1,03
0,25	0,356	1,01
0,125	0,177	1,01
0,0039	$5{,}5 \cdot 10^{-3}$	1,000
0,00098	$1{,}4 \cdot 10^{-3}$	1,000

Ordre observé p = $\log_2$(erreur(h) / erreur(h/2))
Euler explicite est d'ordre 1 : on mesure **1,000**. Un ordre observé différent de l'ordre théorique signale un bug (pas oublié dans un terme, source mal codée) même quand les résultats « ont l'air bons ».

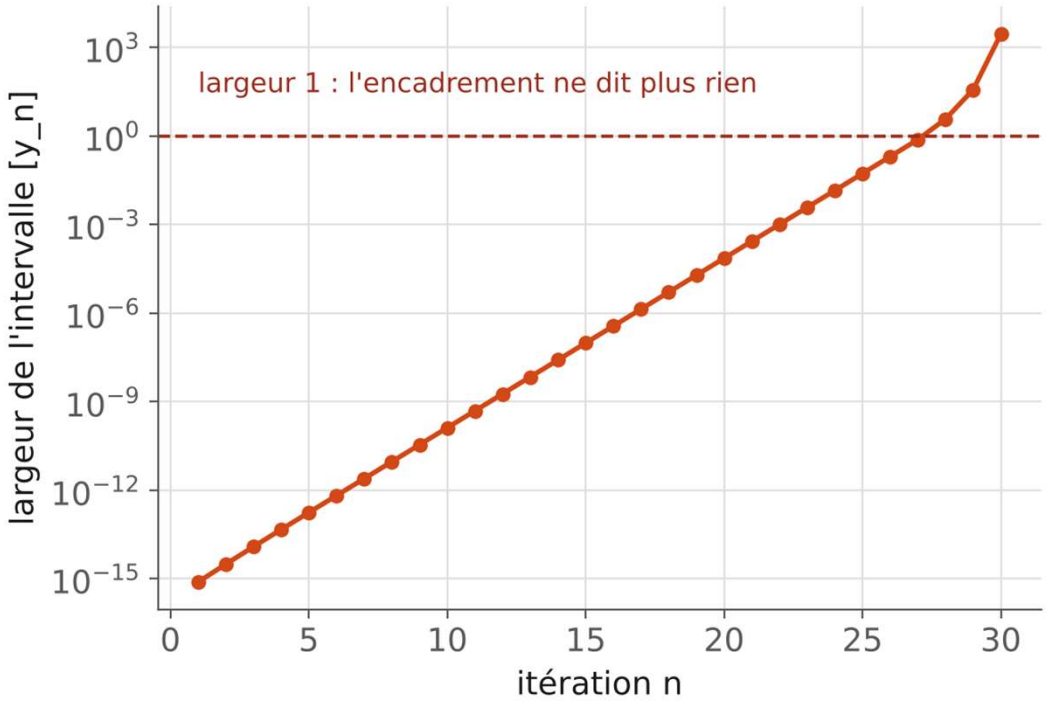
ANALYSE PAR ENCADREMENT : ARITHMÉTIQUE D'INTERVALLES

- Chaque valeur est un intervalle **[x] = [inf ; sup]** qui contient sûrement la vraie valeur (Moore, 1966)
- [a] + [b] = [a.inf + b.inf ; a.sup + b.sup], idem pour −, ×, ÷
- **Arrondis dirigés** : borne inf vers −∞, borne sup vers +∞ (modes de l'IEEE 754)
- Java n'expose pas ces modes : on élargit d'**un ulp** avec `nextDown` / `nextUp`, ce qui reste sûr
- Garantie : le résultat exact est **dans** l'intervalle final

I.java

```
record I(double lo, double hi) {
  I add(I o) {
    return new I(
      Math.nextDown(lo + o.lo),
      Math.nextUp(hi + o.hi));
  }
  I sub(I o) {
    return new I(
      Math.nextDown(lo - o.hi),
      Math.nextUp(hi - o.lo));
  }
  // mul : min et max des 4 produits
}
```

QUAND L'INTERVALLE S'ÉLARGIT



Suite logistique $y \leftarrow 3{,}75 \cdot y \cdot (1 - y)$, $y_0 = 0{,}5$

- **Dépendance :** x = [0,1] arrondi : x − x = $[-2{,}8 \cdot 10^{-17} ; 2{,}8 \cdot 10^{-17}]$, pas 0
- **Itérations :** largeur $1{,}3 \cdot 10^{-10}$ (n = 10), $7{,}1 \cdot 10^{-5}$ (n = 20), 0,053 (n = 25), 2 833 (n = 30)
- Un intervalle **étroit prouve** que le résultat est juste
- Un intervalle **large** ne prouve pas qu'il est faux : il signale instabilité ou surestimation

ENCADRER LA PETITE RACINE DU SOLVEUR

$a = 1, b = 10^8, c = 1$ calculé entièrement en arithmétique d'intervalles (exécuté). Vraie racine $\approx -1{,}0 \cdot 10^{-8}$:

Formule	Intervalle obtenu	Largeur
Naïve	$[-3{,}73 \cdot 10^{-8}$; $\mathbf{+1{,}49 \cdot 10^{-8}}]$	$5{,}2 \cdot 10^{-8}$
Stable	$[-1{,}0000000000000007 \cdot 10^{-8}$; $-9{,}999999999999995 \cdot 10^{-9}]$	$1{,}2 \cdot 10^{-23}$

Un oracle automatique

Avec la formule naïve, l'intervalle **contient 0** : on ne connaît même pas le signe de la racine, et le programme le dit tout seul. Avec la formule stable, 15 chiffres sont garantis. Test possible : `assertTrue(largeur(r) <= 1e-12 * Math.abs(milieu(r)))`.

ENCADRER SANS EXÉCUTER : ANALYSE DE BORNES

```c
/*@ requires 0 <= a && 0 <= b; */
int milieu(int a, int b) {
  return (a + b) / 2;
}

// Alarme émise par Frama-C / Eva :
/*@ assert Eva: signed_overflow:
      a + b <= 2147483647; */

int milieuSur(int a, int b) {
  return a + (b - a) / 2;  // OK
}
```

- **Analyse de bornes** (value range analysis) : calculer l'intervalle de chaque variable pour **toutes** les entrées possibles
- $a, b \in [0 ; 2^{31}-1] \Rightarrow a + b \in [0 ; 2^{32}-2]$: dépasse `INT_MAX`, **alarme**
- Avec $a + (b - a)/2$: chaque opération reste dans les bornes, aucune alarme
- C'est le bug de la recherche dichotomique vu en séance 2
- Fondement : l'**interprétation abstraite** (séance 11)

MARS CLIMATE ORBITER (1999)

CAS RÉEL

Contexte
Sonde de la NASA lancée le 11 décembre 1998 pour étudier le climat et l'atmosphère de Mars, et servir de relais au futur atterrisseur.

Défaillance
Perdue le 23 septembre 1999 à l'insertion en orbite : elle est passée à environ 57 km d'altitude au lieu d'une orbite sûre et s'est désintégrée.

Cause technique
Un logiciel au sol (Lockheed Martin) fournissait les impulsions des propulseurs en **livre-force·seconde** ; la navigation (JPL) les lisait en **newton·seconde**. Facteur 4,45 sur chaque correction de trajectoire.

LEÇON V&V

Les unités font partie du **type** d'une donnée. Une interface qui échange des nombres nus est une faute en attente. Analyse dimensionnelle, typage des grandeurs et tests d'interface l'auraient détectée.

HOMOGÉNÉITÉ DES ÉQUATIONS DU FIL ROUGE

Dimensions de base : M (masse), L (longueur), T (temps). Une équation est fausse si ses deux membres n'ont pas la même dimension.

Expression	Dimensions	Verdict
$v' = g - (k/m)\cdot v$	$L\cdot T^{-2} = L\cdot T^{-2} - (M\cdot T^{-1} / M)\cdot L\cdot T^{-1}$	homogène
$x = \frac{1}{2}\cdot g\cdot t^2$	$L = L\cdot T^{-2}\cdot T^2$	homogène
$x = \frac{1}{2}\cdot g\cdot t$	$L \neq L\cdot T^{-1}$	**erreur**
v(t) = (m·g/k)(1 − e^(−k·t/m))	argument de exp : $M\cdot T^{-1}\cdot T / M = 1$	homogène
$E = \frac{1}{2}\cdot m\cdot v^2 + m\cdot g\cdot h$	$M\cdot L^2\cdot T^{-2}$ pour chaque terme	homogène
$t = \sqrt{(2h/g)}$	$\sqrt{(L / L\cdot T^{-2})} = T$	homogène

Règles

On n'additionne que des grandeurs de même dimension ; l'argument d'une exponentielle, d'un logarithme ou d'un sinus est **sans dimension**.

TYPER LES GRANDEURS EN JAVA (JSR 385)

Indriya

```java
import javax.measure.Quantity;
import javax.measure.quantity.*;
import tech.units.indriya.quantity.Quantities;
import static tech.units.indriya.unit.Units.*;

Quantity<Length> h =
    Quantities.getQuantity(45, METRE);
Quantity<Acceleration> g = Quantities
    .getQuantity(9.81, METRE_PER_SQUARE_SECOND);

Quantity<?> t2 = h.multiply(2).divide(g);
System.out.println(t2.getUnit());   // s²

// h.add(g);  ne compile pas :
//   Quantity<Length> ≠ Quantity<Acceleration>
```

- JSR 385 (Units of Measurement API 2.0), implémentation de référence **Indriya**
- Le **compilateur** refuse d'additionner des grandeurs de dimensions différentes
- Les conversions (pieds, livres) sont explicites : `h.to(FOOT)`
- Coût : objets au lieu de `double` ; réserver aux **interfaces** et aux calculs critiques

PYTHON, F# ET CHANGEMENT D'UNITÉS

Pint (exécuté)

```python
import pint
u = pint.UnitRegistry()
h = 45 * u.meter
g = 9.81 * u.meter / u.second**2
t = (2 * h / g) ** 0.5    # 3.0289126640769135 second
h + g     # DimensionalityError: Cannot convert from 'meter' ([length])
          #   to 'meter / second ** 2' ([length] / [time] ** 2)
```

- **F#** : `[<Measure>] type m` et `let g = 9.81<m/s^2>` ; `h + g` est une **erreur de compilation**, sans coût à l'exécution
- **Relation métamorphique** : changer d'unités ne doit pas changer la physique. En pieds : h = 147,64 ft, g = 32,185 ft/s² ⇒ t = **3,0289126640769135 s**, identique (exécuté)
- Test : `assertEquals(t(h, g), t(h.to(FOOT), g.to(FOOT_PER_S2)), tol)` détecte toute constante codée en dur dans une unité

ANALYSE PAR PERTURBATION : ARITHMÉTIQUE STOCHASTIQUE

D

CESTAC (Vignes, années 1970) et CADNA

Chaque opération flottante est arrondie **aléatoirement** vers le haut ou vers le bas. On exécute N = 3 fois : les chiffres communs aux 3 résultats sont **exacts**, les autres sont du bruit d'arrondi.

- Nombre de chiffres significatifs exacts : $\mathbf{C = \log_{10}(\sqrt{N} \cdot |\,moyenne\,| \;/\; (\tau \cdot \sigma))}$, τ = 4,303 (Student, N = 3, 95 %)
- **CADNA** (LIP6, Sorbonne Université) : bibliothèque C/C++/Fortran qui implémente CESTAC et signale les instabilités
- **Verificarlo** : arithmétique de Monte-Carlo par instrumentation LLVM, sans modifier le code
- Avantage : ni solution exacte, ni réécriture en intervalles ; on mesure la **confiance** dans chaque résultat

PERTURBER LES ARRONDIS DU SOLVEUR

DÉMO

Petite racine de $x^2 + b \cdot x + 1 = 0$, chaque opération arrondie au hasard à ± 1 ulp, 3 exécutions (code/s06/cestac.py) :

b	Naïve : 3 résultats	Naïve : chiffres exacts	Stable
10^2	$-1{,}000100 \cdot 10^{-2}$ (×3)	≈ 11,8	≈ 15,3
10^4	$-1{,}000000 \cdot 10^{-4}$ (×3)	≈ 7,9	≈ 14,9
10^6	$-1{,}000066$ / $-1{,}000066$ / $-1{,}000008 \cdot 10^{-6}$	≈ 4,1	≈ 15,3
10^8	$-9{,}9 \cdot 10^{-324}$ / $7{,}45 \cdot 10^{-9}$ / $-2{,}98 \cdot 10^{-8}$	**aucun**	≈ 15,1

Lecture

La formule naïve perd environ **$2 \cdot \log_{10}(b)$** chiffres ; à $b = 10^8$ le résultat n'a même plus de signe fiable. La perturbation le détecte **sans connaître la vraie racine** : c'est un oracle de précision.

INVARIANTS, CONSERVATION ET CAS LIMITES

B Invariant métier

- Émetteur + destinataire + frais collectés = constante
- Solde ≥ 0, cumul du jour ≤ 1 000 000 FCFA
- Vérifié après **chaque** transfert, y compris en panne

E Loi de conservation

- Sans frottement : $E = \frac{1}{2} \cdot m \cdot v^2 + m \cdot g \cdot h$ constante
- Avec Euler explicite, la dérive de E doit être en O(h)
- Avec frottement : E décroît, jamais ne croît

A Cas limites connus

- $c = 0 \Rightarrow$ racines 0 et $-b/a$
- $k \to 0 \Rightarrow$ on retrouve la chute libre
- $t \to \infty \Rightarrow v \to m \cdot g/k =$ 52,32 m/s ($m = 80$, $k = 15$)

Principe commun

On vérifie une **propriété** que le résultat exact doit satisfaire, sans connaître ce résultat : ce sont des oracles par propriétés (séance 3) appliqués au calcul.

TESTS DOS-À-DOS : DOUBLE CONTRE BIGDECIMAL

DÉMO

Frais TTC = frais HT × 1,1925 (TVA 19,25 %), arrondis au FCFA. Deux implémentations indépendantes, comparées sur 200 000 montants (exécuté) :

HT	double : ht × 1,1925	arrondi double	BigDecimal
200	238.49999999999997	238	**239**
600	715.4999999999999	715	**716**
1 400	1669.4999999999998	1 669	**1 670**
2 200	2623.4999999999995	2 623	**2 624**

367 divergences sur 200 000

Les deux versions « marchent » sur des exemples pris au hasard. Le dos-à-dos les fait diverger exactement sur les demi-unités. Autre mesure : 10^6 additions de 0,1 en `double` donnent **100000.00000133288**, en `BigDecimal` 100000.0.

QUELLE TECHNIQUE POUR QUEL PROBLÈME ?

Technique	Quand l'utiliser	Outils	Fil rouge
Injection de fautes	Évaluer une suite de tests ; vérifier la tolérance aux pannes	PIT, Mockito, Toxiproxy, Chaos Monkey	B, F
Injection de solutions	Pas de solution exacte connue ; vérifier un ordre de convergence	Code de test dédié	A, E
Encadrement	Garantir un résultat ; détecter instabilité ou débordement	Intervalles, Frama-C Eva, Astrée	A
Analyse dimensionnelle	Formules physiques, interfaces entre équipes	JSR 385, Pint, F#	E
Perturbation	Estimer les chiffres exacts d'un code existant	CADNA, Verificarlo	A
Invariants, cas limites	Propriété connue du résultat	Assertions, tests par propriétés	B, E
Dos-à-dos	Deux implémentations, migration, optimisation	Tests différentiels	B

TP DE LA SÉANCE

TP

- **B : injection de fautes.** Injecter une panne à chaque appel de `Comptes` ; corriger la compensation jusqu'à ce que l'invariant tienne aux 3 points
- **E : solutions manufacturées.** Implémenter le terme source, mesurer l'ordre d'Euler puis de Heun (attendu : 1 et 2) ; le test échoue si l'ordre observé s'écarte de plus de 0,1
- **A : encadrement.** Écrire la classe d'intervalles et un test qui échoue si la largeur relative de la racine dépasse 10^{-12}
- **E : unités.** Typer `ChuteLibre` avec Indriya ; test métamorphique mètres / pieds
- **Livrable** : code, tests, court rapport (tableaux de mesures, techniques choisies et justification)

À RETENIR

1 Injection de fautes

Modèle de faute + point d'injection + comportement attendu. Injecter à chaque point, vérifier un invariant.

2 Injection de solutions

Choisir la solution, fabriquer le problème : oracle exact. Vérifier l'ordre de convergence observé.

3 Encadrement

Intervalle étroit = résultat garanti ; intervalle large = instabilité à examiner.

4 Analyse dimensionnelle

Les unités font partie du type. Tester l'invariance par changement d'unités.

5 Perturbation

Arrondis aléatoires, 3 exécutions : les chiffres communs sont exacts.

6 Invariants et dos-à-dos

Vérifier une propriété du résultat ou comparer à une implémentation indépendante.

TRAVAUX DIRIGÉS : SÉANCES 1 À 6

172

Séance 7 · 4 h · Mobiliser vocabulaire, représentation machine, modèle de test, oracles, conception et vérification des calculs

CHAP VII

CONSIGNES DU TD

- Travail en **binômes**, 10 exercices progressifs, environ **20 minutes** chacun
- Pour chaque réponse : **nommer la technique** utilisée et **justifier** le choix
- Exercice 2 : prévoir d'abord à la main, **puis vérifier en Java** (`jshell` suffit)
- Les fils rouges sont ceux du cours : spécifications rappelées sur chaque énoncé
- Correction collective après chaque bloc de 3 exercices

Bloc	Exercices	Séances
Fondements	1 – 2	1, 2
Modèle et oracle	3 – 4	3
Boîte noire	5 – 7	4
Boîte blanche	8 – 9	5
Calculs	10	6

Rappel
Un bon corrigé ne se limite pas au résultat : il dit **pourquoi** ce test, cet oracle, cette tolérance.

EXERCICE 1 : ERREUR, DÉFAUT, DÉFAILLANCE

EXERCICE

- Spécification : de 5 001 à 50 000 FCFA, frais = **1 %** ; de 50 001 à 500 000 FCFA, frais = **0,8 %** (plafond 2 500)
- Le développeur a lu « au-delà de 50 000 » comme « à partir de 50 000 » et a codé :

```
if (montant.compareTo(new BigDecimal("50000")) < 0) // 1 %
    return montant.multiply(new BigDecimal("0.01"));
return montant.multiply(new BigDecimal("0.008")).min(PLAFOND_FRAIS);

// Suite de tests existante : frais(1000), frais(20000), frais(100000)
```

Questions

(a) Identifier l'erreur, le défaut, la défaillance. (b) Quel test révèle la défaillance ? (c) La suite existante couvre-t-elle 100 % des instructions ? La détecte-t-elle ? (d) Vérification ou validation ?

EXERCICE 2 : PRÉVOIR LE RÉSULTAT DE LA MACHINE

EXERCICE

```
(short) (32767 + 1)
Integer.MAX_VALUE * 2
Math.abs(Integer.MIN_VALUE)
1_000_000 * 1_000_000
(1_500_000_000 + 2_000_000_000) / 2
(int) 3.99e9
0.1 + 0.2 == 0.3
Math.ulp(1.0)    et    Math.ulp(1e16)
1e16 + 1 == 1e16
(float) 16_777_217 == 16_777_216f
0.1f == 0.1
Double.NaN == Double.NaN
0.0 == -0.0    et    1 / -0.0
```

- Pour chaque expression Java : **valeur** ou **vrai / faux**
- Expliquer en une phrase : débordement, saturation, absorption, arrondi, valeur spéciale ?
- Vérifier ensuite dans `jshell`
- Question bonus : laquelle de ces lignes explique le bug historique de `binarySearch` du JDK ?

EXERCICE 3 : MODÉLISER TROIS TESTS

EXERCICE

Pour chaque situation, donner **DE, g, RA, N, Tol** et le verdict possible. Justifier N et Tol.

Fil rouge	Situation	À déterminer
A · Solveur	Racines de $x^2 - 3x + 2 = 0$ avec `Solveur.racines`	Mesure d'erreur indépendante de l'ordre des racines ? Tolérance en ε ?
B · Mobile Money	Frais d'un transfert de 50 000 FCFA	Faut-il une tolérance pour un montant en BigDecimal ?
E · Chute libre	Position à t = 2 s, sans frottement, Euler explicite, h = 0,01 s	Écart attendu entre Euler et $\frac{1}{2} gt^2$? Est-ce un bug ?

Indice pour E

Avec v' = g et x' = v, Euler explicite donne exactement $x_n = \frac{1}{2} \cdot g \cdot t_n \cdot (t_n - h)$. Calculez l'écart à t = 2 s.

EXERCICE 4 : PROPOSER DES RELATIONS MÉTAMORPHIQUES

EXERCICE

Aucun résultat attendu n'est disponible. Proposer **deux relations** par programme et dire si elles sont **exactes en flottant**.

1 sin(x)

Bibliothèque mathématique maison. Que doit vérifier sin pour des entrées liées entre elles ?

2 trier(liste)

Tri d'une liste d'entiers. Quelles transformations de l'entrée ne doivent pas changer la sortie ?

3 plusCourtChemin(G, a, b)

Dijkstra sur un graphe non orienté pondéré (réseau routier entre villes).

A Solveur.racines(a, b, c)

Relier les racines de (a, b, c) à celles d'une équation transformée.

EXERCICE 5 : CLASSES D'ÉQUIVALENCE ET LIMITES

EXERCICE

- **Champ 1 :** date de naissance ; âge entre **16 et 60 ans** (inclus) à la date d'inscription
  - Date d'inscription fixée : **01/10/2026**
- **Champ 2 :** téléphone : `+237` puis **9 chiffres** commençant par **6 ou 2**
  - Regex : `^\+237[62][0-9]{8}$`

- Lister les classes **valides** et **invalides** de chaque champ (format, existence, domaine)
- Donner les **valeurs limites** précises de la date de naissance
- Combien de cas de test en couverture **faible robuste** ? Rappel : une seule classe invalide par cas
- Que doit-on faire pour une personne née un **29 février** ?

EXERCICE 6 : TABLE DE DÉCISION DU TRANSFERT

EXERCICE

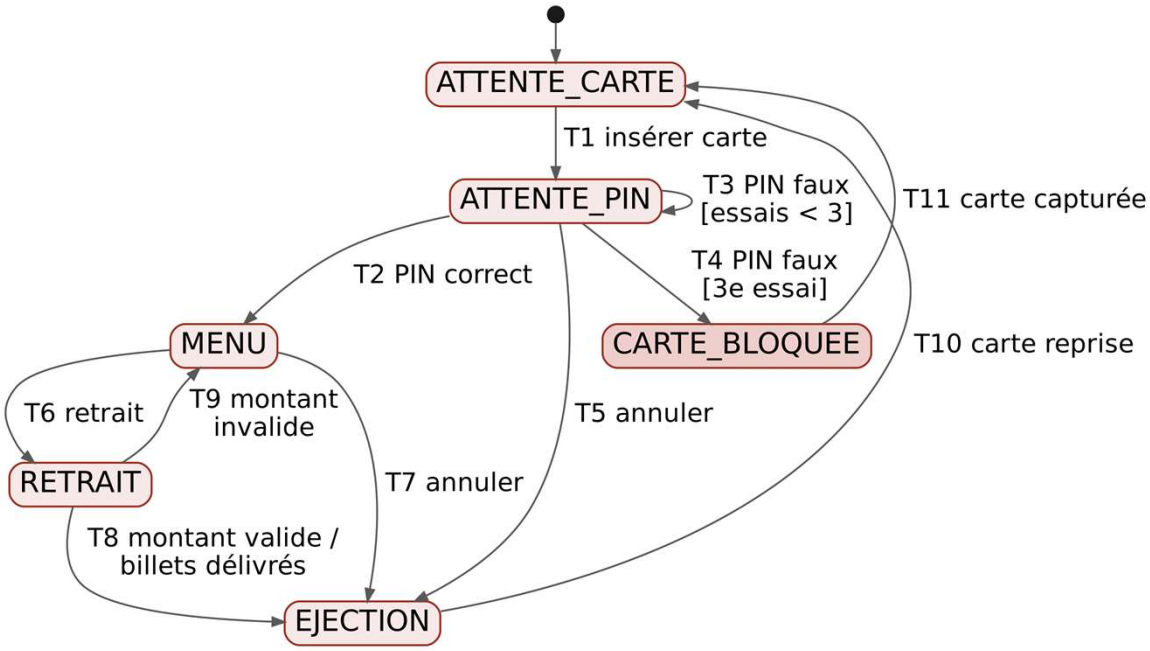
Compléter la table complète (8 règles), puis la **réduire** avec « – » (sans importance).
Combien de tests suffisent ?

Conditions / actions	R1	R2	R3	R4	R5	R6	R7	R8
C1 montant ∈ [100 ; 500 000]	O	O	O	O	N	N	N	N
C2 cumul du jour + montant ≤ 1 000 000	O	O	N	N	O	O	N	N
C3 solde ≥ montant + frais	O	N	O	N	O	N	O	N
A1 refus « montant invalide »	?	?	?	?	?	?	?	?
A2 refus « plafond dépassé »	?	?	?	?	?	?	?	?
A3 refus « solde insuffisant »	?	?	?	?	?	?	?	?
A4 exécuter + SMS	?	?	?	?	?	?	?	?

Piège
R4 : plafond dépassé **et** solde insuffisant. Quel message afficher ? Que dit la spécification ?

EXERCICE 7 : TRANSITIONS D'ÉTATS DU GAB

EXERCICE



- Écrire des séquences de test qui couvrent **tous les états**, puis **toutes les transitions** (0-switch)
- Combien de **sessions de carte** au minimum ? Pourquoi ?
- Écrire la séquence « 3 PIN faux » avec états attendus
- Proposer 2 **transitions invalides** à tester (événement non prévu dans un état) et le résultat attendu

EXERCICE 8 : MCCABE ET MC/DC

EXERCICE

```java
// A : carteValide        B : multiple de 5 000
// C : montant <= solde   D : plafond du jour
static boolean autoriserRetrait(int montant,
        int solde, int retire, boolean carteValide) {
    if (carteValide
        && montant % 5000 == 0
        && montant <= solde
        && retire + montant <= 200_000) {
        return true;
    }
    return false;
}
```

- Dessiner le **graphe de flot de contrôle** en tenant compte du court-circuit `&&`
- Calculer **V(G)**. Combien de branches JaCoCo compte-t-il ?
- Donner un jeu **MC/DC minimal** avec des valeurs concrètes
- Bonus : quel débordement peut fausser D ?

EXERCICE 9 : LES MUTANTS SURVIVENT-ILS ?

EXERCICE

```java
BigDecimal frais(BigDecimal m) {
  if (m.compareTo(MIN) < 0 || m.compareTo(MAX) > 0)
    throw new MontantInvalideException();
  if (m.compareTo(new BigDecimal("5000")) <= 0)
    return new BigDecimal("50");
  if (m.compareTo(new BigDecimal("50000")) <= 0)
    return m.multiply(new BigDecimal("0.01"));
  return m.multiply(new BigDecimal("0.008"))
          .min(new BigDecimal("2500"));
}
// Tests (compareTo) : 1000→50, 20000→200,
//                     100000→800
```

- **M1 :** `<= 0` → `< 0` (borne 5 000)
- **M2 :** `<= 0` → `< 0` (borne 50 000)
- **M3 :** `.min` → `.max`
- **M4 :** suppression de `.min(2500)`
- **M5 :** `compareTo(MIN) < 0` → `<= 0`
- Lesquels sont **tués** ? Lesquels **survivent** ? Y a-t-il un mutant **équivalent** ?
- Ajouter le minimum de tests pour tuer les survivants

EXERCICE 10 : QUELLE TECHNIQUE DE VÉRIFICATION ?

EXERCICE

Pour chaque scénario, choisir la technique la plus adaptée parmi : **injection de fautes, injection de solution, encadrement, analyse dimensionnelle, perturbation**. Justifier.

Scénario	Technique ?
1. Chute libre avec frottement : pas de solution analytique simple, on veut vérifier l'implémentation d'Euler	
2. Un module convertit une vitesse saisie en km/h et l'additionne à une vitesse en m/s	
3. La suite de tests du service Mobile Money détecte-t-elle vraiment les erreurs ?	
4. Un rapport additionne 10 millions de montants en `double` : combien de chiffres du total sont fiables ?	
5. Prouver qu'un calcul embarqué sur `short` ne déborde jamais pour des capteurs entre −500 et 500	

BARÈME ET AUTO-ÉVALUATION

Notez-vous honnêtement : la colonne de droite sert à préparer le **contrôle continu** (séance 8).

Ex.	Compétence évaluée	Points	Acquis / à revoir
1	Distinguer erreur, défaut, défaillance ; vérification vs validation	1,5	
2	Prévoir débordements, saturation, absorption, arrondis	2,5	
3	Modéliser un test et justifier N et Tol	2,5	
4	Construire des oracles métamorphiques	2	
5	Partitionner un domaine par type ; valeurs limites	2,5	
6	Construire et réduire une table de décision	2	
7	Couvrir un diagramme d'états-transitions	2	
8	Calculer V(G) et un jeu MC/DC	2	
9	Raisonner sur les mutants (tués, survivants, équivalents)	1,5	
10	Choisir une technique de vérification des calculs	1,5	

CONTRÔLE CONTINU

185

Séance 8 · 2 h · Évaluation écrite des séances 1 à 7 · 20 % de la note finale

MODALITÉS DU CONTRÔLE CONTINU

1 Durée et format

- **2 heures**, épreuve écrite individuelle
- Sur papier ; pas de machine
- Calculatrice non programmable autorisée

2 Documents

- **Une feuille A4 recto verso** manuscrite personnelle
- Aucun autre document
- Téléphones éteints et rangés

3 Périmètre

- Séances **1 à 7** (TD compris)
- Vocabulaire, représentation machine, modèle de test, oracles
- Boîte noire, boîte blanche, vérification des calculs

4 Types de questions

- Questions courtes de cours
- Prévoir un résultat machine
- Concevoir des tests sur un fil rouge
- Analyser un extrait de code

5 Barème indicatif

- Cours et vocabulaire : 4 pts
- Représentation et oracles : 6 pts
- Conception de tests : 7 pts
- Vérification des calculs : 3 pts

6 Conseils

- **Justifier** chaque choix (technique, N, Tol)
- Donner des **valeurs concrètes**, pas « une valeur limite »
- Relire la spécification : les pièges y sont

GRILLE DE CORRECTION : CE QUI RAPPORTE DES POINTS

Critère	Attendu
Exactitude	Résultat correct, unités et types précisés (int, double, BigDecimal)
Justification	Technique nommée et choix argumenté (pourquoi cette classe, cet oracle)
Précision	Valeurs concrètes aux bornes ; résultats attendus explicites
Rigueur de l'oracle	Mesure d'erreur adaptée et tolérance justifiée, pas arbitraire
Esprit critique	Lacunes de la spécification repérées et signalées

Rappel
Une réponse juste sans justification vaut **au plus la moitié** des points.

NIVEAUX, DOUBLURES ET TDD/BDD

188

Séance 9 · 4 h · Écrire des tests unitaires solides, isoler avec des doublures, piloter le code par les tests, automatiser dans la CI

CHAP IX

OBJECTIFS DE LA SÉANCE

- Situer un test par son **niveau** et son **type**
- Écrire un test unitaire **lisible et fiable** avec JUnit 5
- Isoler le code testé avec les **5 doublures** et Mockito
- Diagnostiquer et supprimer les **tests instables**
- Pratiquer **TDD**, **BDD** (Gherkin) et **ATDD**
- Organiser les tests dans **Maven** et les exécuter dans un **pipeline CI**

Fil rouge de la séance

Le service de transfert **Mobile Money** : frais par tranche, plafond journalier, solde, notification SMS du destinataire. Code et tests compilés, logique exécutée.

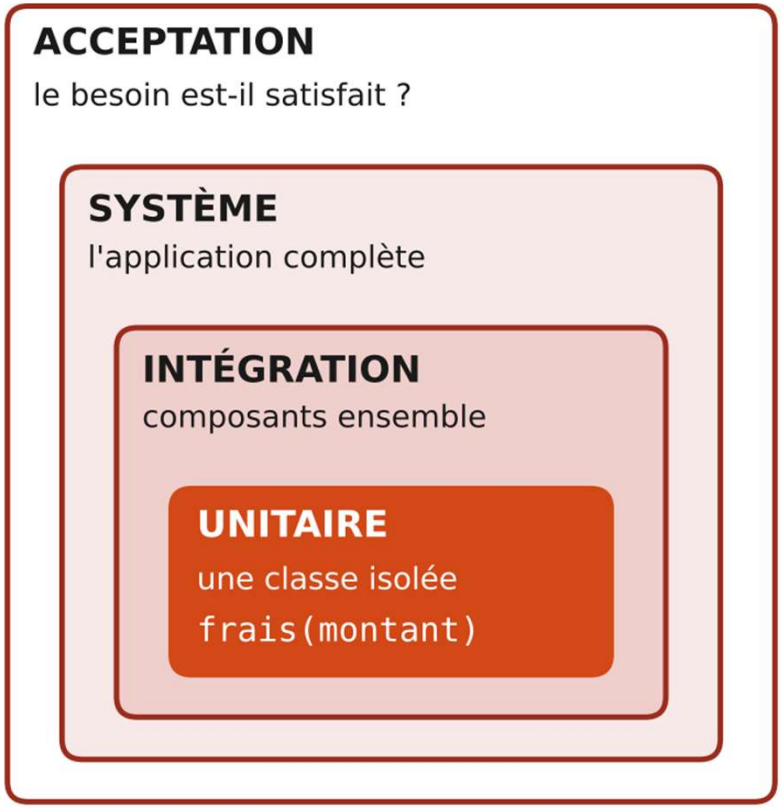
LES NIVEAUX DE TEST

Niveau	Objet testé	Base de test (référentiel)	Environnement	Qui
Unitaire (composant)	Une classe, une méthode, isolée	Conception détaillée, contrats, code	Doublures pour les dépendances ; millisecondes	Développeur
Intégration de composants	Interactions entre modules	Architecture, interfaces internes	Vraies classes, base de test	Développeur
Intégration de systèmes	Interfaces avec l'extérieur (BD, API opérateur)	Contrats d'interface, protocoles	Testcontainers, bacs à sable	Dév. / testeur
Système	L'application complète	Exigences fonctionnelles et non fonctionnelles	Proche de la production	Équipe de test
Acceptation	L'adéquation au besoin	Besoins métier, critères d'acceptation, réglementation	Pré-production, utilisateurs réels	Client, utilisateurs (UAT), exploitation

Correction du cours précédent

Un niveau se définit par **ce qu'on teste** et **par rapport à quoi** (base de test), pas par l'outil utilisé.

LES NIVEAUX S'EMBOÎTENT



- Chaque niveau détecte des défauts **que les autres ne voient pas**
  - unitaire : erreur de borne dans `frais()`
  - intégration : mauvais mapping SQL du solde
  - système : SMS jamais reçu en conditions réelles
  - acceptation : le client attendait les frais à la charge du destinataire
- Plus on monte, plus le test est **lent, coûteux et fragile** : d'où la **pyramide** (beaucoup d'unitaires, peu de bout en bout)

LES TYPES DE TEST : ORTHOGONAUX AUX NIVEAUX

Type	Question posée	Exemple Mobile Money
Fonctionnel	Que fait le système ?	Frais selon la tranche ; refus si solde insuffisant
Non fonctionnel	Comment le fait-il ? (ISO/IEC 25010)	1 000 transferts/s ; p95 < 300 ms ; PIN jamais journalisé
Boîte blanche	La structure est-elle exercée ?	Toutes les branches de `frais()` couvertes
Confirmation (re-test)	Le défaut corrigé l'est-il vraiment ?	Rejouer le test à 50 000 FCFA après le correctif
Régression	La modification a-t-elle cassé autre chose ?	Rejouer toute la suite après ajout d'une tranche

Vocabulaire

« Test de régression » et « test de **non**-régression » désignent la même chose. Un test de performance peut être unitaire (micro-benchmark) ou système.

ANATOMIE D'UN TEST UNITAIRE : AAA

```java
@Test
@DisplayName("50 000 FCFA → 500 FCFA")
void fraisBorneHauteTrancheUnPourcent() {
    // Arrange (Given) : préparer le contexte
    TransfertService service =
        new TransfertService(null, null, null);
    BigDecimal montant = new BigDecimal("50000");

    // Act (When) : UNE seule action
    BigDecimal frais = service.frais(montant);

    // Assert (Then) : vérifier le résultat
    assertEquals(0,
        frais.compareTo(new BigDecimal("500")));
}
```

- **Arrange / Act / Assert** = Given / When / Then
- Une seule action, un seul **comportement** vérifié
- Le nom dit **quoi** et **dans quel cas** : il sert de spécification
- Ici, trois `null` : des **dummies** (dépendances jamais utilisées par `frais()`)

F.I.R.S.T. : CINQ QUALITÉS D'UN BON TEST

F Fast (rapide)

Quelques millisecondes : on lance des milliers de tests à chaque modification. Pas de réseau, pas de disque.

I Isolated (isolé)

Ne dépend ni d'un autre test ni de leur ordre. Chaque test prépare son propre contexte.

R Repeatable (reproductible)

Même résultat partout, tout le temps : pas d'heure système, d'aléa ou de base partagée non maîtrisés.

S Self-validating

Le verdict est automatique (assertion) : personne ne lit des logs pour savoir si ça passe.

T Timely (au bon moment)

Écrit avant ou avec le code, pas « plus tard » : un code écrit sans test est souvent difficile à tester.

! Anti-patterns

Test sans assertion ; `try/catch` qui avale l'échec ; `Thread.sleep` ; un test qui vérifie dix choses.

JUNIT 5 (JUPITER) : L'ESSENTIEL

Élément	Rôle
`@Test`	Déclare une méthode de test
`@DisplayName("…")`	Libellé lisible dans les rapports
`@BeforeEach` / `@AfterEach`	Préparer / nettoyer avant et après **chaque** test
`@BeforeAll` / `@AfterAll`	Une fois par classe (méthodes `static`)
`@Nested`	Classe interne pour regrouper les tests par contexte
`@ParameterizedTest` + `@CsvSource`	Même test, plusieurs jeux de données (aussi `@ValueSource`, `@MethodSource`)
`@Tag("lent")` / `@Disabled("raison")`	Filtrer l'exécution / désactiver en le justifiant
`assertEquals`, `assertThrows`, `assertAll`, `assertTimeout`	Assertions de base (`Assertions.*`)

JUNIT 5 : @NESTED, @DISPLAYNAME, ASSERTTHROWS

FraisTest.java

```java
@DisplayName("Frais de transfert Mobile Money")
class FraisTest {
    TransfertService service = new TransfertService(null, null, null);

    @Nested @DisplayName("Tranche forfaitaire 100 – 5 000 FCFA")
    class Forfait {
        @Test @DisplayName("5 000 FCFA (borne haute) → 50 FCFA")
        void borneHaute() {
            BigDecimal frais = service.frais(new BigDecimal("5000"));
            assertEquals(0, frais.compareTo(new BigDecimal("50")));
        }
    }

    @Test @DisplayName("99 FCFA est refusé")
    void sousLeMinimum() {
        MontantInvalideException e = assertThrows(
            MontantInvalideException.class,
            () -> service.frais(new BigDecimal("99")));
    }
}
```

`assertThrows` **renvoie l'exception** : on peut ensuite vérifier son message ou ses champs.

TESTS PARAMÉTRÉS : LES CLASSES ET LIMITES EN DONNÉES

```
@ParameterizedTest(name = "{0} FCFA → {1} FCFA")
@CsvSource({
    "100,    50",      // borne basse
    "5000,   50",
    "5001,   50.01",   // arrondi ?
    "50000,  500",
    "50001,  400.008",
    "312500, 2500",    // 0,8 % = plafond
    "500000, 2500"     // borne haute
})
void fraisParTranche(BigDecimal montant,
                     BigDecimal attendu) {
    assertEquals(0,
        service.frais(montant).compareTo(attendu));
}
```

- Une ligne CSV = **un cas de test** (DE, RA)
- JUnit convertit les chaînes en `BigDecimal`
- Les valeurs viennent de l'**analyse des valeurs limites** (séance 4)
- **5001 → 50,01 FCFA** : le test révèle une **lacune de la spécification** (aucune règle d'arrondi)

ASSERTIONS : LES PIÈGES À CONNAÎTRE

```java
BigDecimal frais = service.frais(new BigDecimal("50000")); // 500.00

assertEquals(new BigDecimal("500"), frais);   // ÉCHOUE : échelle 0 ≠ 2
assertEquals(0, frais.compareTo(new BigDecimal("500")));          // OK
assertThat(frais).isEqualByComparingTo("500");        // OK (AssertJ)

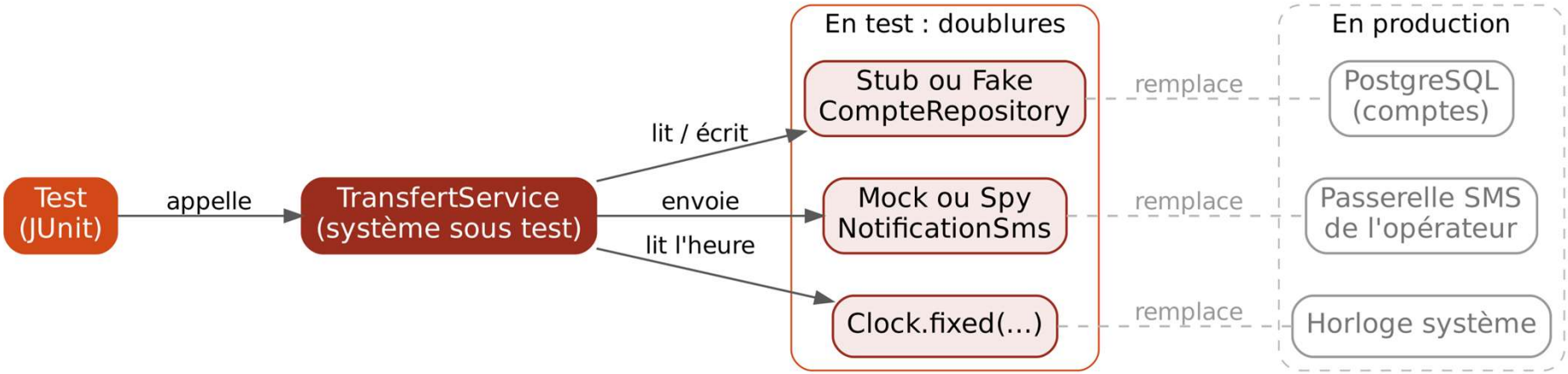
// Plusieurs vérifications liées : toutes évaluées, un seul rapport
assertAll("reçu",
    () -> assertEquals(ALICE, recu.de()),
    () -> assertEquals(0, recu.frais().compareTo(new BigDecimal("100"))));

// double : toujours une tolérance JUSTIFIÉE (séance 3), jamais ==
assertEquals(19.62, x, 0.1);   // Tol = ½·g·t·h pour Euler, h = 0,01
```

BigDecimal
`equals` compare **valeur et échelle** : `new BigDecimal("50").equals(new BigDecimal("50.00"))` est **faux**. Pour un montant, comparer avec `compareTo`.

POURQUOI DES DOUBLURES DE TEST ?

`TransfertService` dépend d'une base, d'une passerelle SMS et de l'horloge : **lentes, coûteuses, non reproductibles**. On les remplace par des doublures.



Condition : le code reçoit ses dépendances par **injection** (constructeur), il ne les crée pas lui-même avec `new`.

LES CINQ DOUBLURES (MESZAROS)

Doublure	Comportement	Ce qu'on vérifie	Exemple Mobile Money
Dummy	Passé en paramètre, jamais utilisé	Rien	`null` pour le SMS dans un test de `frais()`
Stub	Renvoie des réponses préprogrammées	L'**état** du système testé	`trouver(ALICE)` renvoie un compte à 20 000 FCFA
Spy	Stub qui **enregistre** les appels reçus	Les appels, après coup	SMS espion qui mémorise les messages envoyés
Mock	Programmé avec des **attentes** d'appel	Les **interactions**	`verify(sms).envoyer(BOB, …)`
Fake	Vraie implémentation **simplifiée**	L'état, de bout en bout	`CompteRepositoryEnMemoire` (HashMap), base H2

Règle pratique
Stub/fake pour les **requêtes** (on lit), mock pour les **commandes** vers l'extérieur (on envoie). Ne mockez que ce qui vous appartient.

MOCKITO : STUB ET VÉRIFICATION D'INTERACTION

```
class TransfertServiceTest {
    CompteRepository comptes = mock(CompteRepository.class);  // stub
    NotificationSms sms = mock(NotificationSms.class);        // mock
    Clock horloge = Clock.fixed(Instant.parse("2026-11-02T10:00:00Z"),
                                ZoneId.of("Africa/Douala"));
    TransfertService service = new TransfertService(comptes, sms, horloge);

    @Test
    void transfertReussiNotifieLeDestinataire() {
        Compte alice = new Compte(ALICE, new BigDecimal("20000"));
        Compte bob = new Compte(BOB, BigDecimal.ZERO);
        when(comptes.trouver(ALICE)).thenReturn(Optional.of(alice));
        when(comptes.trouver(BOB)).thenReturn(Optional.of(bob));
        when(comptes.totalEnvoye(any(), any())).thenReturn(BigDecimal.ZERO);

        service.transferer(ALICE, BOB, new BigDecimal("10000"));

        assertEquals(0, alice.solde().compareTo(new BigDecimal("9900")));
        verify(sms).envoyer(eq(BOB), anyString());
    }
}
```

MOCKITO : NEVER ET ARGUMENTCAPTOR

```java
@Test
void soldeInsuffisantNiEcritureNiSms() {
    // ... comptes : Alice 10 000 FCFA, Bob 0 FCFA
    assertThrows(SoldeInsuffisantException.class,
        () -> service.transferer(ALICE, BOB, new BigDecimal("10000")));
    verify(comptes, never()).sauver(any());     // aucune écriture
    verifyNoInteractions(sms);                  // aucun SMS
}

@Test
void leSmsContientLeMontant() {
    // ... comptes : Alice 20 000 FCFA, Bob 0 FCFA
    service.transferer(ALICE, BOB, new BigDecimal("10000"));

    ArgumentCaptor<String> message = ArgumentCaptor.forClass(String.class);
    verify(sms).envoyer(eq(BOB), message.capture());
    assertTrue(message.getValue().contains("10000 FCFA"));
}
```

Attention au sur-mocking
Vérifier **chaque** appel interne rend le test fragile : il casse au moindre refactoring sans qu'aucun comportement n'ait changé.

FAKE : TESTER DE BOUT EN BOUT SANS BASE

```java
@Test
void laMasseMonetaireEstConservee() {
  var comptes = new CompteRepositoryEnMemoire();
  comptes.sauver(new Compte("A",
      new BigDecimal("20000")));
  comptes.sauver(new Compte("B", BigDecimal.ZERO));
  var service = new TransfertService(comptes,
      (numero, msg) -> {}, Clock.systemUTC());

  var recu = service.transferer("A", "B",
      new BigDecimal("10000"));

  BigDecimal total = comptes.trouver("A").get()
      .solde()
      .add(comptes.trouver("B").get().solde())
      .add(recu.frais());
  assertEquals(0,
      total.compareTo(new BigDecimal("20000")));
}
```

- Le fake est une **vraie implémentation** en mémoire (HashMap)
- Le test vérifie un **invariant de conservation** (séance 6) : A + B + frais = constant
- Le SMS est un **dummy** écrit en lambda
- Plus robuste qu'un mock : il ne dépend pas de **comment** le service appelle le dépôt

TESTS INSTABLES (FLAKY) : CAUSES ET REMÈDES

Un test **instable** passe ou échoue **sans changement de code**. Il détruit la confiance dans toute la suite.

Cause	Exemple	Remède
Temps	`LocalDate.now()` : plafond journalier faux à minuit	`Clock` injectable, `Clock.fixed(...)`
Aléa	`new Random()` sans graine, UUID dans une assertion	Graine fixée, générateur injecté
Ordre / état partagé	Champ `static`, base non nettoyée entre tests	Contexte recréé dans `@BeforeEach`, données uniques
Concurrence	`Thread.sleep(500)` puis assertion	Attente bornée sur condition (Awaitility), latch
Réseau / externe	Appel réel à l'API SMS de l'opérateur	Doublure, WireMock, Testcontainers
Environnement	Locale, fuseau, encodage, séparateur décimal	Fixer locale et fuseau ; conteneur

MAÎTRISER LE TEMPS : INJECTER UNE CLOCK

```java
// Instable : dépend du jour de l'exécution
LocalDate jour = LocalDate.now();

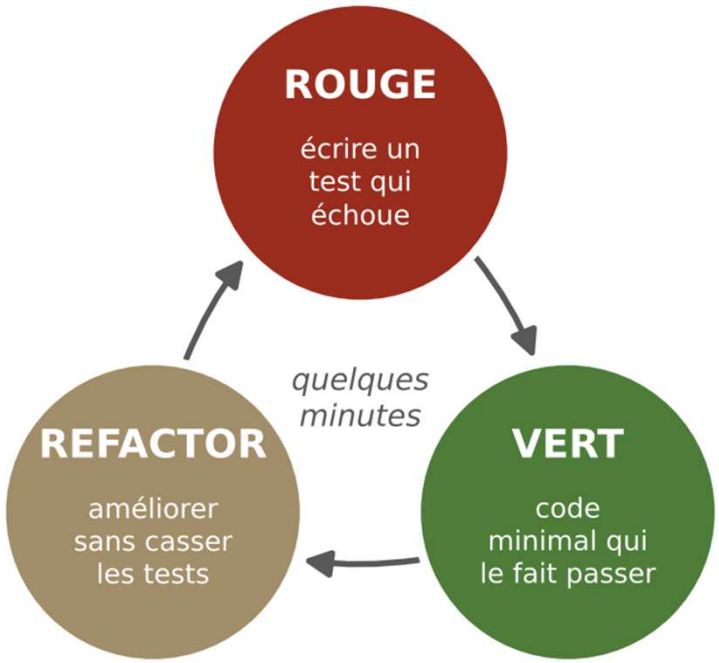
// Testable : l'horloge est une dépendance
LocalDate jour = LocalDate.now(horloge);

// Test : minuit à Douala = 23 h 30 UTC
Clock horloge = Clock.fixed(
    Instant.parse("2026-11-02T23:30:00Z"),
    ZoneId.of("Africa/Douala"));

LocalDate.now(horloge);   // 2026-11-03
// avec ZoneOffset.UTC :     2026-11-02
```

- Le plafond de **1 000 000 FCFA/jour** dépend de la date
- Avec une horloge fixe, le test donne **toujours** le même résultat
- Tester la **frontière de minuit** et le **fuseau** : 23 h 30 UTC, c'est déjà le lendemain à Douala (UTC+1)

TDD : LE CYCLE ROUGE, VERT, REFACTOR



- **Loi 1 :** pas de code de production sans un test qui échoue
- **Loi 2 :** pas plus de test que nécessaire pour échouer (ne pas compiler compte comme un échec)
- **Loi 3 :** pas plus de code que nécessaire pour faire passer le test
- Effets : conception **testable** par construction, filet de sécurité pour le refactoring
- Limite : TDD ne trouve pas les cas **auxquels on n'a pas pensé** : les techniques de conception restent nécessaires

TDD PAS À PAS : LE PLAFOND DES FRAIS

Règle à implémenter : au-delà de 50 000 FCFA, frais = 0,8 % **plafonné à 2 500 FCFA** ; au-delà de 500 000, refus.

1 · Rouge

- Test : `frais(400000)` = 2 500
- Échoue : le code renvoie 3 200

2 · Vert

- Ajouter le plafond avec `min`
- Tous les tests passent

3 · Refactor

- Extraire la constante `PLAFOND_FRAIS`
- Les tests restent verts

4 · Borne

- Test : `frais(312500)` = 2 500
- Passe d'emblée : on le garde comme documentation de la borne

5 · Rouge

- Test : `frais(500001)` doit être refusé
- Échoue, puis on ajoute le contrôle

Un test qui passe **tout de suite** ne pilote rien : se demander s'il apporte une information (ici, oui : il fige la borne).

BDD : LA SPÉCIFICATION EXÉCUTABLE EN GHERKIN

transfert.feature

```gherkin
# language: fr
Fonctionnalité: Transfert d'argent Mobile Money
  Afin d'envoyer de l'argent à un proche
  En tant que client
  Je veux transférer un montant depuis mon compte

  Scénario: Transfert réussi avec frais de 1 %
    Soit un compte "690000001" avec un solde de 20000 FCFA
    Et un compte "670000002" avec un solde de 0 FCFA
    Quand "690000001" transfère 10000 FCFA à "670000002"
    Alors le solde de "690000001" est de 9900 FCFA
    Et le solde de "670000002" est de 10000 FCFA

  Plan du scénario: Frais selon la tranche
    Soit un compte "690000001" avec un solde de 600000 FCFA
    Et un compte "670000002" avec un solde de 0 FCFA
    Quand "690000001" transfère <montant> FCFA à "670000002"
    Alors le solde de "690000001" est de <reste> FCFA

    Exemples:
      | montant | reste  |
      | 5000    | 594950 |
      | 50000   | 549500 |
      | 400000  | 197500 |
```

BDD : RELIER LES PHRASES AU CODE (CUCUMBER)

```java
import io.cucumber.java.fr.*;

public class TransfertSteps {
  CompteRepositoryEnMemoire comptes =
      new CompteRepositoryEnMemoire();
  TransfertService service = new TransfertService(
      comptes, (n, msg) -> {}, Clock.systemUTC());

  @Soit("un compte {string} avec un solde de "
      + "{bigdecimal} FCFA")
  public void unCompte(String num, BigDecimal s) {
    comptes.sauver(new Compte(num, s));
  }

  @Quand("{string} transfère {bigdecimal} FCFA "
      + "à {string}")
  public void transfere(String de, BigDecimal m,
                        String vers) {
    service.transferer(de, vers, m);
  }
  // @Alors("le solde de {string} est de ...")
}
```

- Une **définition d'étape** par phrase type
- `{string}`, `{bigdecimal}` : paramètres extraits de la phrase
- Le mot-clé (Soit, Et…) ne compte pas pour la correspondance
- Les étapes s'appuient sur le **fake** : scénarios rapides et reproductibles

TDD, BDD, ATDD : TROIS BOUCLES COMPLÉMENTAIRES

	TDD	BDD	ATDD
Qui	Le développeur	Métier + dév + test (« trois amigos »)	Client + équipe
Niveau	Unitaire	Comportement, acceptation	Acceptation
Langage	Code (JUnit)	Gherkin (langue naturelle structurée)	Critères d'acceptation exécutables
Question	Le code fait-il ce que je veux ?	Le système se comporte-t-il comme convenu ?	La user story est-elle terminée ?
Quand	Avant chaque petit incrément	Avant de développer la fonctionnalité	Avant le sprint, pendant l'affinage

Double boucle : un scénario d'acceptation rouge (boucle externe ATDD/BDD) est rendu vert par une série de cycles TDD (boucle interne).

ORGANISATION MAVEN : SUREFIRE ET FAILSAFE

```
transfert/
├── pom.xml
└── src/
    ├── main/java/cm/mm/
    │   └── TransfertService.java
    └── test/
        ├── java/cm/mm/
        │   ├── FraisTest.java
        │   ├── TransfertServiceTest.java
        │   ├── CompteRepositoryIT.java
        │   └── TransfertSteps.java
        └── resources/features/
            └── transfert.feature
```

Plugin	Fichiers	Phase
surefire	`*Test`, `Test*`, `*Tests`	`test`
failsafe	`*IT`, `IT*`, `*ITCase`	`integration-test` puis `verify`

failsafe n'échoue qu'en phase `verify` : la phase `post-integration-test` peut arrêter proprement les conteneurs. Lancer `mvn verify`, pas `mvn integration-test`.

PREMIER PIPELINE CI : GITHUB ACTIONS

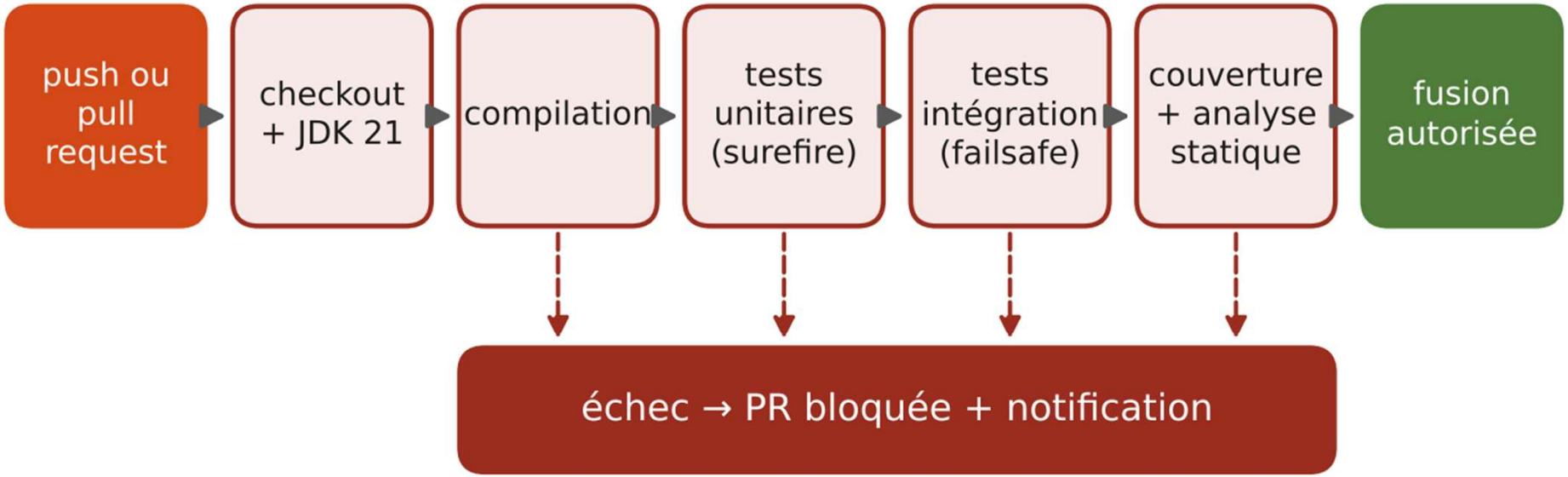
.github/workflows/ci.yml

```yaml
name: ci
on:
  push:
    branches: [main]
  pull_request:

jobs:
  build-et-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
      - name: Compilation et tests
        run: mvn -B verify
      - name: Rapports de tests
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: rapports-tests
          path: target/*-reports/
```

- Déclenché à chaque **push** sur `main` et chaque **pull request**
- `mvn -B verify` : unitaires (surefire) puis intégration (failsafe)
- `if: always()` : les rapports sont publiés **même en cas d'échec**
- Règle de branche : fusion **interdite** si le pipeline est rouge

LE PIPELINE COMME FILET DE SÉCURITÉ



Feedback rapide
Viser **moins de 10 minutes** pour le pipeline d'une pull request : au-delà, les développeurs contournent ou attendent. Les tests lents vont dans un pipeline de nuit.

NON-RÉGRESSION : QUE REJOUER, ET QUAND ?

Stratégie	Principe	Moment
Confirmation (re-test)	Rejouer le test qui avait échoué, après correction	Après chaque correctif
Sélection par impact	Rejouer les tests qui couvrent le code modifié (couverture, dépendances)	À chaque pull request
Priorisation	Ordonner par risque, historique d'échecs, durée	Feedback le plus tôt possible
Smoke test	Petit sous-ensemble critique : « l'appli démarre et transfère »	Après chaque déploiement
Tout rejouer	Suite complète : unitaires, intégration, E2E, performance	Chaque nuit, avant une version

Tout défaut corrigé laisse derrière lui **un test de non-régression** qui reproduit le cas (ex. `frais(50000)` = 500).

À RETENIR

À RETENIR

- Niveau = **quoi** et **par rapport à quoi** ; type = **quelle question** : les deux sont indépendants
- Un bon test unitaire : **AAA**, un comportement, un nom qui spécifie, **F.I.R.S.T.**
- Les doublures exigent l'**injection de dépendances** ; stub/fake pour lire, mock pour les commandes sortantes
- Montants : `compareTo`, jamais `equals` ni `double` ; flottants : tolérance **justifiée**
- Un test instable se **corrige** (horloge, aléa, ordre, réseau), il ne se relance pas
- TDD conçoit, BDD/ATDD alignent sur le besoin ; aucun ne remplace la **conception systématique** des cas
- Le pipeline CI rejoue la non-régression à chaque modification : **rouge = on ne fusionne pas**

TP DE LA SÉANCE

TP

- **1. TDD :** implémenter `frais()` test par test jusqu'à couvrir toutes les classes et limites de la séance 4
- **2. Mockito :** tester `transferer()` : succès, solde insuffisant, plafond journalier, compte inconnu, contenu du SMS
- **3. Fake :** réécrire 2 tests avec `CompteRepositoryEnMemoire` et l'invariant de conservation
- **4. BDD :** 2 scénarios Gherkin + 1 plan avec exemples, exécutés par Cucumber
- **5. CI :** pipeline GitHub Actions vert, rapports publiés

Livrable

Dépôt Git avec historique de commits **rouge → vert → refactor** visible, pipeline vert, et un court README listant les **lacunes de spécification** rencontrées (arrondi, priorité des refus, fuseau du plafond).

ANALYSE STATIQUE, REVUES, QUALITÉ ET GESTION DU TEST

217

Séance 10 · 4 h · Trouver les défauts sans exécuter, mesurer la qualité, piloter l'activité de test

CHAP X

OBJECTIFS DE LA SÉANCE

- Choisir le **type de revue** adapté et conduire une inspection
- Exploiter l'**analyse statique** : ce qu'elle trouve, ce qu'elle rate, faux positifs
- Intégrer SpotBugs, PMD, Checkstyle et une **quality gate** dans le build
- Situer un défaut dans le modèle **ISO/IEC 25010:2023**
- Piloter le test : **risques**, plan, critères, anomalies, traçabilité, métriques

Fil conducteur

La vérification **statique** (revues, outils) examine le code et les documents **sans les exécuter**. Elle trouve tôt et à bas coût des défauts que le test ne voit pas, et inversement.

QUATRE TYPES DE REVUES

Classement ISTQB, du moins formel au plus formel. Plus la revue est formelle, plus elle est coûteuse et plus ses résultats sont mesurables.

Type	Formalisme	Objectif principal	Exemple
Relecture informelle	Aucun	Détecter vite des anomalies évidentes	Relecture d'une pull request par un pair
Walkthrough	Faible	Partager la compréhension, former, explorer des alternatives	L'auteur présente son algorithme de frais à l'équipe
Revue technique	Moyen	Atteindre un consensus technique, évaluer la qualité	Revue de l'architecture de l'API Scolarité par des experts
Inspection	Élevé	Trouver un maximum de défauts, mesurer, améliorer le processus	Inspection du module de calcul des frais Mobile Money

Ordre de grandeur
Les inspections bien conduites détectent une part importante des défauts **avant le premier test** ; leur rendement baisse au-delà d'environ 200 à 400 lignes inspectées par heure.

L'INSPECTION DE FAGAN (IBM, 1976)



Planifier	Présenter	Préparer	Réunir	Corriger	Suivre
Le modérateur vérifie les critères d'entrée, choisit les participants, fixe la date.	L'auteur présente le contexte et les objectifs du produit inspecté.	Chacun étudie seul le document avec la **checklist** et note ses remarques.	Le lecteur paraphrase, les défauts sont **relevés, pas corrigés** ; le scribe consigne.	L'auteur corrige les défauts relevés.	Le modérateur vérifie les corrections et décide d'une nouvelle inspection.

Les rôles

Modérateur (anime, garant du processus) · **Auteur** · **Lecteur** (paraphrase le produit) · **Inspecteurs** (cherchent les défauts) · **Scribe** (consigne). Règle d'or : on critique le produit, jamais l'auteur.

UNE CHECKLIST DE REVUE DE CODE

La checklist oriente la préparation. Elle reprend les défauts vus dans les séances précédentes.

1 Données et types

- Débordement d'entier possible ?
- Flottants comparés avec `==` ?
- Montants en `double` au lieu de `BigDecimal` ?

2 Logique

- Conditions aux limites (`<` ou `<=`) ?
- Tous les cas d'une décision traités ?
- Boucles : terminaison, bornes ?

3 Erreurs et ressources

- Exceptions avalées ?
- Ressources fermées (try-with-resources) ?
- `null` possible non géré ?

4 Sécurité

- Entrées validées (séance 13) ?
- Requêtes SQL paramétrées ?
- Secrets dans le code ?

5 Lisibilité

- Noms explicites ?
- Nombres magiques (`0.008`) ?
- Méthodes trop longues ?

6 Testabilité

- Tests associés présents ?
- Dépendances injectables ?
- Cas limites couverts ?

ANALYSE STATIQUE : CE QU'ELLE VOIT ET CE QU'ELLE RATE

L'analyse statique examine le code source ou le bytecode **sans l'exécuter**, à l'aide de règles et de modèles du programme.

- **Trouve bien :**
  - motifs de bugs : `null` déréférencé, ressource non fermée, égalité flottante
  - débordements typiques, divisions par zéro évidentes
  - violations de conventions et de style
  - failles connues : injection SQL, secrets, dépendances vulnérables
  - code mort, duplication, complexité excessive
- **Rate ou ne peut pas voir :**
  - une règle métier fausse (frais à 1 % au lieu de 0,8 %)
  - une exigence oubliée ou mal comprise
  - les problèmes de performance réelle, de configuration
  - la plupart des erreurs numériques (instabilité)
  - tout ce qui dépend de l'environnement d'exécution

Un outil d'analyse statique ne connaît pas votre **spécification**. Il vérifie que le code est bien écrit, pas qu'il fait la bonne chose.

FAUX POSITIFS ET FAUX NÉGATIFS

	Défaut réel présent	Pas de défaut réel
L'outil signale	**Vrai positif** : on corrige	**Faux positif** : bruit, l'équipe finit par ignorer l'outil
L'outil ne dit rien	**Faux négatif** : fausse confiance	**Vrai négatif**

- **Faux positif typique :** alerte « `null` possible » sur une variable toujours initialisée par un framework (injection de dépendances)
- **Faux négatif typique :** débordement de `b * b` en `double` (+∞) dans le solveur : aucun motif syntaxique ne le révèle

Deux philosophies
Les outils de **motifs** (SpotBugs, PMD, Sonar) acceptent des faux négatifs pour limiter le bruit. Les analyseurs **corrects** (sound) comme Astrée ou Frama-C EVA garantissent zéro faux négatif pour une classe d'erreurs, au prix de fausses alarmes (séance 11).

QUATRE DÉFAUTS, QUATRE ALERTES

Defauts.java

```java
static int milieu(int bas, int haut) {
    return (bas + haut) / 2;            // (1)
}
static boolean estTiers(double x) {
    return x == 1.0 / 3;                // (2)
}
static String premiereLigne(String f)
        throws IOException {
    BufferedReader br =
        new BufferedReader(new FileReader(f));
    return br.readLine();               // (3)
}
static int longueurNom(Map<String, String> m,
                       String id) {
    return m.get(id).length();          // (4)
}
```

- **(1) :** `milieu(1 500 000 000, 2 000 000 000)` renvoie **−397 483 648** (séance 2)
- **(2) :** `estTiers(1 - 2.0/3)` renvoie **false** : 0,33333333333333337 ≠ 0,3333333333333333
- **(3) :** le flux n'est jamais fermé : fuite de descripteurs
- **(4) :** `NullPointerException` si l'identifiant est absent

CE QUE SIGNALENT LES OUTILS

Défaut	SpotBugs (règle typique)	SonarQube (règle)
(1) moyenne qui déborde	`IM_AVERAGE_COMPUTATION_ COULD_OVERFLOW`	selon le contexte
(2) égalité flottante	`FE_FLOATING_POINT_EQUALITY`	`S1244`
(3) flux non fermé	`OBL_UNSATISFIED_OBLIGATION`, `OS_OPEN_STREAM`	`S2095`
(3 bis) encodage implicite	`DM_DEFAULT_ENCODING` (`FileReader` sans charset)	—
(4) déréférencement de `null`	dépend des annotations `@Nullable`	`S2259` (si le chemin est visible)

Leçon
Le défaut (4) n'est trouvé que si le contrat de `Map.get` (peut renvoyer `null`) est connu de l'outil : les **annotations de nullité** (JSpecify, NullAway) rendent l'analyse bien plus efficace.

PANORAMA DES OUTILS (2026)

Langage	Style / conventions	Bugs / qualité	Sécurité
Java	Checkstyle	SpotBugs, PMD, Error Prone	Find Security Bugs, OWASP Dependency-Check
Python	Ruff	Ruff, Pylint, mypy (types)	Bandit, pip-audit
JavaScript / TS	ESLint, Prettier	ESLint, TypeScript (mode strict)	eslint-plugin-security, npm audit
C / C++	clang-format, clang-tidy	Clang Static Analyzer, cppcheck	Frama-C, CodeQL
Multi-langages	—	SonarQube / SonarLint	Semgrep, CodeQL, Snyk

Mise à jour du cours précédent

FindBugs n'est plus maintenu (dernière version 2015) : son successeur est **SpotBugs**. JSHint et Flake8/Pyflakes sont largement remplacés par ESLint et Ruff.

INTÉGRER LES OUTILS DANS LE BUILD MAVEN

pom.xml (extrait)

```
<plugin>
  <groupId>com.github.spotbugs</groupId>
  <artifactId>spotbugs-maven-plugin</artifactId>
  <version>4.8.6.6</version>
  <executions><execution>
    <goals><goal>check</goal></goals>
  </execution></executions>
</plugin>
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-pmd-plugin</artifactId>
  <version>3.26.0</version>
  <executions><execution>
    <goals>
      <goal>check</goal><goal>cpd-check</goal>
    </goals>
  </execution></executions>
</plugin>
<!-- idem : maven-checkstyle-plugin, goal check -->
```

- `mvn verify` **échoue** si une règle bloquante est violée
- `cpd-check` : détection de copier-coller (PMD)
- Rapports XML dans `target/` (SpotBugs, PMD, Checkstyle)
- `mvn spotbugs:gui` pour explorer les alertes
- Vérifier les dernières versions des plugins avant usage

LA QUALITY GATE SONARQUBE

- Une **quality gate** est un ensemble de conditions que le code doit remplir pour être livré
- Gate par défaut « Sonar way », appliquée au **nouveau code** :
  - couverture ≥ 80 %
  - duplication ≤ 3 %
  - notes fiabilité, sécurité, maintenabilité = A
  - 100 % des points sensibles de sécurité revus
- Le pipeline CI **échoue** si la gate échoue

```
# dans le pipeline CI (séance 9)
mvn verify sonar:sonar \
  -Dsonar.host.url=$SONAR_URL \
  -Dsonar.token=$SONAR_TOKEN \
  -Dsonar.qualitygate.wait=true
```

Se concentrer sur le **nouveau code** (« clean as you code ») évite d'exiger la correction immédiate de toute la dette existante.

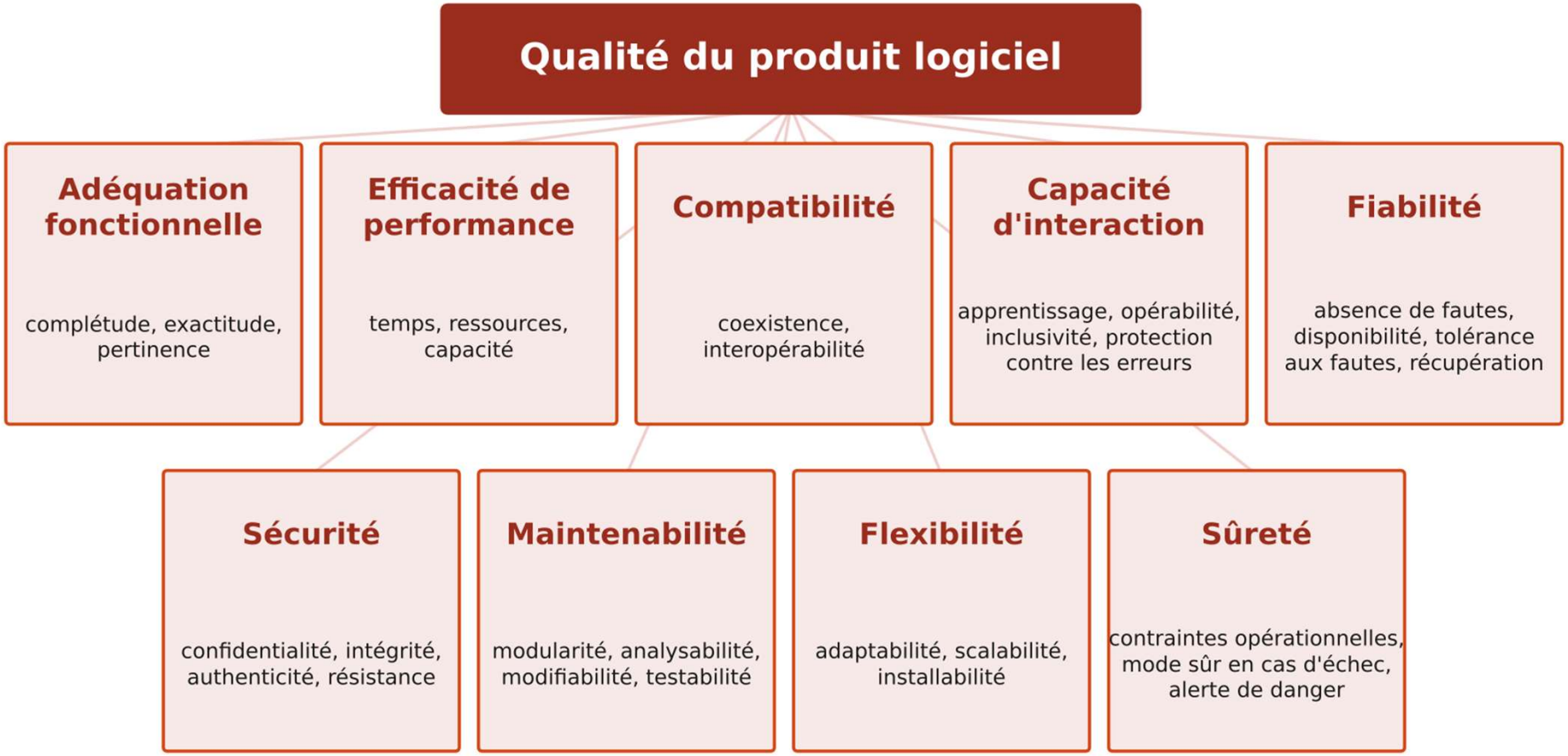
TP : OUTILLER LE SOLVEUR DU SECOND DEGRÉ

TP

- Reprendre le projet Maven du **solveur** (séances 2 à 5)
- Intégrer **Checkstyle, PMD et SpotBugs** dans `mvn verify` (remplace FindBugs du TP d'origine)
- Lancer le build, lire les trois rapports, **classer** chaque alerte : vrai positif, faux positif, style
- Corriger les vrais positifs ; justifier par écrit chaque alerte ignorée (`@SuppressFBWarnings` + raison)
- Ajouter l'analyse SonarQube (ou SonarCloud) et une **quality gate** au pipeline CI
- Question : quelles erreurs numériques de la séance 2 les outils ont-ils trouvées ? Lesquelles ont-ils ratées ?

Livrable : `pom.xml`, rapport court (tableau des alertes et décisions), pipeline vert.

QUALITÉ PRODUIT : ISO/IEC 25010:2023



9 caractéristiques. Nouveautés 2023 : capacité d'interaction (ex-utilisabilité), flexibilité (ex-portabilité), sûreté.

MÉTRIQUES DE CODE

Métrique	Ce qu'elle mesure	Repère	Outil
Complexité cyclomatique	Nombre de chemins indépendants (séance 5)	≤ 10 par méthode	PMD, Sonar
Complexité cognitive	Difficulté de lecture (imbrications)	≤ 15	Sonar
Couplage (CBO)	Nombre de classes dont une classe dépend	faible	CK, Sonar
Cohésion (LCOM)	Les méthodes d'une classe partagent-elles leurs attributs ?	forte cohésion	CK
Duplication	Part de code copié-collé	≤ 3 %	PMD CPD, Sonar
Taille	Lignes par méthode, par classe	méthodes courtes	Checkstyle

Une métrique est un **indicateur**, pas un objectif : optimiser un chiffre (« 100 % de couverture ») sans regarder ce qu'il mesure produit des tests sans assertion.

DETTE TECHNIQUE ET CODE SMELLS

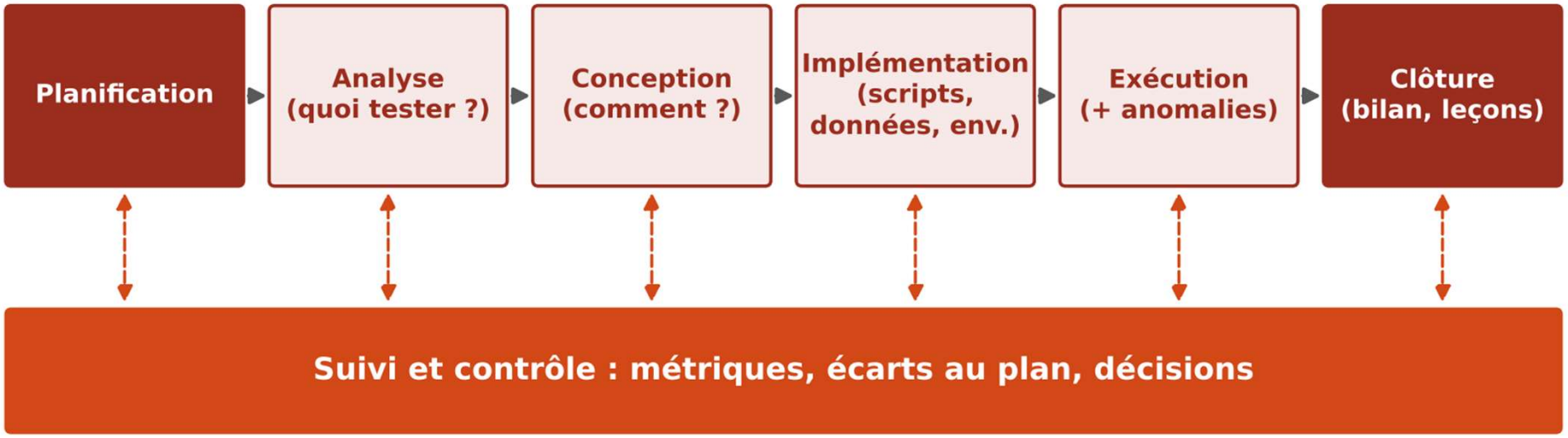
D

Dette technique (Ward Cunningham, 1992)
Coût futur des raccourcis pris aujourd'hui : chaque modification ultérieure paie des « intérêts » tant que la dette n'est pas remboursée par du refactoring.

- **Nombre magique :** `montant.multiply(new BigDecimal("0.008"))` : la règle de frais est cachée dans le code
- **Obsession des primitifs :** un montant en FCFA représenté par un `double` au lieu d'un type `Montant`
- **Méthode trop longue, classe dieu :** `TransfertService` qui calcule, valide, journalise et notifie
- **Duplication :** la même vérification de plafond copiée dans trois méthodes

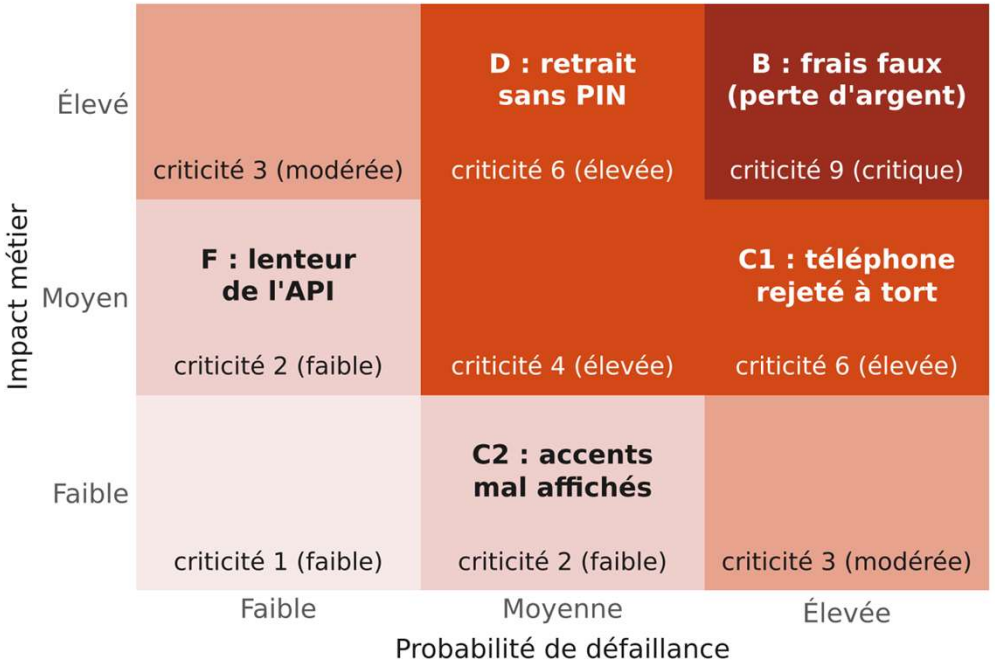
LE PROCESSUS DE TEST (ISTQB, ISO/IEC/IEEE 29119-2)

Les activités se recouvrent et s'itèrent ; en agile, elles se répètent à chaque sprint.



Analyse : quoi tester (conditions de test, à partir des exigences et des risques). **Conception** : comment (cas de test, techniques des séances 4 à 6). **Implémentation** : prêt à exécuter (scripts, données, environnement).

STRATÉGIE DE TEST BASÉE SUR LES RISQUES



- **Risque = probabilité × impact**
- L'effort de test est **proportionnel** au risque
- **Criticité 9 :** frais Mobile Money : MC/DC, mutation ≥ 80 %, inspection
- **Criticité 6 :** retrait sans PIN : transitions d'états complètes, model checking (séance 11)
- **Criticité 2 :** accents à l'affichage : test exploratoire
- Le classement guide aussi l'**ordre** d'exécution

CONTENU D'UN PLAN DE TEST (ISO/IEC/IEEE 29119-3)

Rubrique	Contenu
Contexte	Projet, périmètre, éléments testés et non testés, hypothèses et contraintes
Parties prenantes	Qui fournit, qui valide, qui est informé
Registre des risques	Risques produit et projet, avec leur criticité
Stratégie de test	Niveaux, types, techniques, critères d'entrée et de sortie, automatisation
Données et environnement	Jeux de données, environnements, outils, besoins matériels
Activités et planning	Tâches, estimations, jalons, livrables
Organisation	Rôles, responsabilités, besoins en formation
Métriques et rapports	Ce qui est mesuré, fréquence et destinataires des rapports

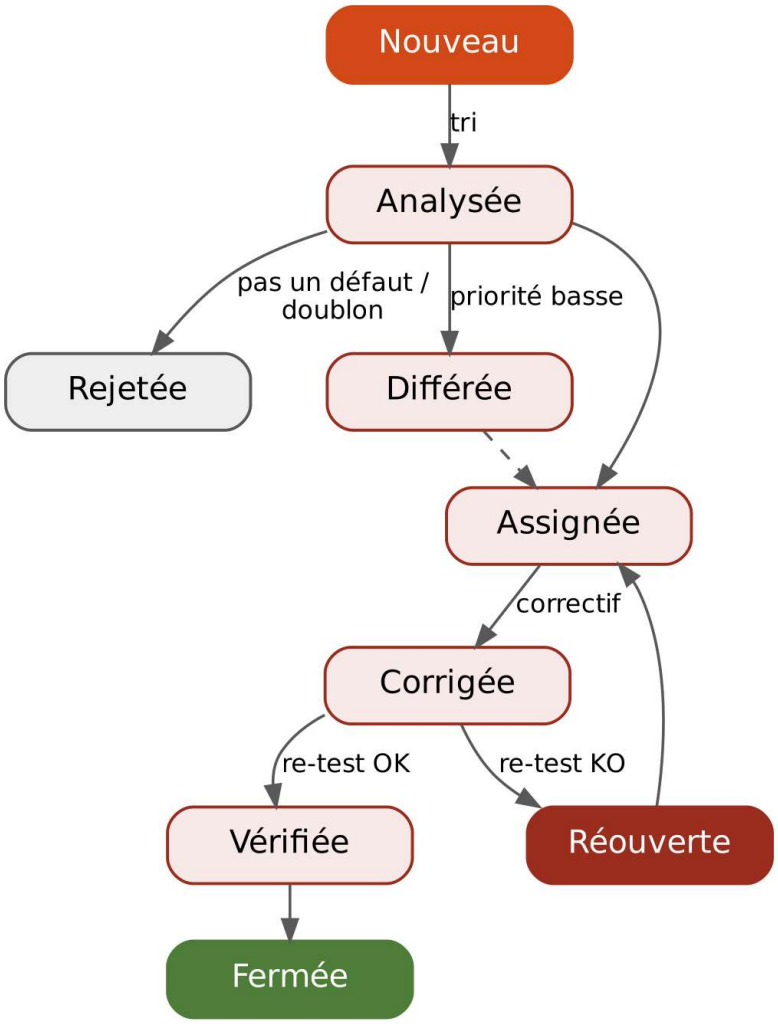
CRITÈRES D'ENTRÉE, DE SORTIE ET ESTIMATION

- **Critères d'entrée :**
  - environnement disponible, build installé
  - tests unitaires au vert, smoke test passé
- **Critères de sortie :**
  - aucune anomalie critique ou majeure ouverte
  - 100 % des exigences à risque élevé couvertes
  - score de mutation ≥ seuil sur le code critique

- **Estimer l'effort :**
  - **ratios** historiques (ex. effort de test / effort de dev)
  - **extrapolation** à partir des premières mesures
  - **Wideband Delphi**, planning poker (experts)
  - **trois points** : E = (o + 4m + p) / 6

Exemple trois points : optimiste 4 j, probable 6 j, pessimiste 14 j → E = (4 + 24 + 14) / 6 = **7 jours**.

CYCLE DE VIE D'UNE ANOMALIE



- **Analysée :** triée : reproductible ? doublon ? vraiment un défaut ?
- **Rejetée :** comportement conforme, ou doublon
- **Différée :** reconnue mais reportée (priorité basse)
- **Corrigée :** le développeur a livré un correctif
- **Vérifiée :** **re-test** OK + non-régression autour
- **Réouverte :** le re-test échoue
- Chaque transition est **datée et attribuée** : c'est la source des métriques

SÉVÉRITÉ ET PRIORITÉ : DEUX AXES DISTINCTS

Sévérité = impact technique sur le système. **Priorité** = urgence métier de la correction.

Sévérité	Priorité	Exemple
Critique	Haute	**B** : frais calculés à 0 FCFA au-delà de 50 000 : perte d'argent à chaque transfert
Critique	Basse	**E** : la simulation diverge pour un pas h > 10 s, jamais utilisé en exploitation
Mineure	Haute	**F** : nom de l'université mal orthographié sur la page d'accueil, la veille de l'inauguration
Mineure	Basse	**C** : info-bulle du champ téléphone mal alignée sur petit écran

UN RAPPORT D'ANOMALIE COMPLET

Champ	Contenu
Identifiant, titre	**SCO-142** · `GET /etudiants/{id}/notes` renvoie une moyenne non pondérée par les crédits
Environnement	API v1.4.2, environnement de recette, Java 21, PostgreSQL 16
Préconditions	Étudiant 24G1234 : INF563 = 12 (6 crédits), ANG501 = 18 (2 crédits)
Étapes	Appeler `GET /etudiants/24G1234/notes` avec un jeton JWT valide
Résultat attendu	moyenne = (12 × 6 + 18 × 2) / 8 = **13,50** (exigence EX-F3)
Résultat obtenu	moyenne = **15,00** (moyenne simple des notes)
Sévérité / priorité	Majeure / Haute (délibérations dans 5 jours)
Pièces jointes	Requête et réponse JSON, extrait de journal, version du build

Un bon rapport est **reproductible**, **factuel** (attendu vs obtenu), **unique** (une anomalie par rapport) et relié à une **exigence**.

MATRICE DE TRAÇABILITÉ EXIGENCES ↔ TESTS

Exigence	Tests	Dernier résultat	Couverte ?
EX-B1 frais = 50 FCFA de 100 à 5 000	TC-01, TC-02	PASS	Oui
EX-B2 frais = 1 % de 5 001 à 50 000	TC-03, TC-04	PASS	Oui
EX-B3 0,8 % plafonné à 2 500 au-delà	TC-05	FAIL	Oui
EX-B4 refus hors [100 ; 500 000]	TC-06, TC-07	PASS	Oui
EX-B5 plafond journalier 1 000 000	—	—	**Non**
EX-B6 conservation des soldes	TC-08 (propriété)	PASS	Oui

La matrice répond à deux questions : **quelles exigences ne sont pas testées** (EX-B5) et **quels tests rejouer** si une exigence change (traçabilité inverse).

MÉTRIQUES DE TEST ET DE FIABILITÉ

Métrique	Définition	Exemple
Densité de défauts	défauts / taille (KLOC ou points de fonction)	18 défauts / 12 KLOC = 1,5
Efficacité de détection (DDE)	défauts trouvés avant livraison / total des défauts	45 / (45 + 5) = **90 %**
Taux de passage	tests réussis / tests exécutés	188 / 200 = 94 %
MTTF	durée moyenne de fonctionnement **avant** une défaillance	720 h
MTTR	durée moyenne de **réparation** (remise en service)	4 h
MTBF	durée moyenne **entre** deux défaillances ≈ MTTF + MTTR	724 h
Disponibilité	MTTF / (MTTF + MTTR)	720 / 724 ≈ **99,45 %**

Correction du support précédent

Le « temps moyen **entre** défaillances » est le **MTBF**. Le MTTF est le temps moyen **avant** défaillance.

OUTILS ET AMÉLIORATION DU PROCESSUS

Outil	Usage
Jira + Xray / Zephyr	cas de test, campagnes, anomalies liées
Squash TM, TestLink, Kiwi TCMS	gestion de tests open source
GitHub / GitLab Issues	anomalies au plus près du code
Allure, rapports Surefire	rapports d'exécution automatisés

CMMI	Niveau de maturité
1	Initial : processus imprévisibles
2	Géré : projets planifiés et suivis
3	Défini : processus standard de l'organisation
4	Géré quantitativement : pilotage par les mesures
5	En optimisation : amélioration continue

PDCA (roue de Deming)
Plan (objectif, métrique) → **Do** (appliquer) → **Check** (mesurer : DDE, densité) → **Act** (standardiser ou corriger). Exemple : ajouter la checklist de revue, mesurer la DDE sur 3 sprints.

À RETENIR

À RETENIR

- Revues et analyse statique trouvent **tôt** des défauts que le test ne voit pas, et inversement
- Un outil statique vérifie que le code est **bien écrit**, pas qu'il respecte la spécification
- Faux positifs : configurer les règles, sinon l'équipe ignore l'outil ; quality gate sur le **nouveau code**
- ISO/IEC 25010:2023 : **9 caractéristiques** pour ne rien oublier dans la stratégie
- L'effort de test suit le **risque** (probabilité × impact)
- Anomalies : sévérité ≠ priorité ; rapport reproductible ; traçabilité vers les exigences
- MTTF (avant), MTTR (réparation), MTBF (entre) ; DDE mesure l'efficacité du test

INITIATION AUX MÉTHODES FORMELLES

244

Séance 11 · 4 h · Spécifier, prouver, explorer tous les états, analyser toutes les exécutions

CHAP XI

OBJECTIFS DE LA SÉANCE

- Écrire des **contrats** (pré, post, invariants) en JML
- Prouver un petit programme avec la **logique de Hoare**
- Calculer une **plus faible précondition**
- Spécifier un système à états en **TLA+** et lire un contre-exemple
- Comprendre l'**interprétation abstraite** et la preuve d'absence de débordement
- Savoir **quand** les méthodes formelles valent leur coût

Lien avec le cours

Les tests **échantillonnent** les entrées. Les méthodes formelles raisonnent sur **toutes** les entrées, pour une propriété donnée, sur un modèle donné.

POURQUOI ALLER AU-DELÀ DU TEST

2^{64}

entrées possibles pour `f(int a, int b)`

$\approx$ 585 ans

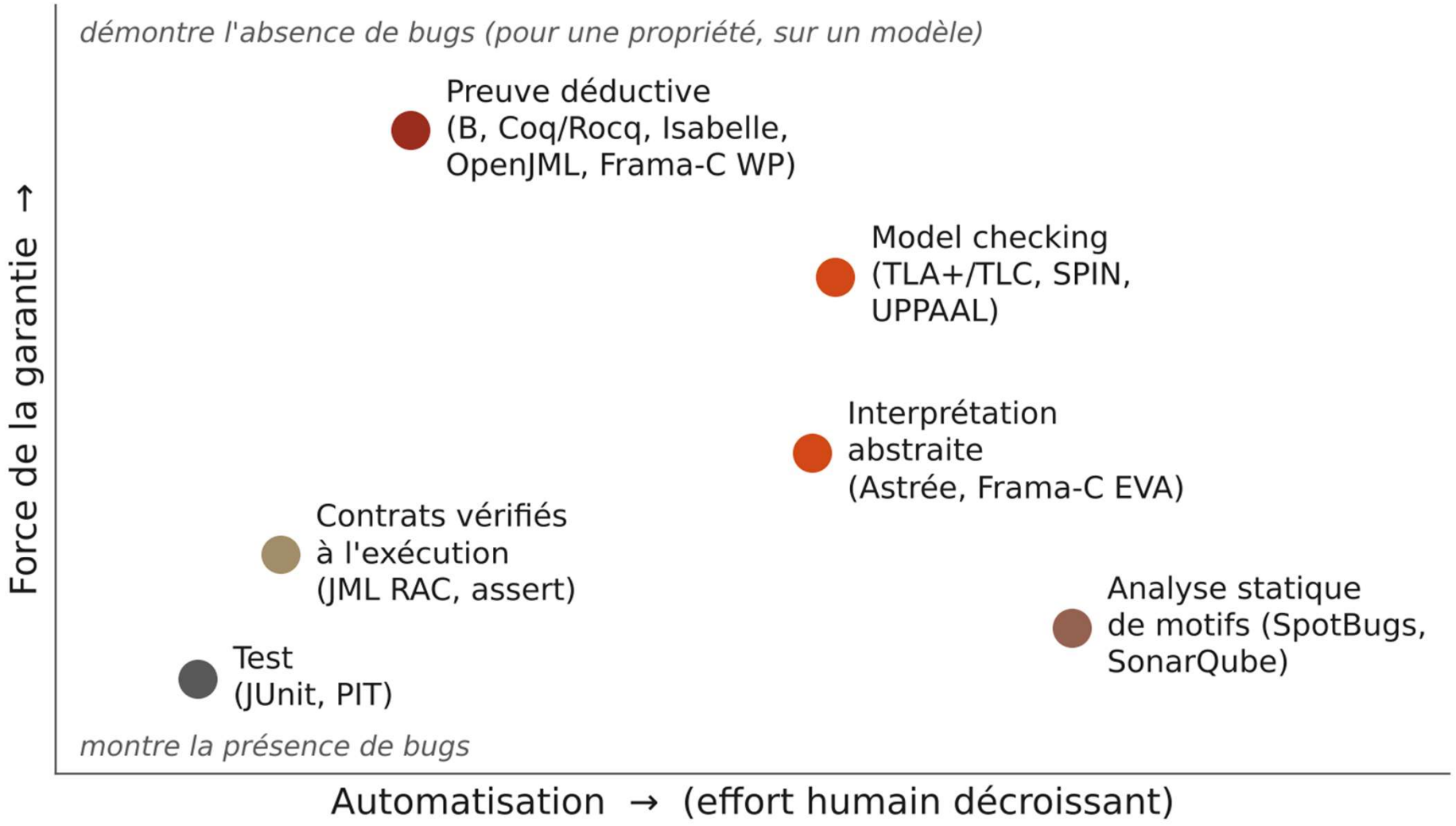
pour toutes les tester à un milliard de tests par seconde

1

seul contre-exemple suffit à invalider le code

- « Le test de programmes peut servir à montrer la présence de bugs, mais jamais leur absence » (E. W. Dijkstra, 1970)
- Les classes d'équivalence (séance 4) reposent sur une **hypothèse** d'uniformité, jamais prouvée
- Une preuve formelle couvre **toutes** les exécutions, mais seulement pour la propriété et le modèle écrits

PANORAMA DES TECHNIQUES



Plus la garantie est forte, plus il faut d'effort humain (spécifier, guider la preuve) ou restreindre la classe de propriétés.

QUATRE FAMILLES DE MÉTHODES

1 Spécification formelle

- Décrire **ce que** doit faire le système dans un langage mathématique : B, Z, TLA+, Alloy, JML, ACSL
- Lever les ambiguïtés avant de coder

2 Preuve déductive

- Démontrer que le code respecte sa spécification : logique de Hoare, plus faible précondition
- Outils : Atelier B, Coq/Rocq, Isabelle, Frama-C WP, OpenJML

3 Model checking

- Explorer **tous les états** d'un modèle fini et vérifier des propriétés temporelles
- Contre-exemple automatique. Outils : TLC, SPIN, UPPAAL

4 Interprétation abstraite

- Calculer une **sur-approximation** de toutes les exécutions (Cousot, 1977)
- Prouve l'absence d'erreurs à l'exécution. Outils : Astrée, Frama-C EVA

DES SUCCÈS INDUSTRIELS

Projet	Méthode	Résultat
Météor, ligne 14 du métro de Paris (1998)	Méthode B (Atelier B)	Logiciel de sécurité (environ 86 000 lignes d'Ada) généré à partir de modèles prouvés ; aucun défaut trouvé dans ce logiciel après la preuve
Airbus A340 puis A380 (2003–2005)	Interprétation abstraite (Astrée)	Absence d'erreurs à l'exécution prouvée dans le logiciel de commandes de vol écrit en C
Amazon Web Services (depuis 2011)	TLA+ et model checker TLC	Bugs de conception subtils trouvés dans S3, DynamoDB…, dont un nécessitant une trace de 35 étapes
seL4 (2009)	Preuve (Isabelle/HOL)	Correction fonctionnelle prouvée d'un micro-noyau d'environ 8 700 lignes de C

PROGRAMMATION PAR CONTRAT

D

Contrat (Bertrand Meyer, Eiffel, années 1980)

Une méthode et son appelant sont liés par un contrat : si le **client** respecte la précondition, le **fournisseur** garantit la postcondition, et l'objet respecte toujours ses invariants.

Élément	Obligation de…	Exemple (Mobile Money)
Précondition	l'appelant	montant entre 100 et 500 000 FCFA
Postcondition	la méthode	frais > 0 et frais ≤ 2 500
Invariant	la classe (avant et après chaque méthode publique)	solde ≥ 0 ; total des soldes + frais collectés constant

JML : LE CONTRAT DES FRAIS

Frais.java

```
/*@ public normal_behavior
  @   requires montant != null
  @     && montant.compareTo(MIN) >= 0
  @     && montant.compareTo(MAX) <= 0;
  @   ensures \result.signum() > 0
  @     && \result.compareTo(PLAFOND) <= 0;
  @ also public exceptional_behavior
  @   requires montant == null
  @     || montant.compareTo(MIN) < 0
  @     || montant.compareTo(MAX) > 0;
  @   signals_only MontantInvalideException;
  @*/
public static BigDecimal frais(BigDecimal montant)
        throws MontantInvalideException { ... }
```

- Le contrat est un **commentaire** `/*@ … @*/` : le code compile avec `javac`
- `normal_behavior` : cas nominal, aucune exception
- `exceptional_behavior` : quand et quelle exception
- `\result` : la valeur renvoyée
- Exécuté : frais(50 000) = 500, frais(50 001) = **400**, frais(500 000) = 2 500

Le contrat ne dit pas que les frais **croissent** avec le montant : 50 001 FCFA coûte moins cher que 50 000. Spécifier oblige à se poser la question au client.

JML : INVARIANTS DU GAB

Gab.java

```
public class Gab {
  //@ public invariant 0 <= essais && essais <= 3;
  //@ public invariant etat == Etat.RETRAIT ==> pinOK;
  //@ public invariant essais == 3
  //@                  ==> etat == Etat.BLOQUEE;
  private /*@ spec_public @*/ Etat etat;
  private /*@ spec_public @*/ int essais;
  private /*@ spec_public @*/ boolean pinOK;

  /*@ requires etat == Etat.ATTENTE_PIN;
    @ assignable etat, essais, pinOK;
    @ ensures correct ==> etat == Etat.MENU && pinOK;
    @ ensures !correct ==> essais == \old(essais) + 1;
    @*/
  public void saisirPin(boolean correct) { ... }
}
```

- **Invariants de classe** : vrais après chaque méthode publique
- `==>` : implication
- `\old(e)` : valeur de e à l'entrée de la méthode
- `assignable` : ce que la méthode a le droit de modifier (cadre)
- `spec_public` : champ privé visible dans la spécification

VÉRIFIER UN CONTRAT : À L'EXÉCUTION OU STATIQUEMENT

	À l'exécution (RAC)	Statique (ESC, preuve)
Outils	`assert` Java, OpenJML `-rac`, Guava Preconditions	OpenJML `-esc` + solveur SMT, KeY, Frama-C WP
Couvre	les exécutions **testées**	**toutes** les exécutions
Coût	faible : ajouter des assertions	élevé : invariants de boucle, annotations
Résultat	défaillance détectée au plus tôt	preuve, ou contre-exemple / échec de preuve

Piège
Les `assert` Java sont **désactivés par défaut**. Exécuté : `diviser(-3, 5)` lève `AssertionError` avec `java -ea`, mais renvoie silencieusement q = 0, r = −3 sans `-ea`.

LOGIQUE DE HOARE

D

Triplet de Hoare {P} S {Q} (C. A. R. Hoare, 1969)

Si la précondition **P** est vraie avant d'exécuter **S**, et si S termine, alors la postcondition **Q** est vraie après.

- **Correction partielle** : {P} S {Q} (si S termine)
- **Correction totale** : correction partielle **et** terminaison prouvée
- Exemple : { x = 5 } x := x + 1 { x = 6 }
- Exemple faux : { vrai } y := 10 / x { y > 0 } (x = 0 ou x < 0)
- Les règles d'inférence permettent de **calculer** au lieu de deviner

LES RÈGLES DE LA LOGIQUE DE HOARE

Règle	Schéma	Lecture
Affectation	$\{ Q[x \leftarrow E] \}\ x := E\ \{ Q \}$	Q avec E à la place de x doit être vraie avant
Séquence	$\{P\}\ S_1\ \{R\}$ et $\{R\}\ S_2\ \{Q\} \vdash \{P\}\ S_1; S_2\ \{Q\}$	on enchaîne par une assertion intermédiaire R
Conditionnelle	$\{P \wedge B\}\ S_1\ \{Q\}$ et $\{P \wedge \neg B\}\ S_2\ \{Q\} \vdash \{P\}$ if B then S_1 else S_2 $\{Q\}$	chaque branche doit établir Q
Boucle	$\{I \wedge B\}\ S\ \{I\} \vdash \{I\}$ while B do S $\{I \wedge \neg B\}$	I est l'**invariant de boucle**
Conséquence	$P \Rightarrow P', \{P'\}\ S\ \{Q'\}, Q' \Rightarrow Q \vdash \{P\}\ S\ \{Q\}$	renforcer la pré, affaiblir la post

PLUS FAIBLE PRÉCONDITION (DIJKSTRA, 1975)

wp(S, Q) est la précondition la plus faible qui garantit Q après S. Prouver {P} S {Q} revient à montrer **P ⇒ wp(S, Q)**.

Programme S	Postcondition Q	wp(S, Q)
x := x + 1	x > 0	x + 1 > 0, soit **x ≥ 0** (entiers mathématiques)
x := x + 1 (int Java)	x > 0	**0 ≤ x < Integer.MAX_VALUE** (sinon débordement, séance 2)
y := 2 * x; z := y − 1	z > 5	2x − 1 > 5, soit **x > 3**
if x < 0 then y := −x else y := x	y ≥ 0	vrai… sauf en Java pour **x = Integer.MIN_VALUE**

PREUVE COMPLÈTE (1/3) : LE PROBLÈME

```
{ x ≥ 0 ∧ y > 0 }      -- P
q := 0;
r := x;
while (r ≥ y) {
    r := r - y;
    q := q + 1;
}
{ x = q·y + r ∧ 0 ≤ r < y }  -- Q
```

- Division entière par soustractions : l'**exemple de l'article de Hoare (1969)**
- Q dit : q est le quotient et r le reste
- **Invariant choisi : I ≡ x = q·y + r ∧ r ≥ 0**
- Idée : I « affaiblit » Q en oubliant r < y, qui ne sera vrai qu'à la sortie
- Trois choses à prouver : **initialisation, préservation, sortie** ; puis la terminaison

PREUVE COMPLÈTE (2/3) : INITIALISATION ET PRÉSERVATION

- **1. Initialisation :** montrer $\{P\}\ q := 0; r := x\ \{I\}$
  - $wp(q := 0; r := x, I) = x = 0 \cdot y + x \wedge x \geq 0$, c'est-à-dire $\mathbf{x \geq 0}$
  - $P \Rightarrow x \geq 0$: **vrai**
- **2. Préservation :** montrer $\{I \wedge r \geq y\}\ r := r - y; q := q + 1\ \{I\}$
  - $wp(q := q + 1, I) = x = (q + 1) \cdot y + r \wedge r \geq 0$
  - $wp(r := r - y, \ldots) = x = (q + 1) \cdot y + (r - y) \wedge r - y \geq 0$
  - ce qui se simplifie en $\mathbf{x = q \cdot y + r \wedge r \geq y}$
  - $I \wedge r \geq y$ implique exactement cela : **vrai**

Rappel
Invariant $I : x = q \cdot y + r \wedge r \geq 0$; condition de boucle $r \geq y$; $P : x \geq 0 \wedge y > 0$.

PREUVE COMPLÈTE (3/3) : SORTIE ET TERMINAISON

- **3. Sortie :** $I \wedge \neg(r \geq y) \Rightarrow Q$?
  - $x = q \cdot y + r \wedge r \geq 0 \wedge r < y$: c'est **Q**
- **4. Terminaison :** variant **v = r**
  - $v \geq 0$ tant que la boucle tourne (par I)
  - v diminue de $y > 0$ à chaque tour
  - donc la boucle s'arrête
- **Correction totale** prouvée pour **toutes** les entrées vérifiant P

```
static int[] diviser(int x, int y) {
  assert x >= 0 && y > 0;            // P
  int q = 0, r = x;
  while (r >= y) {
    assert x == q*y + r && r >= 0; // I
    r = r - y; q = q + 1;
  }
  assert x == q*y + r
      && 0 <= r && r < y;            // Q
  return new int[] { q, r };
}
```

Les mêmes formules deviennent des **assertions** vérifiées à l'exécution : `diviser(17, 5)` renvoie q = 3, r = 2 (exécuté avec `-ea`).

MODEL CHECKING

D

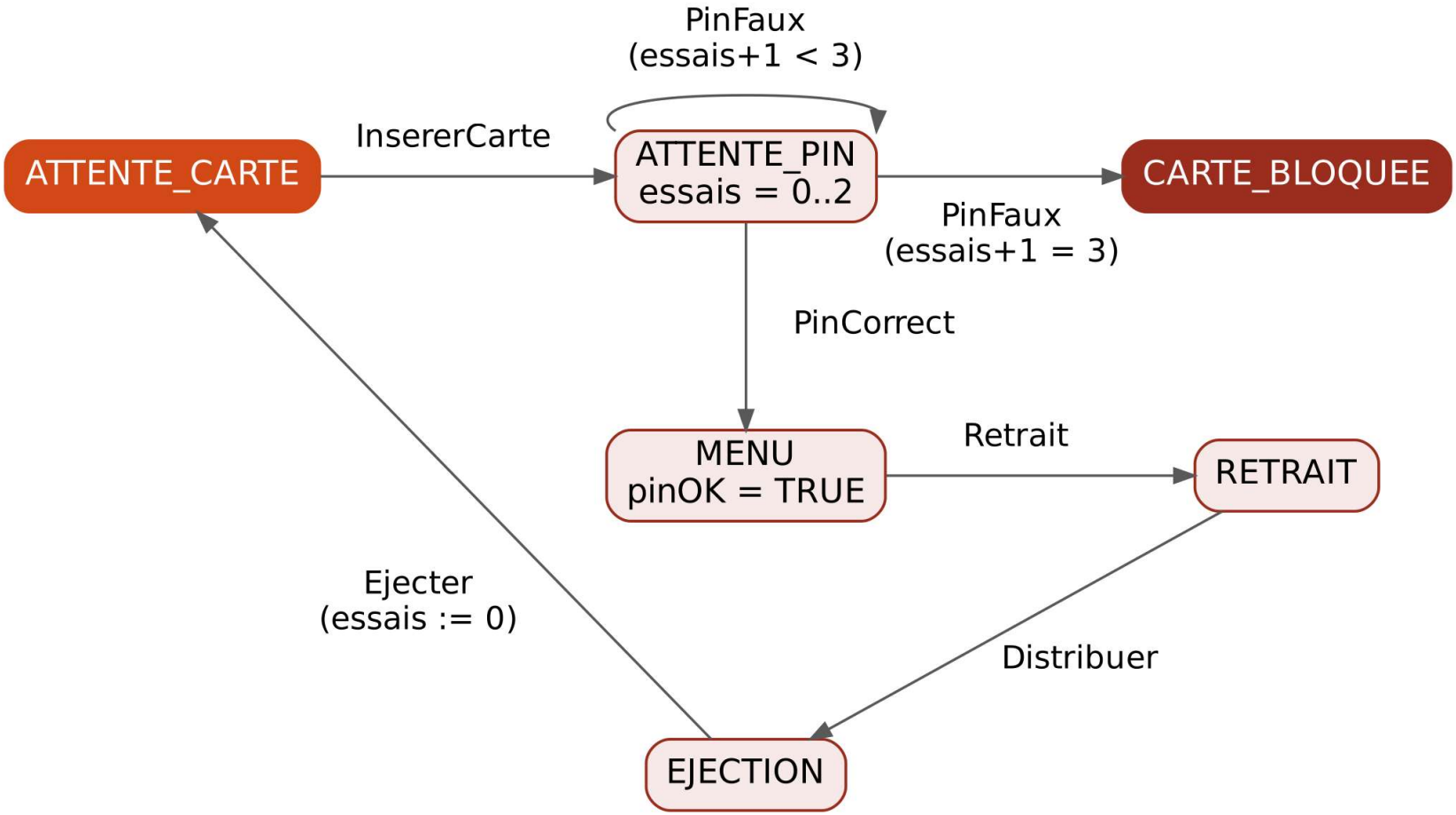
Model checking (Clarke, Emerson, Sifakis — prix Turing 2007)

Explorer **automatiquement et exhaustivement** tous les états atteignables d'un modèle fini pour vérifier une propriété ; en cas d'échec, l'outil fournit un **contre-exemple** (une trace).

Propriété	Sens	Exemple GAB	Contre-exemple
Sûreté (safety)	rien de mauvais n'arrive	jamais de retrait sans PIN validé	une trace finie
Vivacité (liveness)	quelque chose de bon finit par arriver	toute session entamée se termine	une boucle infinie

LE MODÈLE DU GAB

Trois variables : `etat`, `essais` (0 à 3), `pinOK` (booléen). Six actions. Ce graphe est ce que le model checker explore.



Graphe d'états du modèle TLA+ (MaxEssais = 3). CARTE_BLOQUEE est un état terminal.

LA SPÉCIFICATION TLA+ (1/2)

GAB.tla

```
---------------------- MODULE GAB ----------------------
EXTENDS Naturals
CONSTANT MaxEssais
VARIABLES etat, essais, pinOK
vars == <<etat, essais, pinOK>>

Init == etat = "ATTENTE_CARTE" /\ essais = 0 /\ pinOK = FALSE

InsererCarte == /\ etat = "ATTENTE_CARTE" /\ etat' = "ATTENTE_PIN"
                /\ UNCHANGED <<essais, pinOK>>
PinCorrect   == /\ etat = "ATTENTE_PIN" /\ etat' = "MENU"
                /\ pinOK' = TRUE /\ UNCHANGED essais
PinFaux      == /\ etat = "ATTENTE_PIN" /\ essais' = essais + 1
                /\ etat' = IF essais + 1 = MaxEssais
                           THEN "CARTE_BLOQUEE" ELSE "ATTENTE_PIN"
                /\ UNCHANGED pinOK
Retrait      == /\ etat = "MENU" /\ etat' = "RETRAIT"
                /\ UNCHANGED <<essais, pinOK>>
Distribuer   == /\ etat = "RETRAIT" /\ etat' = "EJECTION"
                /\ UNCHANGED <<essais, pinOK>>
Ejecter      == /\ etat = "EJECTION" /\ etat' = "ATTENTE_CARTE"
                /\ essais' = 0
                /\ pinOK' = FALSE
```

LA SPÉCIFICATION TLA+ (2/2) ET LE VERDICT

```
Next == InsererCarte \/ PinCorrect \/ PinFaux
        \/ Retrait \/ Distribuer \/ Ejecter
Spec == Init /\ [][Next]_vars /\ WF_vars(Next)

\* Sûreté : invariants
PasDeRetraitSansPin ==
    etat \in {"RETRAIT", "EJECTION"} => pinOK
BlocageApresEchecs ==
    essais >= MaxEssais => etat = "CARTE_BLOQUEE"

\* Vivacité
SessionTerminee == (etat = "ATTENTE_PIN")
    ~> (etat \in {"ATTENTE_CARTE", "CARTE_BLOQUEE"})
=============================================
```

- **GAB.cfg :** `MaxEssais = 3`, deux invariants, une propriété
- `=>` : implication ; `~>` : « mène à » (finit par)
- `WF_vars(Next)` : équité faible
- **Résultat TLC (exécuté) :**
  - 17 états générés, **14 distincts**, profondeur 7
  - « Model checking completed. **No error has been found.** »

UN BUG, UN CONTRE-EXEMPLE

On introduit une erreur de borne : `IF essais + 1 > MaxEssais` au lieu de `= MaxEssais`.

```
Error: Invariant BlocageApresEchecs is violated.
State 1: <Initial predicate>
/\ pinOK = FALSE
/\ etat = "ATTENTE_CARTE"
/\ essais = 0
State 2: <InsererCarte ...>
/\ etat = "ATTENTE_PIN"
/\ essais = 0
State 3: <PinFaux ...>
/\ essais = 1
State 4: <PinFaux ...>
/\ essais = 2
State 5: <PinFaux ...>
/\ etat = "ATTENTE_PIN"
/\ essais = 3
```

- TLC trouve la **plus courte** trace qui viole l'invariant
- Après 3 PIN faux, la carte **n'est pas bloquée** : un 4e essai est possible
- C'est une erreur de **valeur limite** (séance 4), trouvée ici sans écrire un seul cas de test
- La trace se rejoue telle quelle comme **test de non-régression** sur le code Java

L'EXPLOSION COMBINATOIRE

14

états pour le GAB à 3 variables

2^n

états pour n variables booléennes : $2^{64} \approx 1{,}8 \times 10^{19}$ pour 64

× 41

au plus, si l'on ajoute un montant de 0 à 200 000 par pas de 5 000

- Le nombre d'états est le **produit** des domaines des variables (et des entrelacements des processus)
- **Abstraction** : ne modéliser que ce qui compte pour la propriété (pas les montants exacts)
- **Bornes** et symétries : 3 clients et 2 GAB suffisent souvent à trouver les bugs
- Techniques symboliques : BDD, model checking borné avec solveurs SAT/SMT

INTERPRÉTATION ABSTRAITE

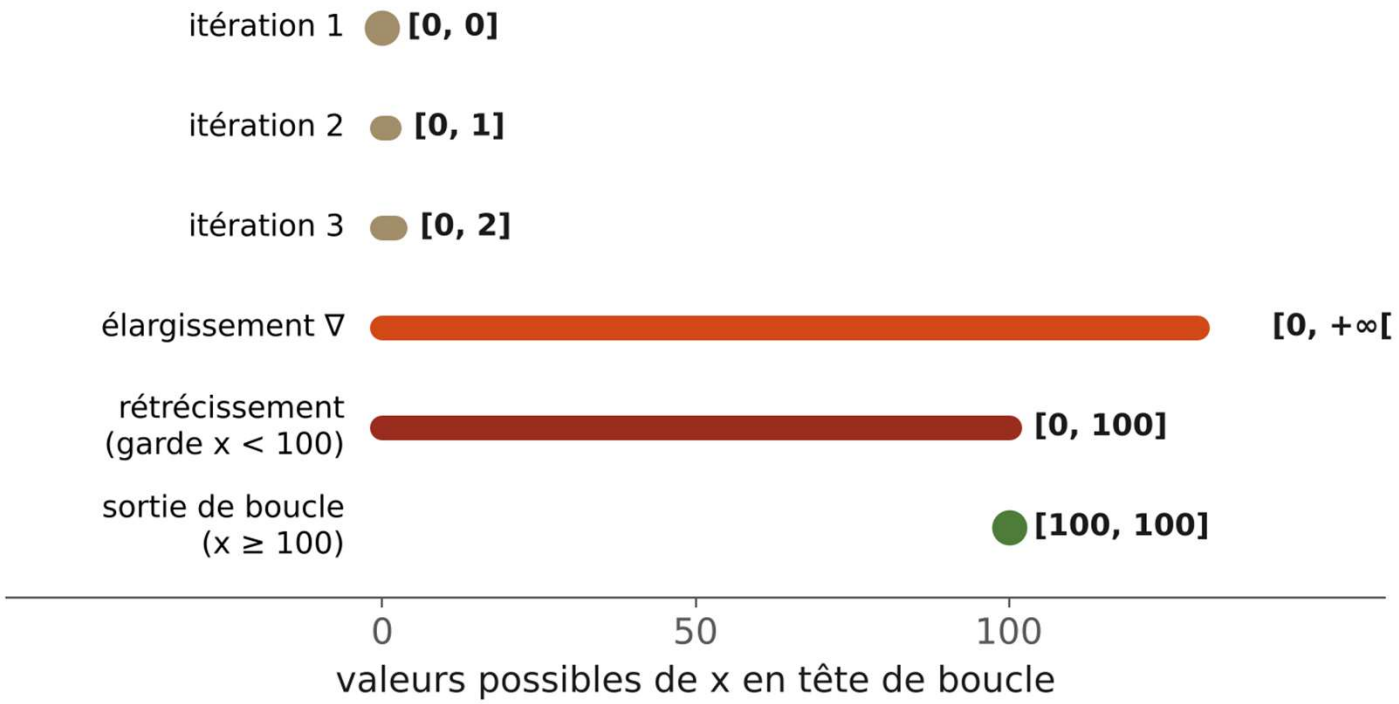
D

Interprétation abstraite (Patrick et Radhia Cousot, 1977)
Calculer automatiquement une **sur-approximation** de toutes les valeurs que peuvent prendre les variables, dans un domaine abstrait (signes, intervalles, octogones…).

- **Correcte** (sound) : si l'outil ne signale rien, l'erreur est **impossible**
- Incomplète : une alarme peut être **fausse** (la sur-approximation est trop large)
- C'est l'**encadrement** de la séance 6, appliqué à tout le programme et à toutes les entrées
- Prouve l'absence de : débordements, divisions par zéro, accès hors tableau, `null`

INTERVALLES ET ÉLARGISSEMENT

Programme : `x = 0; while (x < 100) x = x + 1;` On cherche l'ensemble des valeurs de x en tête de boucle.



Itérer naïvement prendrait 100 tours (ou une infinité). L'**élargissement** (∇) saute à [0, +∞[pour garantir la terminaison de l'analyse ; le **rétrécissement** avec la garde x < 100 retrouve [0, 100]. À la sortie : x = 100, exactement.

PROUVER L'ABSENCE DE DÉBORDEMENT

carre.c

```c
/*@ requires 0 <= n <= 1000; */
int carre(int n) {
    return n * n;  /* [0, 1e6] : sûr */
}

/*@ requires 0 <= n <= 100000; */
int carre2(int n) {
    return n * n;  /* [0, 1e10] */
}
```

- Les préconditions ACSL bornent n
- EVA calcule $n \in [0, 1\,000] \Rightarrow n \cdot n \in [0, 1\,000\,000] \subset$ int : **aucune alarme**
- Pour carre2 : $n \cdot n$ peut atteindre $10^{10} > 2^{31} - 1$: **alarme**
- Commande : `frama-c -eva` avec l'option `-main carre2`

Sortie typique de Frama-C EVA

```
[eva:alarm] carre.c:8: Warning: signed overflow. assert n * n ≤ 2147483647;
```

LIMITES, COÛT ET CAS D'USAGE

- **Limites :**
  - on prouve **le modèle** et **la spécification** : une spécification fausse donne un logiciel « prouvé » faux (c'est un problème de **validation**)
  - hypothèses sur le compilateur, le matériel, l'environnement
  - coût en expertise et en temps ; explosion des états
- **Quand les utiliser :**
  - normes de sûreté : DO-178C + DO-333 (avionique), EN 50128 (ferroviaire), ISO 26262 (automobile)
  - protocoles distribués et concurrents (TLA+)
  - noyaux, cryptographie, règles financières critiques
  - de façon **légère** partout : contrats, assertions, types

Approche pragmatique : spécifier formellement **la partie critique** (ex. frais et plafonds Mobile Money), tester le reste.

TP : VÉRIFICATION FORMELLE LÉGÈRE

TP

- **1. TLA+ (fil rouge D) :** lancer TLC sur `GAB.tla` ; introduire le bug de borne ; lire la trace ; l'écrire en test JUnit
  - ajouter une variable `retire` et l'invariant « retrait du jour ≤ 200 000 » ; observer le nombre d'états
- **2. Hoare :** prouver { n ≥ 0 } s := 0; i := 0; while (i < n) { i := i + 1; s := s + i } { s = n(n+1)/2 }
  - puis traduire l'invariant en `assert` Java et exécuter avec `-ea`
- **3. JML (fil rouge B) :** écrire les contrats de `TransfertService` : frais, plafond journalier, invariant de conservation des soldes
- **4. Option :** Frama-C EVA sur `carre.c` et sur `milieu(a, b)`

À RETENIR

À RETENIR

- Le test montre la **présence** de bugs ; une preuve montre leur **absence**, pour une propriété et un modèle
- Contrat = précondition (appelant) + postcondition (méthode) + invariants (classe)
- Hoare : **{P} S {Q}** ; l'invariant de boucle est la clé ; wp se calcule à l'envers
- Model checking : exploration exhaustive d'un modèle fini, **contre-exemple** automatique
- Interprétation abstraite : sur-approximation **correcte**, prouve l'absence de débordement
- Une spécification fausse donne un logiciel « prouvé » faux : la **validation** reste indispensable

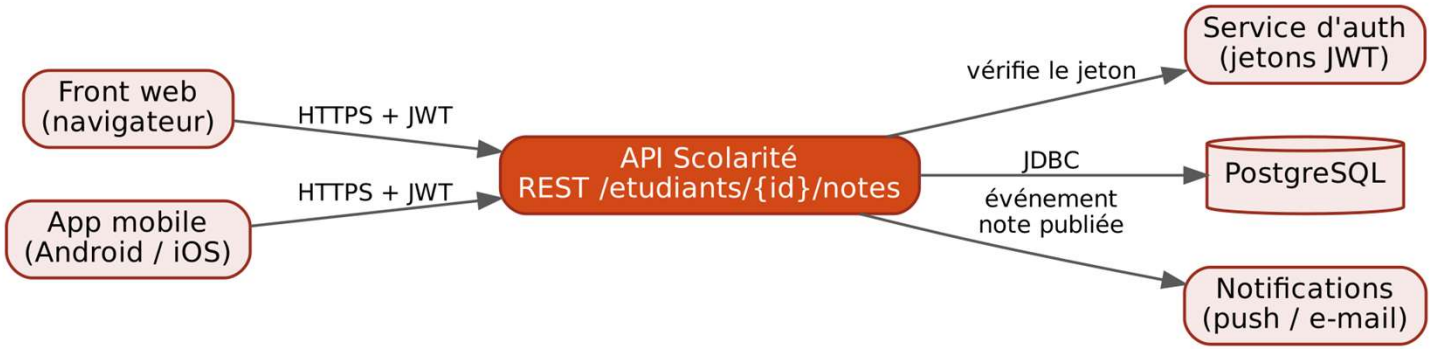
INTÉGRATION, API, BOUT EN BOUT ET PERFORMANCE

272

Séance 12 · 4 h · Objectifs : tester les interactions entre composants, les API et le système complet, puis mesurer ses performances

CHAP XII

OBJECTIFS DE LA SÉANCE



- Choisir une **stratégie d'intégration** et les doublures associées (bouchons, pilotes)
- Tester une API REST : statuts, corps, **schéma**, erreurs ; sécuriser son évolution par des **contrats** (Pact)
- Écrire des tests **de bout en bout** web et mobile maintenables
- Concevoir un **test de performance** : profil de charge, métriques, SLO
- Diagnostiquer avec le **profilage** : temps CPU, mémoire, fuites

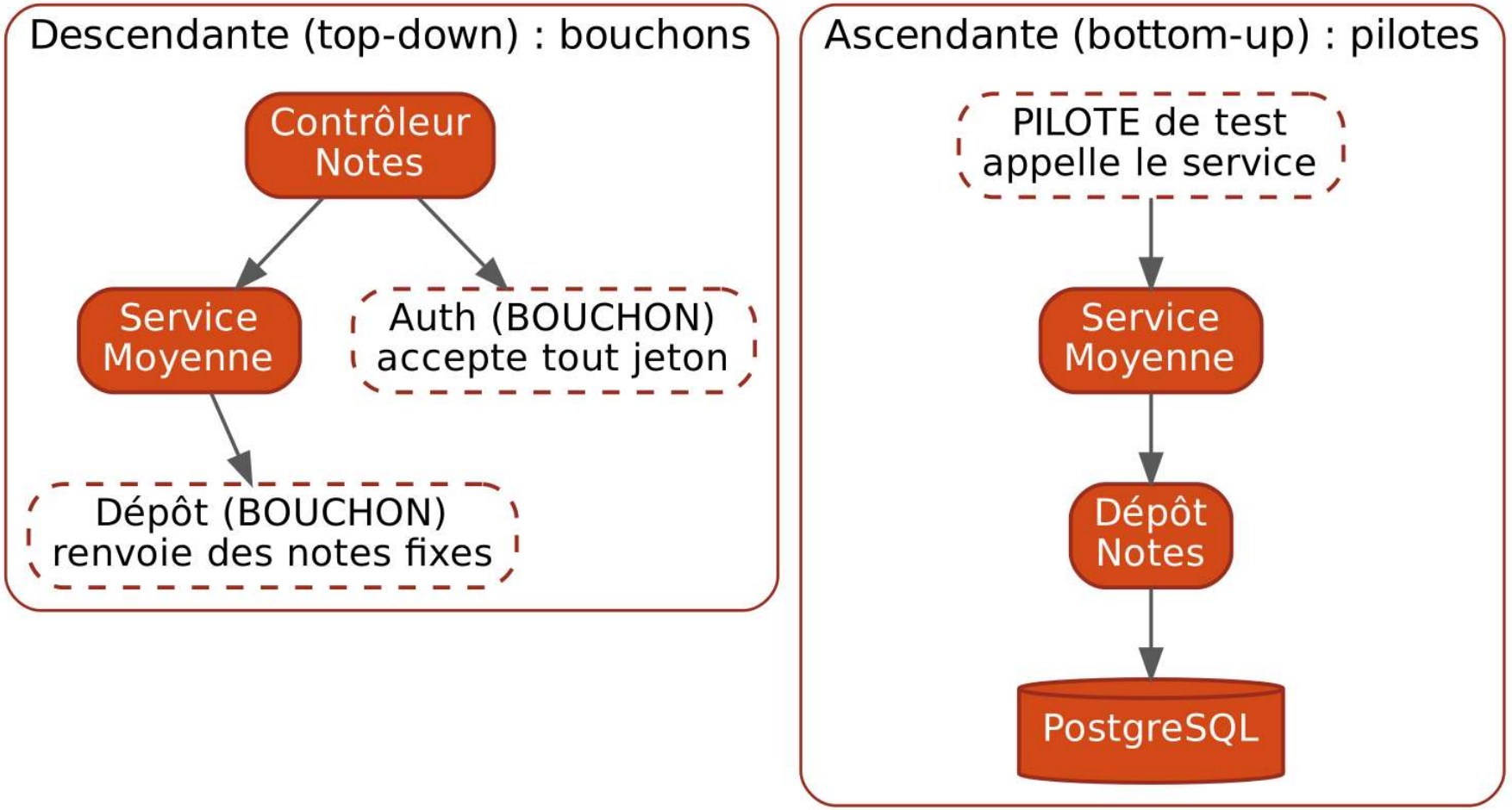
STRATÉGIES D'INTÉGRATION

Question : **dans quel ordre** assemble-t-on les composants, et que met-on à la place de ceux qui manquent ?

Stratégie	Principe	Avantage	Limite
Big bang	Tout assembler d'un coup, puis tester	Aucune doublure à écrire	Défaut difficile à localiser, détection tardive
Descendante (top-down)	Du haut (IHM, contrôleur) vers le bas ; le bas est remplacé par des **bouchons**	Squelette démontrable tôt	Beaucoup de bouchons ; couches basses testées tard
Ascendante (bottom-up)	Du bas (accès aux données) vers le haut ; des **pilotes** appellent les modules	Fondations solides testées tôt	Pas de vision utilisateur avant la fin
Sandwich (mixte)	Les deux à la fois, rencontre sur une couche cible	Parallélisable	Organisation plus lourde
Continue	Intégrer et tester à **chaque commit** (CI)	Défauts trouvés en minutes	Exige une suite automatisée rapide

En pratique
Intégration continue + tests d'intégration ciblés sur les **frontières** (base, API externes, files de messages).

BOUCHONS ET PILOTES SUR L'API SCOLARITÉ



Pointillés = doublures de test ; en couleur = composants réels intégrés

TESTER CONTRE UNE VRAIE BASE : TESTCONTAINERS

NotesRepositoryIT.java

```java
@Testcontainers
@SpringBootTest
class NotesRepositoryIT {
  @Container @ServiceConnection
  static PostgreSQLContainer<?> pg =
      new PostgreSQLContainer<>("postgres:16-alpine");

  @Autowired NotesRepository repo;

  @Test
  void moyennePondereeCalculeeEnSql() {
    repo.save(new Note("24G1234", "INF563", 14.0, 5));
    repo.save(new Note("24G1234", "INF561", 11.0, 3));
    assertThat(repo.moyennePonderee("24G1234"))
        .isCloseTo(12.875, within(1e-9));
  }
}
```

- Un **conteneur Docker PostgreSQL** jetable démarre pour la classe de test
- `@ServiceConnection` injecte l'URL et les identifiants (Spring Boot ≥ 3.1)
- Même moteur qu'en production : pas de surprise SQL due à une base en mémoire différente
- Oracle : (14×5 + 11×3) / 8 = **12,875**

QUE VÉRIFIE UN TEST D'API ?

1 Statut HTTP

200, 201, 400, 401, 403, 404, 409… Le **bon** code, pas seulement « pas 500 ».

2 Corps et schéma

Valeurs attendues **et** structure : types, champs obligatoires, formats (JSON Schema, OpenAPI).

3 En-têtes

`Content-Type`, cache, CORS, en-têtes de sécurité, `Location` après un POST.

4 Effets de bord

La note est-elle **réellement** enregistrée ? Un GET après le POST le confirme.

5 Erreurs et autorisations

Entrées invalides, jeton absent, accès aux notes d'un **autre** étudiant.

6 Contrat et temps

Conformité au contrat publié ; temps de réponse sous un seuil indicatif.

REST ASSURED : LIRE LES NOTES

Style **given / when / then** : préconditions, appel, vérifications. L'API tourne sur un port aléatoire (`@SpringBootTest(webEnvironment = RANDOM_PORT)`).

NotesApiTest.java

```
@Test
void notesDUnEtudiant() {
  given()
       .auth().oauth2(jetonEtudiant("24G1234"))
  .when()
       .get("/etudiants/{id}/notes", "24G1234")
  .then()
       .statusCode(200)
       .contentType(ContentType.JSON)
       .body("notes.size()", equalTo(2))
       .body("moyenne", closeTo(12.875, 0.0005))
       .body(matchesJsonSchemaInClasspath("notes.schema.json"));
}
```

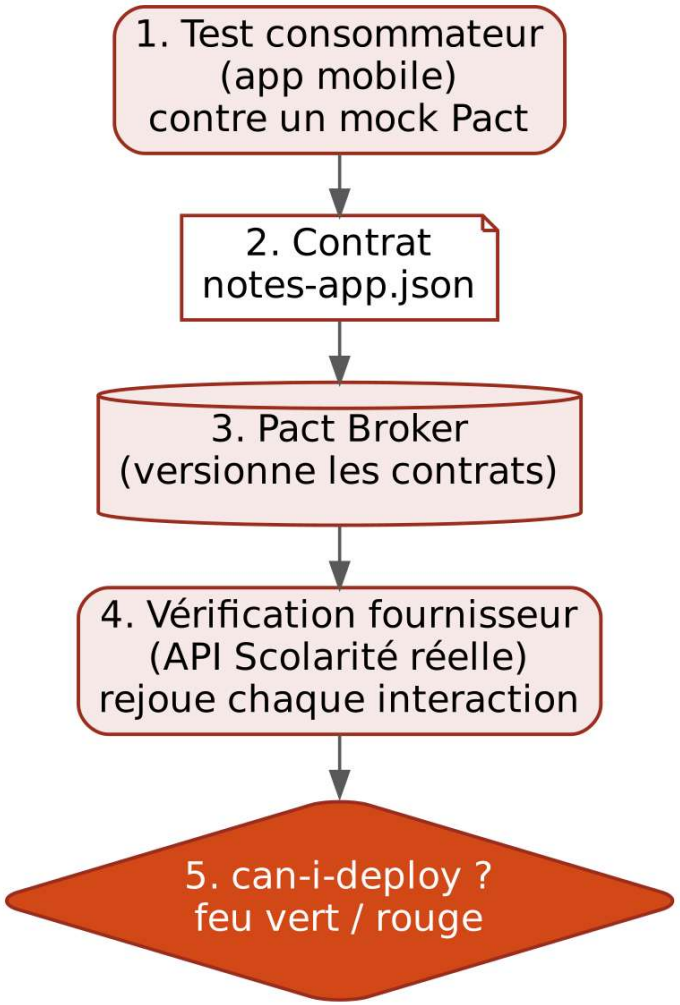
Modèle de test (séance 3)
DE = identifiant + jeton · g = API · RA = 12,875 · N = $|y - RA|$ · Tol = 0,0005 · oracle complété par le **schéma JSON**.

CLASSES D'ÉQUIVALENCE SUR L'API

```
@Test
void noteHorsBornesRefusee() {
  given()
       .auth().oauth2(jetonEnseignant())
       .contentType(ContentType.JSON)
       .body("""
         {"ue":"INF563","note":21.0,"credits":5}
         """)
  .when()
       .post("/etudiants/{id}/notes", "24G1234")
  .then()
       .statusCode(400)
       .body("erreur", containsString("note"));
}
```

Classe d'entrée (POST)	Statut
Note dans [0 ; 20], enseignant	201
Note = 20,0 (limite)	201
Note = 20,01 ou −0,5	400
Corps JSON mal formé	400
Sans jeton	401
Jeton étudiant	403
Étudiant inexistant	404
Même UE déjà notée	409

TESTS DE CONTRAT PILOTÉS PAR LE CONSOMMATEUR



- Problème : l'app mobile et l'API évoluent **séparément** ; les versions mobiles anciennes restent installées des mois
- Le **consommateur** écrit ses attentes : c'est le contrat
- Le **fournisseur** rejoue le contrat contre son code réel, à chaque build
- Le Pact Broker répond à « puis-je déployer cette version ? »
- Plus rapide et plus ciblé qu'un test de bout en bout ; ne remplace pas les tests fonctionnels du fournisseur

PACT CÔTÉ APP MOBILE (JAVASCRIPT)

notes.pact.test.js

```
const { PactV3, MatchersV3 } =
  require('@pact-foundation/pact');
const { eachLike, decimal } = MatchersV3;
const provider = new PactV3({
  consumer: 'app-mobile', provider: 'api-scolarite' });

test('affiche les notes', () => {
  provider
    .given('24G1234 a des notes')
    .uponReceiving('une demande de notes')
    .withRequest({ method: 'GET',
                   path: '/etudiants/24G1234/notes' })
    .willRespondWith({ status: 200, body: {
      moyenne: decimal(12.88),
      notes: eachLike({ ue: 'INF563', note: 14.0 }) } });
  return provider.executeTest(async (mock) => {
    const r = await new ClientNotes(mock.url)
      .notes('24G1234');
    expect(r.moyenne).toBeCloseTo(12.88, 2);
  });
});
```

`given` : état exigé du fournisseur · `eachLike`, `decimal` : on fixe le **type**, pas la valeur exacte · le test produit le contrat JSON · côté Java, le fournisseur le vérifie (`@Provider`, `@PactBroker`).

BOUT EN BOUT WEB : PLAYWRIGHT ET PAGE OBJECTS

```
public class PageNotes {
  private final Page page;
  public PageNotes(Page page) { this.page = page; }
  public PageNotes ouvrir(String id) {
    page.navigate("/etudiants/" + id + "/notes");
    return this;
  }
  public Locator moyenne() {
    return page.getByTestId("moyenne");
  }
  public Locator lignes() {
    return page.getByRole(AriaRole.ROW);
  }
}

@Test
void afficheLaMoyenne() {
  PageNotes p = new PageNotes(page).ouvrir("24G1234");
  assertThat(p.moyenne()).hasText("12,88");
  assertThat(p.lignes()).hasCount(3);
}
```

- **Page object** : les sélecteurs vivent dans une seule classe ; le test lit comme un scénario
- Sélecteurs stables : `data-testid`, rôles ARIA, jamais de XPath fragile
- Les assertions Playwright **attendent** automatiquement : pas de `sleep`
- 3 lignes = en-tête + 2 UE ; `baseURL` fixée dans le contexte

BOUT EN BOUT MOBILE

Outil	Plateforme	Langage	Particularité
Espresso	Android	Kotlin / Java	Boîte grise, dans le processus de l'app : rapide et synchronisé avec le fil UI
XCUITest	iOS	Swift	Framework natif d'Apple, intégré à Xcode
Appium	Android + iOS	Tous (WebDriver)	Boîte noire multiplateforme ; plus lent, plus fragile
Detox	React Native	JavaScript	Synchronisation automatique avec l'app RN
Maestro	Android + iOS	YAML	Scénarios déclaratifs, prise en main rapide

Spécificités à tester sur mobile

Permissions (accordées / refusées), **API natives** simulées (GPS, caméra), perte de réseau et mode hors ligne, rotation, mise en arrière-plan, tailles d'écran, versions d'OS, appareils d'entrée de gamme.

PIÈGES DES TESTS DE BOUT EN BOUT

- **Lenteur :** des secondes à des minutes par test : la suite ne peut pas tourner à chaque commit
- **Fragilité (flakiness) :** échecs aléatoires dus au temps, au réseau, aux données partagées
- **Diagnostic :** un échec E2E dit « quelque chose ne va pas », rarement **où**
- **Coût de maintenance :** chaque changement d'IHM peut casser de nombreux tests

- Peu de tests E2E : les **parcours critiques** seulement (connexion, saisie de note, consultation)
- Attentes explicites, jamais de `sleep` fixe
- Données **isolées** par test, créées via l'API
- Captures, vidéos et traces à chaque échec
- Test instable → **quarantaine** et correction, pas de relance automatique sans fin

Pyramide
Beaucoup de tests unitaires, des tests d'intégration et d'API ciblés, **quelques** tests de bout en bout.

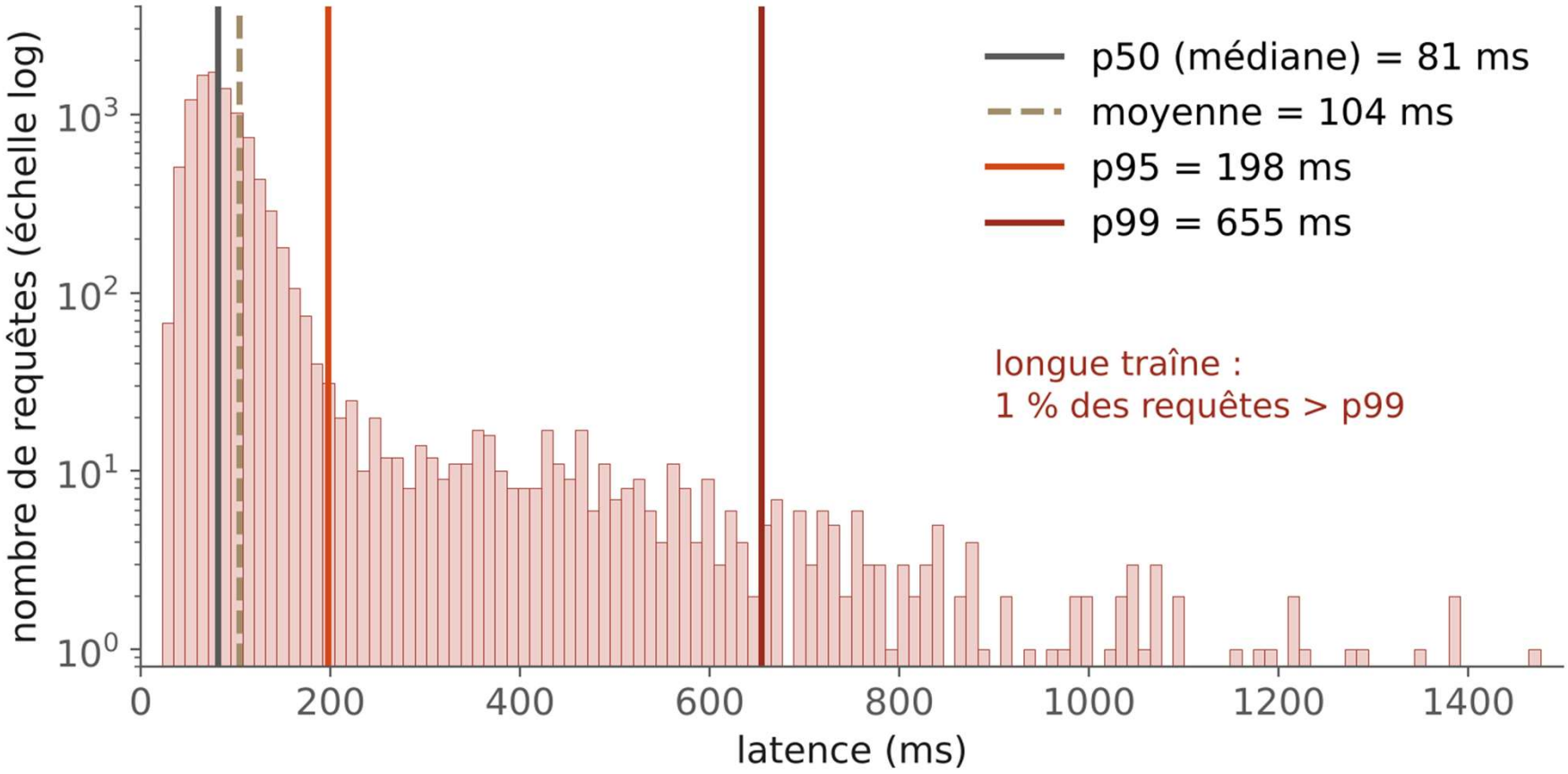
LES TYPES DE TESTS DE PERFORMANCE

Type	Profil de charge	Question posée
Charge (load)	Charge nominale attendue, en palier	Respecte-t-on les SLO en conditions normales ?
Montée en charge (scalability)	Charge croissante par paliers	Jusqu'où le système suit-il ? Où est le coude ?
Stress	Au-delà de la capacité prévue	Comment le système se dégrade-t-il ? Se rétablit-il ?
Pic (spike)	Saut brutal (ex. publication des résultats)	Absorbe-t-on l'afflux soudain ?
Endurance (soak)	Charge nominale pendant des heures	Fuites mémoire, saturation de connexions, journaux qui gonflent ?

Scénario réaliste pour le fil rouge F

Le jour de la publication des notes, des milliers d'étudiants se connectent dans la même heure : c'est un **test de pic**.

MOYENNE OU PERCENTILES ?



Latences de 10 000 requêtes simulées (distribution log-normale avec une queue lente de 5 %)

La **moyenne** cache la queue. Le **p95** dit : 95 % des requêtes sont plus rapides que cette valeur. Le p99 révèle ce que subissent les utilisateurs les plus mal servis.

MÉTRIQUES, SLI, SLO, SLA

1 Latence

Temps de réponse en **percentiles** (p50, p95, p99), mesuré côté client.

2 Débit

Requêtes (ou transactions) traitées par seconde.

3 Taux d'erreur

Part des réponses en erreur (5xx, délais dépassés). Une API rapide qui échoue n'est pas performante.

4 Saturation

CPU, mémoire, connexions de base, files d'attente : ce qui va lâcher en premier.

SLI → SLO → SLA

SLI = l'indicateur mesuré (p95 de latence) · **SLO** = l'objectif interne (p95 < 300 ms) · **SLA** = l'engagement contractuel, avec pénalités, en général moins strict que le SLO.

LOI DE LITTLE

D

$L = \lambda \times W$

Nombre moyen de requêtes **en cours** dans le système = débit d'arrivée × temps moyen passé dans le système. Valable pour tout système stable, quelle que soit la distribution.

- API Scolarité : $\lambda = 200$ requêtes/s, $W = 0{,}25$ s ⇒ **$L = 50$** requêtes en cours en moyenne
- Conséquence : un pool de 20 connexions à la base est sous-dimensionné si chaque requête garde sa connexion pendant tout son traitement
- Si W double (base lente), L double : les files se remplissent et la latence s'emballe
- Sert à **vérifier la cohérence** des résultats d'un test de charge (débit mesuré × latence mesurée ≈ concurrence observée)

SCRIPT K6 AVEC SEUILS

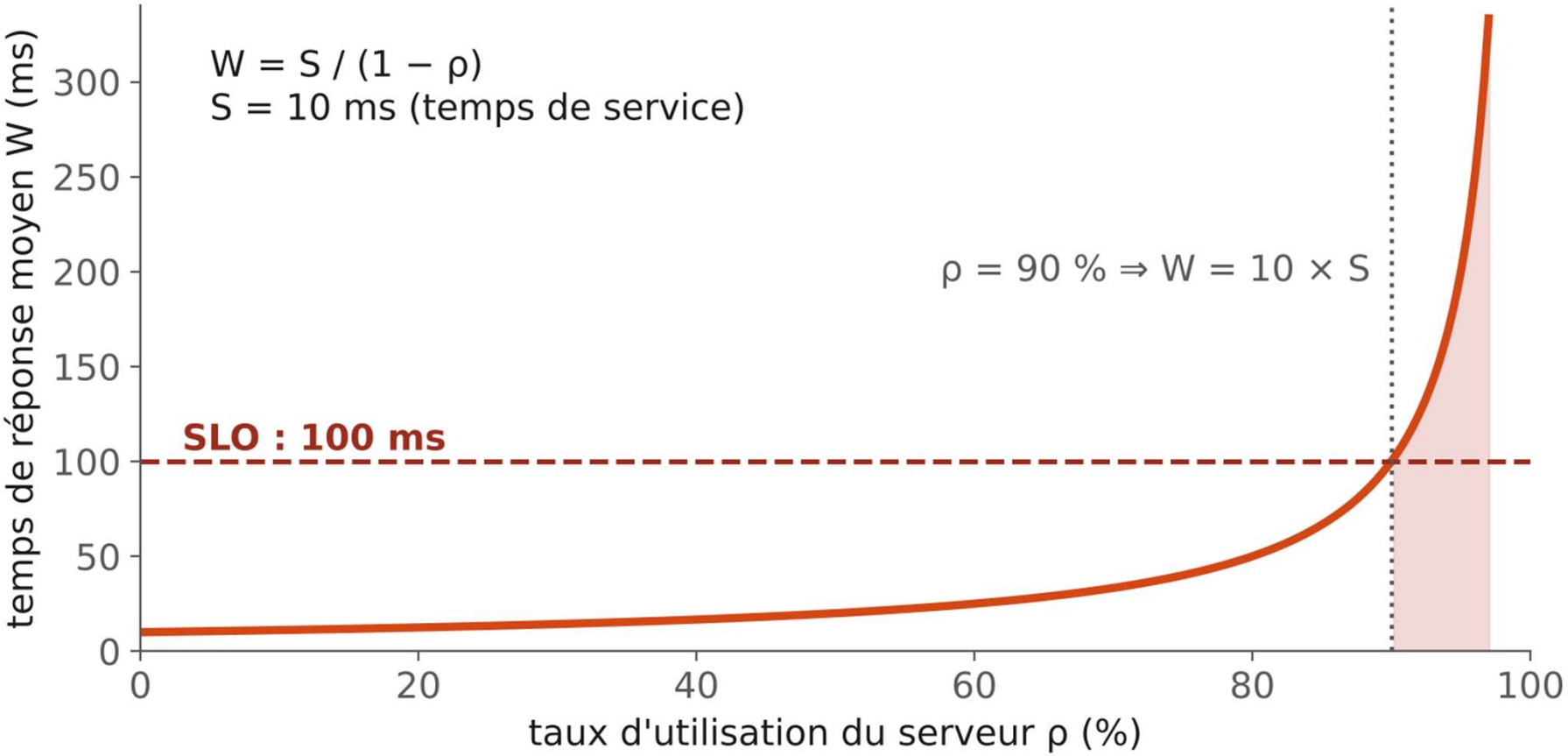
notes_charge.js

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 100 },  // montée
    { duration: '5m', target: 100 },  // palier
    { duration: '1m', target: 0 },    // descente
  ],
  thresholds: {
    http_req_duration: ['p(95)<300', 'p(99)<800'],
    http_req_failed: ['rate<0.01'],
  },
};
export default function () {
  const n = 1000 + Math.floor(Math.random() * 500);
  const url = `${__ENV.BASE_URL}/etudiants/24G${n}/notes`;
  const res = http.get(url, {
    headers: { Authorization: `Bearer ${__ENV.TOKEN}` },
  });
  check(res, { 'statut 200': (r) => r.status === 200 });
  sleep(1);                        // réflexion
}
```

- 100 utilisateurs virtuels (VU)
- `thresholds` = **oracle** : k6 renvoie un code d'erreur si un seuil est violé, ce qui fait échouer le pipeline
- Identifiants variés : évite de ne mesurer que le cache
- `sleep(1)` simule le temps de réflexion

POURQUOI LA LATENCE EXPLOSE PRÈS DE LA SATURATION



Modèle illustratif de file d'attente M/M/1 (un serveur, arrivées et services aléatoires), et non une mesure réelle

À 50 % d'utilisation, W = 2 × S ; à 90 %, W = 10 × S. Le « coude » de la courbe fixe la capacité utile : on dimensionne pour rester **bien en dessous**.

LE MODÈLE DE TEST APPLIQUÉ À LA PERFORMANCE

Élément (séance 3)	Test de charge de l'API Scolarité
Données d'entrée DE	Profil de charge : 100 VU, montée 2 min, palier 5 min, identifiants variés
Système sous test g	API + base + réseau, dans un environnement **proche de la production**
Mesure d'erreur N	p95 et p99 de la latence ; taux d'erreur
Résultat attendu / Tol	SLO : p95 < 300 ms, p99 < 800 ms, erreurs < 1 %
Oracle O	N ≤ Tol pour chaque métrique (seuils k6)
Verdict	PASS / FAIL ; INCONCLUSIF si le générateur de charge est lui-même saturé

Variabilité

Une mesure de performance est **aléatoire** : répéter les exécutions, comparer à une référence (baseline) et surveiller les régressions, pas seulement un chiffre isolé.

OUTILS DE TEST DE CHARGE

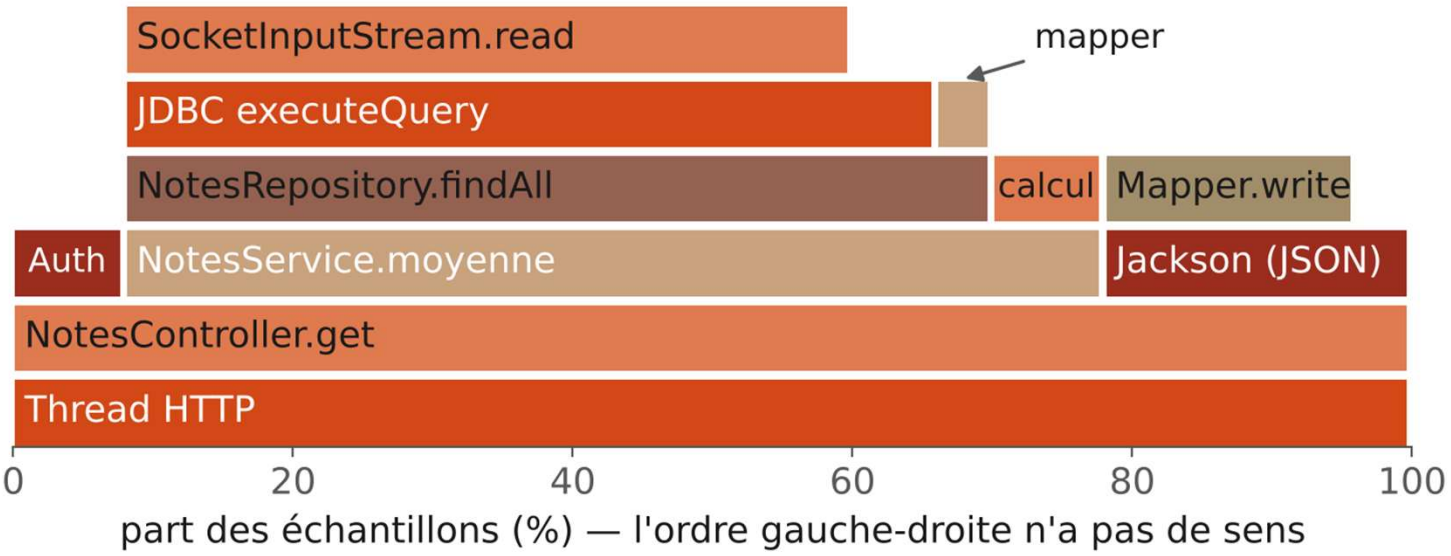
Outil	Scénarios écrits en	Points forts	Limites
k6	JavaScript	Léger, seuils natifs, adapté à la CI	Pas un navigateur complet
Apache JMeter	Interface graphique, XML	Très répandu, nombreux protocoles	Plans de test lourds à versionner, gourmand
Gatling	Java, Kotlin, Scala	Très performant, rapports HTML détaillés	Courbe d'apprentissage
Locust	Python	Simple, distribué	Moins de débit par machine

Le choix compte moins que la **méthode** : profil réaliste, environnement représentatif, injecteur surveillé, seuils explicites, résultats comparés dans le temps.

ANALYSE DYNAMIQUE : OUTILS DE PROFILAGE ACTUALISÉS

Langage	Temps CPU	Mémoire
Java	Java Flight Recorder + JDK Mission Control, async-profiler, VisualVM	JFR, heap dump + Eclipse MAT, VisualVM
Python	cProfile, py-spy, Scalene	tracemalloc, memray, Scalene
JavaScript	Chrome DevTools (Performance), profileur CPU de Node, Clinic.js	DevTools (Memory), heap snapshots
PHP	Xdebug, Blackfire, SPX	Blackfire, Xdebug
C / C++	perf, Callgrind (Valgrind), gprof	Valgrind (Memcheck, Massif), heaptrack, AddressSanitizer
Android	Android Studio Profiler, Perfetto	Android Studio Memory Profiler, LeakCanary

LIRE UN FLAME GRAPH



Exemple schématique d'une requête GET /notes

- Chaque barre = une fonction ; au-dessus = les fonctions **qu'elle appelle** ; **largeur** = part des échantillons, donc coût
- Chercher les **plateaux larges** en haut : c'est là que le temps est réellement dépensé
- Ici, 52 % du temps est passé à **attendre la base** : optimiser le calcul de moyenne ne servirait à rien

FUITE MÉMOIRE : UN CACHE QUI NE SE VIDE JAMAIS

```
private static final Map<String, List<Double>> CACHE =
    new HashMap<>();

static List<Double> notes(String id, long req) {
  String cle = id + "#" + req;   // clé unique par appel !
  return CACHE.computeIfAbsent(cle,
      k -> chargerDepuisBase(id));
}
```

Requêtes	Tas après GC
50 000	210 Mo
100 000	420 Mo
150 000	628 Mo
200 000	837 Mo

Signature d'une fuite

La mémoire **après ramasse-miettes** croît linéairement avec le nombre de requêtes. Détection : test d'endurance + heap dump + arbre des dominateurs (Eclipse MAT) → `HashMap` statique. Correction : clé correcte et cache borné avec expiration (ex. Caffeine).

PROFILER EN PRATIQUE (JVM)

```
# Enregistrer 60 s avec Java Flight Recorder au démarrage
java -XX:StartFlightRecording=duration=60s,filename=api.jfr -jar api.jar

# ... ou sur un processus déjà lancé
jcmd <pid> JFR.start duration=60s filename=api.jfr
jfr print --events jdk.ExecutionSample api.jfr | head

# Flame graph CPU avec async-profiler (30 s)
asprof -d 30 -f flame.html <pid>

# Vidage du tas pour analyser une fuite (Eclipse MAT)
jcmd <pid> GC.heap_dump /tmp/api.hprof
```

Démarche

1. reproduire sous charge (k6) · 2. enregistrer (JFR, async-profiler) · 3. identifier le plateau ou la croissance · 4. corriger · 5. **re-mesurer** et ajouter un test de non-régression de performance.

À RETENIR

À RETENIR

1 Intégration

Tester les **frontières** réelles : Testcontainers pour la base, contrats pour les API consommées par d'autres équipes.

2 API

Statut, corps, schéma, en-têtes, effets de bord ; classes d'équivalence et autorisations testées explicitement.

3 Bout en bout

Peu de tests, parcours critiques, page objects, attentes explicites, instabilité traitée.

4 Performance

Profil réaliste, percentiles, SLO comme oracle, loi de Little pour la cohérence, profilage avant d'optimiser.

TP DE LA SÉANCE : L'API SCOLARITÉ SOUS CONTRÔLE

TP

- **Intégration** : un test Testcontainers du dépôt de notes (moyenne pondérée calculée en SQL)
- **API** : avec REST Assured, au moins une classe valide et quatre invalides (400, 401, 403, 404), plus un schéma JSON
- **Contrat** : un test Pact consommateur pour l'app mobile et la vérification côté fournisseur
- **E2E** : un parcours Playwright « consulter ses notes » avec un page object
- **Charge** : un script k6 avec seuils p95 < 300 ms et erreurs < 1 % ; rapport avec la vérification par la loi de Little
- **Profilage** : un enregistrement JFR sous charge et une analyse en 5 lignes du point chaud principal

Livrable : dépôt Git + court rapport ; le pipeline doit lancer les tests d'intégration et d'API

SÉCURITÉ, DEVSECOPS, IA, ÉTHIQUE ET SYNTHÈSE

299

Séance 13 · 4 h · Objectifs : tester la sécurité, l'automatiser dans le pipeline, tester avec et pour l'IA, assumer sa responsabilité

CHAP XIII

OBJECTIFS DE LA SÉANCE

- Situer le **test de sécurité** dans la V&V et utiliser les référentiels OWASP
- Modéliser les menaces avec **STRIDE**
- Choisir entre **SAST, DAST, IAST, SCA** et le **fuzzing**
- Construire un pipeline **DevSecOps** avec des quality gates
- Connaître le **cadre légal camerounais** qui s'applique aux données de test
- Utiliser l'**IA** pour tester, et tester des systèmes d'IA
- Mesurer la **responsabilité éthique** du testeur

Fils rouges du jour
B Mobile Money pour les menaces (argent, PIN, transferts) · **C Formulaire** pour les injections et le fuzzing · **F API Scolarité** pour le pipeline.

LA SÉCURITÉ DANS LA V&V

- **Vérification :** les **exigences de sécurité** sont-elles implémentées ? (mots de passe hachés, jetons expirés, contrôle d'accès sur chaque route)
- **Validation :** le système **résiste-t-il à un attaquant réel**, qui n'utilise pas le logiciel comme prévu ?
- **Différence clé :** un test fonctionnel vérifie ce que le système **doit** faire ; un test de sécurité vérifie ce qu'il **ne doit pas** permettre

ISO/IEC 25010:2023	Sécurité : sous-caractéristiques
Confidentialité	données accessibles aux seuls autorisés
Intégrité	pas de modification non autorisée
Non-répudiation	une action ne peut être niée
Responsabilité	actions traçables à leur auteur
Authenticité	identité prouvée
Résistance	fonctionner sous attaque (ajout 2023)

Approche **shift-left** : la sécurité se teste dès la conception (modélisation des menaces), pas seulement par un audit avant la mise en production.

OWASP TOP 10:2025

Rang	Catégorie	Exemple sur nos fils rouges
A01	Broken Access Control	Un étudiant lit les notes d'un autre (`/etudiants/24G9999/notes`) — inclut désormais le SSRF
A02	Security Misconfiguration	Console d'administration exposée, traces d'erreur détaillées en production
A03	Software Supply Chain Failures	Dépendance Maven compromise ou obsolète dans l'API
A04	Cryptographic Failures	PIN Mobile Money stocké en clair ou haché sans sel
A05	Injection	Matricule injecté dans une requête SQL du formulaire C
A06	Insecure Design	Aucun plafond de transfert prévu dans la conception
A07	Authentication Failures	Pas de blocage après N essais de PIN (fil rouge D)
A08	Software or Data Integrity Failures	Mise à jour de l'app sans vérification de signature
A09	Security Logging & Alerting Failures	Transferts frauduleux non journalisés, aucune alerte
A10	Mishandling of Exceptional Conditions	Exception non gérée qui laisse un transfert à moitié exécuté

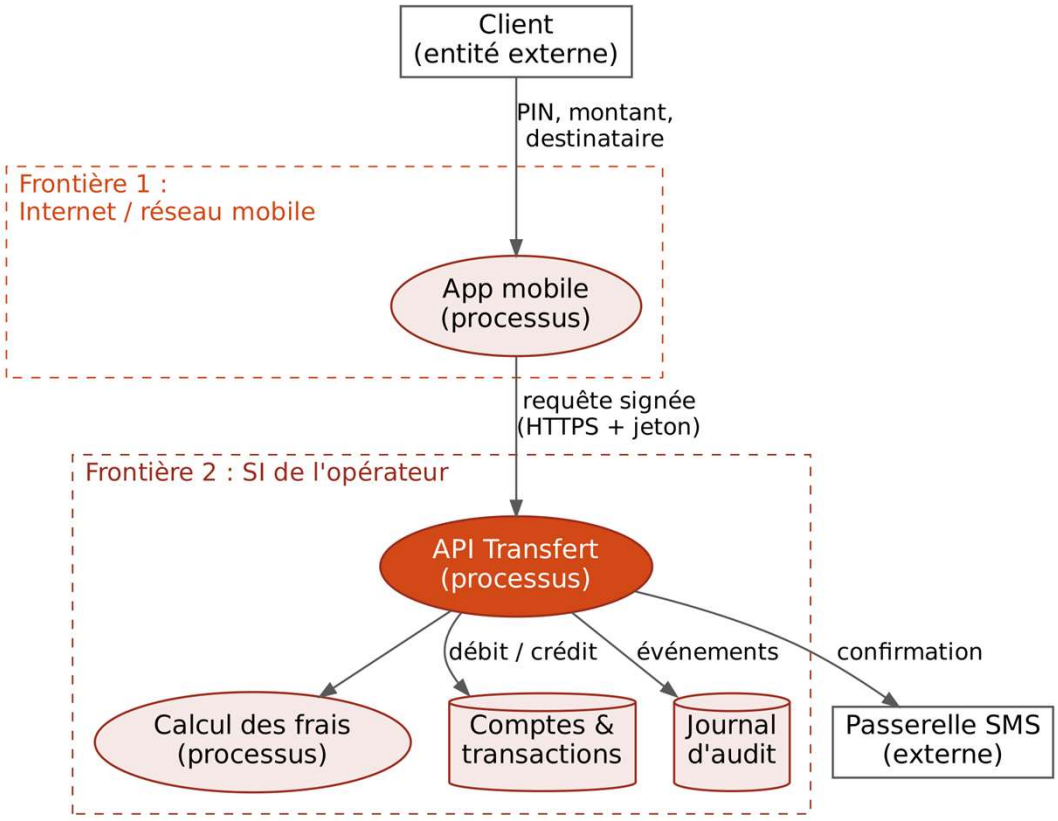
OWASP ASVS : DES EXIGENCES TESTABLES

- L'**ASVS** (Application Security Verification Standard), version **5.0** publiée en mai 2025, liste des exigences de sécurité **vérifiables**
- Le Top 10 sensibilise ; l'ASVS sert de **référentiel de test** et de critères d'acceptation
- Chaque exigence peut devenir un **cas de test** traçable (matrice de traçabilité, séance 10)

Niveau	Usage
L1	Socle minimal pour toute application
L2	Applications métier manipulant des données sensibles : **API Scolarité**
L3	Haute assurance (finance, santé, infrastructures critiques) : **Mobile Money**

Exemple d'exigence transformée en test
« Les jetons de session expirent après une durée d'inactivité définie » → test API : un jeton inactif depuis 31 min reçoit **401**.

MODÉLISER LES MENACES : FLUX DE DONNÉES



DFD du transfert Mobile Money

- **Rectangles** : entités externes ; **ellipses** : processus ; **cylindres** : stockages
- **Pointillés** : frontières de confiance
- Les menaces se concentrent là où un flux **franchit une frontière** : chaque flèche qui traverse un pointillé mérite une analyse STRIDE

STRIDE APPLIQUÉ AU MOBILE MONEY

Menace	Propriété	Scénario	Test qui la vérifie
Spoofing (usurpation)	Authenticité	Rejouer le jeton d'un client depuis un autre appareil	Jeton lié à l'appareil : rejeu → 401
Tampering (altération)	Intégrité	Modifier le montant dans la requête après signature	Montant modifié → signature invalide → rejet
Repudiation	Non-répudiation	Le client nie avoir fait un transfert	Chaque transfert crée une entrée d'audit horodatée
Information disclosure	Confidentialité	Lire le solde d'un autre compte (IDOR)	`GET /comptes/{autre}` → 403
Denial of service	Disponibilité	Rafale de demandes de transfert	Plus de N requêtes/min → 429
Elevation of privilege	Autorisation	Un agent appelle une route d'administration	Rôle agent sur `/admin` → 403

SAST, DAST, IAST, SCA

	Ce qui est analysé	Forces / limites	Outils
SAST	Code source, sans exécution (statique)	Tôt, localise la ligne ; faux positifs, ne voit pas la configuration	Semgrep, SonarQube, SpotBugs + Find Security Bugs, CodeQL
DAST	Application en marche, de l'extérieur (boîte noire)	Voit la configuration réelle ; tard, ne localise pas la ligne	OWASP ZAP, Burp Suite
IAST	Application instrumentée pendant les tests	Précis, peu de faux positifs ; dépend de la couverture des tests	Contrast Security, agents des éditeurs
SCA	Dépendances et leurs vulnérabilités connues (CVE)	Indispensable (A03:2025) ; ne voit pas votre propre code	OWASP Dependency-Check, Trivy, Dependabot, Snyk
Secrets	Clés et mots de passe commis dans Git	Simple et très rentable	Gitleaks, GitHub secret scanning

Ces outils sont **complémentaires** : SAST et SCA à chaque commit, DAST sur l'environnement de recette, test d'intrusion humain avant les jalons majeurs.

INJECTION SQL DANS LE FORMULAIRE

```
// VULNÉRABLE : concaténation
String sql =
  "SELECT 1 FROM etudiant WHERE matricule='"
  + matricule + "'";
st.executeQuery(sql);

// matricule = ' OR '1'='1
// → matricule='' OR '1'='1'
// → toujours vrai !
```

```
// CORRIGÉ : requête paramétrée
String sql =
  "SELECT 1 FROM etudiant WHERE matricule=?";
PreparedStatement ps =
    cnx.prepareStatement(sql);
ps.setString(1, matricule);
ps.executeQuery();

// la saisie reste une DONNÉE,
// jamais du code SQL
```

Défense en profondeur

Requêtes paramétrées (correctif réel) + validation du format du matricule `^[0-9]{2}[A-Z][0-9]{4}$` (réduit la surface) + compte de base de données aux droits minimaux.

UN TEST QUI DÉTECTE L'INJECTION ET LE XSS

```java
@ParameterizedTest
@ValueSource(strings = {
    "' OR '1'='1",
    "24G1234'; DROP TABLE etudiant; --",
    "<script>alert(1)</script>",
    "\"><img src=x onerror=alert(1)>" })
void saisieMalveillanteSansEffet(String charge) throws Exception {
  assertFalse(dao.existe(charge));       // aucune ligne trouvée
  assertEquals(3, compterEtudiants());  // table intacte
  String html = Html.echapper(charge);
  assertFalse(html.contains("<") || html.contains("\""));
}
```

Classes d'entrée **malveillantes** (séance 4). Avec `existeVulnerable`, la première charge renvoie **vrai** : le test échoue et révèle la faille. `Html.echapper` transforme `<` en `<` (vérifié en exécutant les charges XSS).

LE FUZZING : CHERCHER CE QUI FAIT PLANTER

Fuzzing : générer automatiquement un très grand nombre d'entrées (aléatoires ou mutées) et observer le programme. L'oracle est **implicite** (séance 3) : plantage, exception non gérée, délai, fuite.



Corpus	Mutation	Exécution	Oracle implicite	Minimisation
Quelques entrées valides de départ (`+237699999999`, `2003-05-14`…)	Inverser des bits, tronquer, répéter, insérer des caractères spéciaux	Des milliers d'exécutions par seconde, guidées par la **couverture** de code	Exception non gérée, crash, dépassement de temps ou de mémoire	Réduire l'entrée fautive au plus petit cas, l'ajouter aux tests

En pratique
On lance le fuzzer en continu (CI nocturne) et chaque plantage trouvé devient un **test de non-régression**.

FUZZER LE FORMULAIRE

Jazzer (JUnit 5)

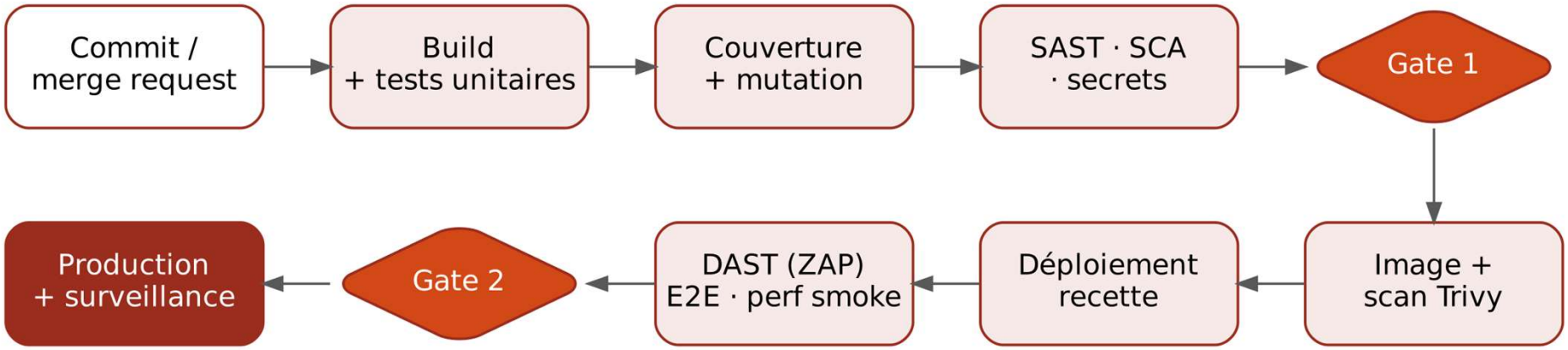
```
class ValidateurFuzzTest {
  @FuzzTest
  void telephone(FuzzedDataProvider data) {
    String s = data.consumeRemainingAsString();
    Validateur.telephoneValide(s);
    // oracle implicite : jamais d'exception,
    // seulement vrai ou faux
  }
}

// Le validateur fautif :
String n = tel.substring(4); // avant de
return tel.startsWith("+237") // vérifier !
    && n.length() == 9 && ...
```

Entrée trouvée	Exception
"175"	StringIndexOutOf-BoundsException (téléphone)
"2+7+6416"	DateTimeParse (date de naissance)

Résultats **réels** d'un mini-fuzzer aléatoire (100 000 entrées, graine 42) lancé sur notre validateur : deux exceptions non gérées, soit une **erreur 500** côté API et une faille A10:2025. Jazzer (Java) et AFL++ / libFuzzer (C, C++) font la même chose, guidés par la couverture.

PIPELINE DEVSECOPS



Chaque gate bloque la suite du pipeline si un critère n'est pas satisfait

Exemples de critères de gate

Gate 1 : tests verts, couverture des lignes ≥ 80 %, score de mutation ≥ 70 %, aucune vulnérabilité SAST critique, aucune dépendance avec un CVSS ≥ 7. Gate 2 : aucune alerte ZAP haute, parcours E2E critiques verts, p95 sous le SLO.

LE PIPELINE EN GITHUB ACTIONS

.github/workflows/ci.yml

```yaml
name: ci-devsecops
on: [push, pull_request]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '21', cache: maven }
      - name: Build, tests, couverture (JaCoCo)
        run: mvn -B verify
      - name: Mutation (PIT) - seuil dans le pom
        run: mvn -B org.pitest:pitest-maven:mutationCoverage
      - name: SAST (Semgrep)
        run: pipx run semgrep scan --config p/java --error
      - name: SCA - dependances vulnerables
        run: mvn -B org.owasp:dependency-check-maven:check -DfailBuildOnCVSS=7
  dast:
    needs: build-test
    runs-on: ubuntu-latest
    steps:
      - name: Scan ZAP de l'API en recette
        uses: zaproxy/action-api-scan@v0.9.0
        with: { target: 'https://recette.example.cm/v3/api-docs' }
```

OUTILS D'INTÉGRATION CONTINUE

Outil	Modèle	Remarques
GitHub Actions	Cloud, YAML dans le dépôt	Très large catalogue d'actions ; gratuit pour les dépôts publics
GitLab CI/CD	Intégré à GitLab, `.gitlab-ci.yml`	Auto-hébergeable : utile si les données doivent rester sur site
Jenkins	Serveur auto-hébergé, `Jenkinsfile`	Très flexible, nombreux plugins ; maintenance à la charge de l'équipe
Azure DevOps, CircleCI	Cloud	Fréquents en entreprise
Travis CI	Cloud	Historiquement populaire pour l'open source, aujourd'hui en net déclin

Le choix dépend de l'hébergement du code et des contraintes de **souveraineté des données** (voir le cadre légal, slide suivante).

CADRE LÉGAL CAMEROUNAIS

1 Loi n°2010/012 du 21 déc. 2010

Cybersécurité et cybercriminalité : définit les infractions (accès frauduleux, atteinte aux systèmes, aux données), encadre la sécurité des réseaux et systèmes d'information, prévoit des audits de sécurité.

2 Loi n°2024/017 du 23 déc. 2024

Protection des données à caractère personnel, **en vigueur depuis le 23 juin 2026** : consentement, minimisation, sécurité, droits d'accès, de rectification, d'effacement et d'opposition, transferts encadrés, sanctions.

3 ANTIC

Agence nationale des technologies de l'information et de la communication : régulation des activités de sécurité électronique pour le compte de l'État.

4 Convention de Malabo (UA)

Convention de l'Union africaine sur la cybersécurité et la protection des données (adoptée en 2014, en vigueur depuis juin 2023) : cadre continental.

Conséquences pour le testeur

Pas de **données réelles** en recette sans base légale : données synthétiques ou anonymisées. Le droit à l'effacement et la journalisation des accès deviennent des **exigences à tester**.

L'IA POUR GÉNÉRER DES TESTS

- **Atouts :** squelettes de tests, données variées, cas limites oubliés, montée en compétence rapide sur un framework
- **Risque 1 : oracle faux :** l'IA lit le **code** et en déduit le résultat attendu : elle fige les bugs existants
- **Risque 2 : tests tautologiques :** l'attendu est recalculé avec la même formule que le code testé
- **Risque 3 : couverture sans vérification :** tests qui exécutent tout mais n'assertent presque rien

- **Garde-fou 1 :** oracle issu de la **spécification** (tables de décision, contrats), jamais du code
- **Garde-fou 2 :** **revue humaine** de chaque assertion
- **Garde-fou 3 :** **test de mutation** : un test qui ne tue aucun mutant ne vérifie rien
- **Garde-fou 4 :** le test doit **échouer** sur une version volontairement fausse

UN TEST GÉNÉRÉ QUI FIGE UN BUG

```java
// Implémentation : bug à 50 000
// (< au lieu de <=)
BigDecimal seuil = new BigDecimal("50000");
if (montant.compareTo(seuil) < 0) {
  return pourcent(montant, "0.01");
}
return plafonner(pourcent(montant, "0.008"));

// Test proposé par l'assistant,
// déduit du CODE :
@Test
void fraisA50000() {
  BigDecimal m = new BigDecimal("50000");
  assertThat(service.frais(m))
      .isEqualByComparingTo("400"); // faux !
}
```

- La spécification (fil rouge B) : 5 001 à 50 000 FCFA → **1 %**, soit **500**
- Le test passe, la couverture est bonne, et il **tue** les mutants qui touchent ce calcul
- Seul un oracle **indépendant du code** (table de décision, valeurs limites) révèle le bug

TESTER UN SYSTÈME À BASE DE ML OU DE LLM

Il n'existe souvent **pas de résultat attendu exact** : on retrouve le problème de l'oracle dans sa forme la plus dure.

Technique	Principe	Exemple
Relations métamorphiques	Une transformation de l'entrée a un effet prévisible sur la sortie	Reformuler une question ne doit pas changer la réponse factuelle
Jeu d'évaluation (benchmark)	Cas annotés + métrique + seuil (N et Tol)	Exactitude ≥ 90 % sur 500 questions de scolarité annotées
LLM juge (LLM-as-judge)	Un modèle évalue les réponses selon une grille	Utile, mais lui-même à valider contre un échantillon jugé par des humains
Robustesse et sécurité	Entrées perturbées ou hostiles	Fautes de frappe, injection de consignes (prompt injection)
Surveillance de la dérive	Les données réelles évoluent après le déploiement	Suivi en production de la distribution des entrées et des scores

THERAC-25 (1985 – 1987)

CAS RÉEL

Contexte
Appareil de radiothérapie (AECL, Canada). Les verrous de sécurité matériels des modèles précédents ont été remplacés par des **contrôles logiciels**.

Défaillance
Six accidents de surdosage massif aux États-Unis et au Canada entre 1985 et 1987 ; plusieurs patients sont morts.

Cause technique
Condition de concurrence : une saisie rapide de l'opérateur laissait l'appareil dans un état incohérent. Code réutilisé, peu de tests d'intégration, messages obscurs (« MALFUNCTION 54 »).

LEÇON V&V

Le logiciel ne doit pas être l'**unique barrière** de sécurité. Les signalements des utilisateurs sont des **données de test**. Le testeur doit pouvoir exiger une enquête.

BOEING 737 MAX ET LE MCAS (2018 – 2019)

CAS RÉEL

Contexte
Le MCAS, logiciel de commandes de vol, abaisse le nez de l'avion quand l'angle d'attaque mesuré est élevé.

Défaillance
Accidents de Lion Air 610 (29 octobre 2018) et Ethiopian Airlines 302 (10 mars 2019) : **346 morts**. Flotte mondiale clouée au sol de mars 2019 à novembre 2020.

Cause technique
MCAS alimenté par **un seul** capteur d'angle d'attaque, capable de se réactiver à répétition ; fonction peu documentée pour les pilotes ; hypothèses sur le temps de réaction des équipages non validées.

LEÇON V&V

Une **hypothèse non testée** sur le comportement humain est une exigence non vérifiée. Analyser les modes de défaillance d'un capteur fait partie de la V&V.

HORIZON ET LA POSTE BRITANNIQUE (1999 – 2024)

CAS RÉEL

Contexte
Logiciel comptable Horizon (Fujitsu) déployé dans les bureaux de poste du Royaume-Uni à partir de 1999.

Défaillance
Des écarts de caisse inexpliqués ont conduit à poursuivre des centaines de gérants de bureaux de poste pour vol ou fraude ; **plus de 900 condamnations** entre 1999 et 2015, des faillites, des suicides.

Cause technique
Bugs, erreurs et défauts d'Horizon reconnus par la Haute Cour en 2019 (Bates c. Post Office) ; la fiabilité du logiciel avait été présumée devant les tribunaux ; des informations sur les défauts n'avaient pas été divulguées.

LEÇON V&V

Les données produites par un logiciel ne sont pas une **preuve** en soi. Cacher des défauts connus a des conséquences humaines. Une loi de 2024 a annulé les condamnations.

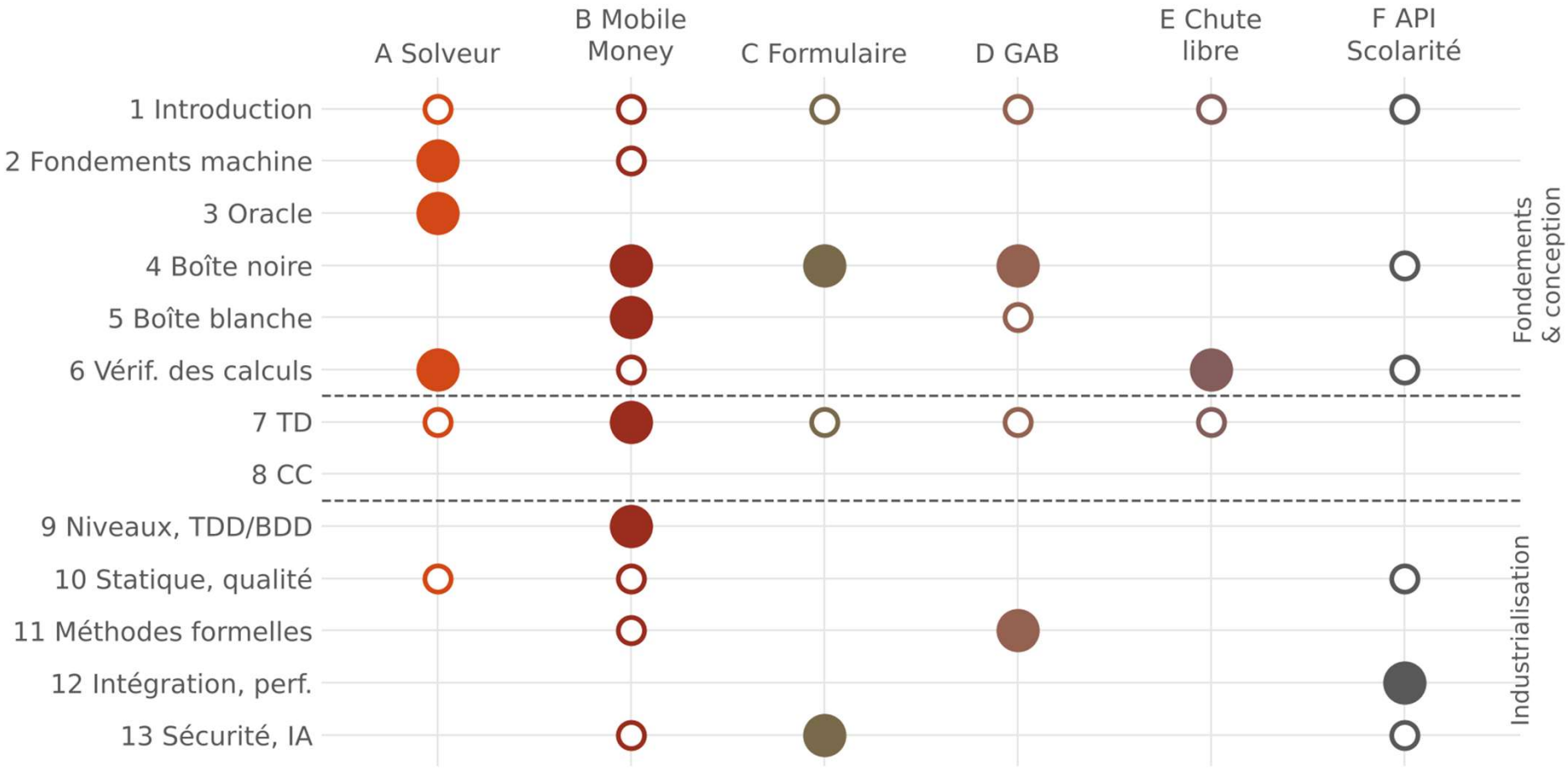
LA RESPONSABILITÉ DU TESTEUR

- **Code ACM (2018) :** 2.5 : fournir des évaluations **complètes et rigoureuses** des systèmes et de leurs impacts, y compris l'analyse des risques
- **Code de l'ingénierie logicielle IEEE-CS / ACM :** 3.10 : assurer des tests, un débogage et une revue **adéquats** du logiciel ; principe 1 : agir dans l'intérêt du public

- Ne jamais **maquiller** un résultat ni baisser un seuil pour « faire passer » une livraison
- Documenter et **signaler** un risque, par écrit, jusqu'au niveau de décision adéquat
- Dire ce qui **n'a pas** été testé : le rapport de test doit montrer ses limites
- Protéger les données personnelles utilisées pour tester

Tester, c'est fournir une **information honnête sur le risque** à ceux qui décident. La décision de livrer leur appartient ; la qualité de l'information est la responsabilité du testeur.

SYNTHÈSE : LE COURS ET SES FILS ROUGES



Disque plein = fil rouge central de la séance ; cercle = fil rouge utilisé ponctuellement

LE COURS EN SIX IDÉES

1 La machine calcule faux

Débordements, trous entre flottants, conditionnement : les tolérances se **justifient**.

2 Tout test a un oracle

DE, g, O, RA, Tol, N. Sans oracle indépendant, pas de vérification.

3 Concevoir, pas improviser

Classes d'équivalence, limites, décisions, états, couverture, mutation.

4 Vérifier les calculs

Injection, encadrement, dimensions, perturbation, invariants.

5 Automatiser et mesurer

Pipeline, quality gates, intégration, performance, sécurité.

6 Prouver et assumer

Méthodes formelles pour le critique ; honnêteté sur le risque restant.

PROJET FINAL : CONSIGNES

PROJET

- Choisir **un fil rouge** (A à F) ; groupes de 3 ou 4
- Rédiger une **stratégie de test** basée sur les risques (2 pages)
- Concevoir les tests avec au moins **3 techniques** justifiées (classes, limites, décisions, états, métamorphique…)
- Automatiser à **au moins 3 niveaux** (unitaire, intégration ou API, E2E ou performance)
- Construire un pipeline CI avec **quality gates** chiffrés (couverture, mutation, SAST, SCA)
- Fournir un **rapport de test** final : résultats, défauts, risques résiduels, limites

Livrable	Quand
Choix du fil rouge et du groupe	S+1
Stratégie de test	S+2
Dépôt Git + pipeline vert	S+4
Rapport + soutenance (15 min)	S+5

PROJET FINAL : GRILLE D'ÉVALUATION

Critère	Ce qui est attendu	Points
Stratégie et risques	Risques identifiés et priorisés, choix de techniques justifiés	4
Conception des tests	Classes, limites, oracles et tolérances **explicites et justifiés**	5
Automatisation	Tests lisibles, isolés, rapides ; 3 niveaux couverts	4
Efficacité mesurée	Couverture et score de mutation commentés, pas seulement affichés	2
Pipeline et gates	CI reproductible, seuils versionnés et bloquants, SAST/SCA	2
Rapport et soutenance	Défauts trouvés, risques résiduels, limites honnêtes	3
Total		**20**

GLOSSAIRE (1/2)

Terme	Définition
Erreur, défaut, défaillance	Action humaine fautive → imperfection dans le produit → écart observable à l'exécution
Oracle	Mécanisme qui décide si le résultat observé est correct
Tolérance, mesure d'erreur	Écart acceptable (Tol) et façon de mesurer l'écart (N) entre obtenu et attendu
Classe d'équivalence	Sous-ensemble du domaine dont les valeurs sont supposées traitées de la même façon
Valeur limite	Valeur à la frontière d'une classe, où les défauts se concentrent
Couverture, MC/DC	Proportion d'éléments du code exercés ; MC/DC : chaque condition influence seule la décision
Mutant, score de mutation	Version du code modifiée volontairement ; part des mutants détectés par les tests
Relation métamorphique	Relation attendue entre les sorties de plusieurs exécutions liées

GLOSSAIRE (2/2)

Terme	Définition
Doublure (stub, mock, fake)	Composant de remplacement utilisé pour isoler l'objet testé
Test de contrat	Vérification que fournisseur et consommateur respectent le même contrat d'interface
SLI, SLO, SLA	Indicateur mesuré, objectif interne, engagement contractuel
Quality gate	Critère chiffré qui bloque le pipeline s'il n'est pas atteint
SAST, DAST, SCA	Analyse de sécurité du code, de l'application en marche, des dépendances
Fuzzing	Génération massive d'entrées pour provoquer des défaillances, oracle implicite
Modélisation des menaces	Analyse des menaces sur l'architecture (DFD, STRIDE) avant le codage
Risque résiduel	Risque qui subsiste après les tests ; à communiquer aux décideurs

MERCI

328

EVARIS FOMEKONG